



ফ্লাটার সম্পূর্ণ শিক্ষা বই

শূন্য থেকে চাকরির প্রস্তুতি পর্যন্ত — একটি পূর্ণাঙ্গ বাংলা রেফারেন্স

Dart সম্পূর্ণ

Flutter সম্পূর্ণ

৬টি প্রজেক্ট

১১০+ ইন্টারভিউ প্রশ্ন

Firebase

Architecture

১৫ দিনের রোডম্যাপের সঙ্গী পূর্ণাঙ্গ পাঠ্যবই

সূচিপত্র

পর্ব ১ — ভিত্তি ও প্রস্তুতি

- অধ্যায় ১ ফ্লাটার পরিচিতি ও অভ্যন্তরীণ কাঠামো
- অধ্যায় ২ পরিবেশ সেটআপ ও টুলিং

পর্ব ২ — Dart ভাষা সম্পূর্ণ

- অধ্যায় ৩ ভ্যারিয়েবল, টাইপ ও Null Safety
- অধ্যায় ৪ অপারেটর, কন্ডিশন ও লুপ
- অধ্যায় ৫ ফাংশন ও ক্লোজার
- অধ্যায় ৬ Collection — List, Set, Map
- অধ্যায় ৭ OOP — Class থেকে Mixin
- অধ্যায় ৮ Generic, Enum, Extension ও Exception
- অধ্যায় ৯ Async, Future, Stream ও Isolate

পর্ব ৩ — ফ্লাটার মৌলিক

- অধ্যায় ১০ Widget, Element ও RenderObject
- অধ্যায় ১১ Stateless, Stateful ও Lifecycle
- অধ্যায় ১২ BuildContext, Key ও Constraint

পর্ব ৪ — UI নির্মাণ

- অধ্যায় ১৩ মৌলিক Widget সমূহ
- অধ্যায় ১৪ লেআউট Widget সমূহ
- অধ্যায় ১৫ স্ক্রলিং, ListView ও Sliver
- অধ্যায় ১৬ নেভিগেশন ও রাউটিং
- অধ্যায় ১৭ ফর্ম, ইনপুট ও ভ্যালিডেশন
- অধ্যায় ১৮ থিম, স্টাইল ও রেসপন্সিভ ডিজাইন
- অধ্যায় ১৯ অ্যানিমেশন ও CustomPainter

পর্ব ৫ — ডেটা ও স্টেট

- অধ্যায় ২০ নেটওয়ার্কিং, REST API ও JSON
- অধ্যায় ২১ স্টেট ম্যানেজমেন্ট সম্পূর্ণ
- অধ্যায় ২২ লোকাল স্টোরেজ ও ডেটাবেজ
- অধ্যায় ২৩ Firebase সম্পূর্ণ
- অধ্যায় ২৪ ডিভাইস ফিচার ও পারমিশন

পর্ব ৬ — পেশাদার ডেভেলপমেন্ট

- অধ্যায় ২৫ আর্কিটেকচার ও ক্লিন কোড
- অধ্যায় ২৬ টেস্টিং
- অধ্যায় ২৭ পারফরম্যান্স ও ডিবাগিং
- অধ্যায় ২৮ ডিপ্লয়মেন্ট ও CI/CD

পর্ব ৭ — প্রজেক্ট

- প্রজেক্ট ১ নোট অ্যাপ (SQLite + CRUD)
- প্রজেক্ট ২ ই-কমার্স অ্যাপ (API + Provider + Cart)
- প্রজেক্ট ৩ আবহাওয়া অ্যাপ (GPS + API)
- প্রজেক্ট ৪ চ্যাট অ্যাপ (Firebase Realtime)
- প্রজেক্ট ৫ খরচের হিসাব (Chart + Local DB)
- প্রজেক্ট ৬ অ্যাটেনডেন্স অ্যাপ (GPS + Camera)

পর্ব ৮ — ইন্টারভিউ প্রস্তুতি

- অধ্যায় ২৯ Dart ইন্টারভিউ প্রশ্ন (৪০টি)
- অধ্যায় ৩০ Flutter ইন্টারভিউ প্রশ্ন (৪৫টি)
- অধ্যায় ৩১ আর্কিটেকচার ও অ্যাডভান্স প্রশ্ন (২০টি)
- অধ্যায় ৩২ কোডিং সমস্যা ও লাইভ টাস্ক
- অধ্যায় ৩৩ সিভি, পোর্টফোলিও ও চাকরির কৌশল

পরিশিষ্ট

- ক Widget ও কমান্ড চিটশিট
- খ এরর অভিধান
- গ প্যাকেজ তালিকা ও রিসোর্স

এই বইটি কীভাবে পড়বেন

এটি **রেফারেন্স বই** — একবারে প্রথম থেকে শেষ পর্যন্ত পড়ার দরকার নেই। ১৫ দিনের রোডম্যাপ অনুসরণ করুন, আর প্রতিদিন যে বিষয় পড়ছেন সেই অধ্যায়টি এই বই থেকে গভীরভাবে দেখে নিন। ইন্টারভিউয়ের আগে পর্ব ৮ পড়ুন। প্রতিটি কোড উদাহরণ চালিয়ে দেখুন — না চালালে শেখা হবে না।

পর্ব ১

ভিত্তি ও প্রস্তুতি

ফ্লাটার ভিতরে কীভাবে কাজ করে, কেন এত দ্রুত, এবং কাজ শুরু করার জন্য পরিবেশ কীভাবে সাজাবেন।

অধ্যায় ১ — ফ্লটার পরিচিতি ও অভ্যন্তরীণ কাঠামো

১.১ ফ্লটার আসলে কী

Flutter হলো Google-এর তৈরি একটি ওপেন সোর্স **UI SDK**। এটি শুধু একটি লাইব্রেরি নয় — এটি একটি সম্পূর্ণ ফ্রেমওয়ার্ক যার নিজস্ব রেন্ডারিং ইঞ্জিন আছে। এই একটি বৈশিষ্ট্যই Flutter-কে React Native বা Xamarin থেকে মৌলিকভাবে আলাদা করে দেয়।

১.২ Flutter কীভাবে অন্যদের থেকে আলাদা

পদ্ধতি	কীভাবে কাজ করে	সমস্যা
নেটিভ (Kotlin/Swift)	প্রতিটি প্ল্যাটফর্মের নিজস্ব ভাষা ও UI টুল	দুটি আলাদা কোডবেস, দ্বিগুণ খরচ ও সময়
React Native	JavaScript লিখে নেটিভ widget -কে নিয়ন্ত্রণ করে (bridge-এর মাধ্যমে)	bridge-এ দেরি হয়, প্ল্যাটফর্মভেদে চেহারা বদলায়
WebView (Cordova/Ionic)	অ্যাপের ভিতরে একটি ব্রাউজার চালিয়ে ওয়েবসাইট দেখায়	ধীর, নেটিভ অনুভূতি পাওয়া যায় না
Flutter	নিজের ইঞ্জিন (Skia/Impeller) দিয়ে প্রতিটি পিক্সেল নিজে আঁকে	অ্যাপের সাইজ একটু বড় হয় (~৫-৭ MB বেশি)

মূল ধারণা: Flutter একটি "গেম ইঞ্জিনের" মতো

Flutter নেটিভ বাটন ব্যবহার করে না — এটি একটি ফাঁকা ক্যানভাস নেয় এবং তার উপর নিজে বাটন আঁকে দেয়। তাই একই কোড Android ও iOS-এ **হুবহু একরকম** দেখায়। এটি সুবিধাও, আবার এর মানে হলো নতুন OS ফিচার আসতে Flutter-এ একটু দেরি হতে পারে।

১.৩ Flutter-এর স্থাপত্য (Architecture) — তিনটি স্তর

```
১. Framework স্তর (Dart ভাষায় লেখা)
  Material / Cupertino ← আপনি এখানে কাজ করেন|
  Widgets
  Rendering (layout ও painting)
  Foundation (মৌলিক ক্লাস)

২. Engine স্তর (C/C++ ভাষায় লেখা)
  Skia / Impeller → গ্রাফিক্স আঁকা
  Dart Runtime → কোড চালানো
```

Text layout → লেখা সাজানো

৩. Embedder স্তর (প্রতি প্ল্যাটফর্মে আলাদা) |
Android (Java/Kotlin) · iOS (ObjC/Swift) |
Windows · macOS · Linux · Web

১.৪ Dart কেন বেছে নেওয়া হলো

- **দুই ধরনের কম্পাইল** — ডেভেলপমেন্টে **JIT** (Just-In-Time) যা Hot Reload সম্ভব করে; রিলিজে **AOT** (Ahead-Of-Time) যা নেটিভ মেশিন কোড বানিয়ে অ্যাপ দ্রুত করে।
- **Sound Null Safety** — কম্পাইল টাইমেই null-এর ভুল ধরা পড়ে।
- **একক থ্রেড + Event Loop** — lock ও race condition-এর ঝামেলা নেই।
- **দ্রুত মেমোরি বরাদ্দ** — Flutter সেকেন্ডে হাজারো অবজেক্ট বানায় ও ফেলে দেয়; Dart-এর generational garbage collector এই কাজে বিশেষভাবে উপযোগী।

ইন্টারভিউতে খুব আসে

প্রশ্ন: Hot Reload কীভাবে কাজ করে?

উত্তর: পরিবর্তিত Dart কোড JIT-এ কম্পাইল হয়ে চলমান Dart VM-এ ইনজেক্ট হয়, তারপর widget tree আবার build হয় — কিন্তু **State অবজেক্টগুলো বেঁচে থাকে**। তাই ফর্মের ডেটা বা কাউন্টারের মান মুছে যায় না। কিন্তু `main()`, `initState()`, global variable বা enum/class-এর গঠন বদলালে Hot Reload যথেষ্ট নয় — **Hot Restart** লাগে।

১.৫ Flutter দিয়ে কী কী বানানো যায় — সীমাবদ্ধতাসহ

উপযুক্ত	কম উপযুক্ত
ই-কমার্স, ডেলিভারি, ব্যাংকিং, সোশ্যাল, ড্যাশবোর্ড, ফর্ম-ভিত্তিক ব্যবসায়িক অ্যাপ, MVP, স্টার্টআপ প্রোডাক্ট	ভারী 3D গেম (Unity ভালো), খুব গভীর নেটিভ ইন্টিগ্রেশন (AR/ML নেটিভ SDK), অ্যাপের সাইজ যেখানে ২ MB-এর নিচে রাখতেই হবে

অধ্যায় ২ — পরিবেশ সেটআপ ও টুলিং

২.১ ইনস্টলেশন যাচাই

```
flutter --version      # সংস্করণ দেখুন
flutter doctor -v      # বিস্তারিত সমস্যা তালিকা
flutter doctor --android-licenses # লাইসেন্স মেনে নিন
```

২.২ চ্যানেল — কোনটি ব্যবহার করবেন

```
flutter channel      # বর্তমান চ্যানেল
flutter channel stable # ★ সবসময় stable ব্যবহার করুন
flutter upgrade
```

stable — পরীক্ষিত, প্রোডাকশনের জন্য। **beta** — নতুন ফিচার, কিছু বাগ থাকতে পারে। **master** — সর্বশেষ কোড, প্রায়ই ভাঙে। নতুনরা শুধু **stable** ব্যবহার করবেন।

২.৩ প্রজেক্ট তৈরির সব অপশন

```
# সাধারণ
flutter create my_app

# কোম্পানির ডোমেইনসহ (Play Store-এ package name হবে com.mycompany.my_app)
flutter create --org com.mycompany my_app

# শুধু নির্দিষ্ট প্ল্যাটফর্ম
flutter create --platforms=android,ios my_app

# প্যাকেজ/লাইব্রেরি বানাতে
flutter create --template=package my_package

# নেটিভ কোডসহ প্লাগইন
flutter create --template=plugin my_plugin
```

২.৪ pubspec.yaml — সম্পূর্ণ ব্যাখ্যা

```
name: my_app          # প্যাকেজের নাম (snake_case)
description: আমার প্রথম অ্যাপ
publish_to: 'none'     # pub.dev-এ প্রকাশ করব না
version: 1.0.0+1       # versionName+versionCode

environment:
  sdk: '>=3.0.0 <4.0.0' # কোন Dart সংস্করণে চলবে
```

```
dependencies:
  flutter:
    sdk: flutter
  http: ^1.2.0          # ^ = 1.2.0 থেকে 2.0.0-এর আগ পর্যন্ত
  provider: ^6.1.0
  intl: 0.18.1          # ^ ছাড়া = ঠিক এই সংস্করণ

dev_dependencies:      # শুধু ডেভেলপমেন্টে লাগে, অ্যাপে যায় না
  flutter_test:
    sdk: flutter
  flutter_lints: ^3.0.0

flutter:
  uses-material-design: true

assets:
  - assets/images/      # পুরো ফোল্ডার
  - assets/data/cities.json # একক ফাইল

fonts:
  - family: HindSiliguri
    fonts:
      - asset: assets/fonts/HindSiliguri-Regular.ttf
      - asset: assets/fonts/HindSiliguri-Bold.ttf
        weight: 700
      - asset: assets/fonts/HindSiliguri-Light.ttf
        weight: 300
```

YAML-এর তিনটি কঠোর নিয়ম

১. Tab ব্যবহার করা যাবে না — শুধু স্পেস।
২. ইন্ডেন্টেশন সবসময় ২ স্পেস, এবং একই স্তরের সব লাইনে সমান।
৩. কোলনের পরে একটি স্পেস — `name: my_app` , `name:my_app` নয়।

২.৫ সংস্করণ নির্দেশের অর্থ

লেখা	অর্থ
<code>^1.2.3</code>	$1.2.3 \leq \text{সংস্করণ} < 2.0.0$ (সবচেয়ে বেশি ব্যবহৃত ও নিরাপদ)
<code>1.2.3</code>	ঠিক এই সংস্করণ, আর কিছু নয়
<code>>=1.2.0</code> <code><2.0.0</code>	নির্দিষ্ট পরিসর
<code>any</code>	যেকোনো (ব্যবহার করবেন না — ভাঙার ঝুঁকি)

২.৬ অপরিহার্য টার্মিনাল কমান্ড

```
flutter run # ডিবাগ মোডে চালানো
flutter run --release # রিলিজ মোডে (দ্রুত, ডিবাগ টুল নেই)
flutter run --profile # পারফরম্যান্স মাপার মোড
flutter run -d chrome # নির্দিষ্ট ডিভাইসে
flutter run --dart-define=API_URL=https://x.com # এনভায়রনমেন্ট ভ্যারিয়েবল

flutter devices # যুক্ত ডিভাইস
flutter emulators # ইমুলেটর তালিকা
flutter emulators --launch NAME # ইমুলেটর চালু

flutter pub add http # প্যাকেজ যোগ
flutter pub remove http # প্যাকেজ বাদ
flutter pub get # নির্ভরতা ডাউনলোড
flutter pub upgrade # আপডেট
flutter pub outdated # কোনগুলোর নতুন সংস্করণ আছে

flutter analyze # স্ট্যাটিক বিশ্লেষণ
dart format . # কোড ফরম্যাট
dart fix --apply # স্বয়ংক্রিয় সংশোধন

flutter clean # বিন্দু ক্যাশ মুছে ফেলা
flutter test # টেস্ট চালানো
flutter build apk --release # রিলিজ বিন্দু
```

২.৭ তিনটি বিন্দু মোড

মোড	কম্পাইল	কখন ব্যবহার
Debug	JIT	ডেভেলপমেন্ট। Hot Reload ও DevTools চলে। অ্যাপ ধীর — এটি দিয়ে গতি বিচার করবেন না।
Profile	AOT	পারফরম্যান্স মাপা। রিলিজের মতো দ্রুত, কিন্তু মাপার টুল চালু থাকে।
Release	AOT	ব্যবহারকারীর জন্য। সবচেয়ে দ্রুত ও ছোট। assert ও print বাদ পড়ে।

VS Code-এর অপরিহার্য এক্সটেনশন ও শর্টকাট

এক্সটেনশন: **Flutter, Dart, Awesome Flutter Snippets, Error Lens**।

শর্টকাট: **Ctrl+.** (কোড অ্যাকশন — widget মোড়ানো/সরানোর সবচেয়ে দরকারি টুল) · **Alt+Shift+F** (ফরম্যাট) · **stless/stful** (স্নিপেট) · **F5** (ডিবাগ চালু) · **Ctrl+Shift+P** → **Flutter: Open DevTools**।

২.৮ লিন্ট নিয়ম কঠোর করা (পেশাদার অভ্যাস)

analysis_options.yaml ফাইলে:

```
include: package:flutter_lints/flutter.yaml

linter:
  rules:
    - prefer_const_constructors
    - prefer_final_locals
    - avoid_print
    - always_declare_return_types
    - require_trailing_commas

analyzer:
  errors:
    invalid_annotation_target: ignore
  exclude:
    - "**/*.g.dart"          # জেনারেট হওয়া ফাইল বাদ
```

পর্ব ২

Dart ভাষা সম্পূর্ণ

ভ্যারিয়েবল থেকে Isolate পর্যন্ত — Flutter-এ যা যা লাগে তার
সবটুকু, উদাহরণসহ।

অধ্যায় ৩ — ভ্যারিয়েবল, টাইপ ও Null Safety

৩.১ Dart-এর টাইপ ব্যবস্থা

```
void main() {  
  int age = 25;           // পূর্ণসংখ্যা  
  double price = 99.50;   // দশমিক  
  num anything = 10;      // int ও double দুটোরই বাবা  
  String name = 'রহিম';   // লেখা  
  bool isActive = true;   // সত্য/মিথ্যা  
  
  List<int> numbers = [1, 2, 3];  
  Set<String> tags = {'নতুন', 'জনপ্রিয়'};  
  Map<String, int> scores = {'রহিম': 90};  
  
  dynamic x = 5;          // টাইপ চেক বন্ধ – এড়িয়ে চলুন  
  x = 'এখন আমি String'; // অনুমোদিত, কিন্তু বিপজ্জনক  
  
  Object o = 5;           // dynamic এর নিরাপদ বিকল্প  
  
  var inferred = 'Dart নিজেই বুঝে নিল'; // String  
  final fixed = 'একবার সেট';  
  const compileTime = 3.1416;  
}
```

dynamic কেন ব্যবহার করবেন না

`dynamic` ব্যবহার করলে Dart-এর টাইপ চেক বন্ধ হয়ে যায় — ভুল কম্পাইল টাইমে ধরা না পড়ে রানটাইমে অ্যাপ ক্র্যাশ করায়। JSON পার্স করার সময় বাধ্য হয়ে ব্যবহার করলেও সাথে সাথে মডেল ক্লাসে রূপান্তর করে ফেলুন।

৩.২ final বনাম const — গভীর পার্থক্য

```
void main() {  
  // final – রানটাইমে মান নির্ধারিত হয়  
  final now = DateTime.now();           // ✓ চলবে  
  
  // const – কম্পাইল টাইমেই মান জানা থাকতে হবে  
  // const now2 = DateTime.now();       X এরর  
  const pi = 3.1416;                   // ✓  
  
  // const লিস্ট পরিবর্তন করা যায় না  
  const list1 = [1, 2, 3];  
  // list1.add(4);                       X রানটাইম এরর  
  
  final list2 = [1, 2, 3];
```

```
list2.add(4); // ✓ ভিতরের মান বদলানো যায়
// list2 = [5, 6]; X পুরো ভ্যারিয়েবল বদলানো যায় না
}
```

Flutter-এ const কেন এত গুরুত্বপূর্ণ

`const` widget প্রতিবার rebuild-এ নতুন করে তৈরি হয় না — Flutter একই অবজেক্ট পুনঃব্যবহার করে। বড় অ্যাপে এটি উল্লেখযোগ্য পারফরম্যান্স পার্থক্য গড়ে দেয়। **যেখানেই সম্ভব `const` লিখুন।**

৩.৩ Null Safety সম্পূর্ণ

```
void main() {
  String a = 'নিশ্চিত মান'; // non-nullable – null হতে পারবে না
  String? b; // nullable – null হতে পারে

  // ১. ?? – null হলে বিকল্প মান
  print(b ?? 'কিছু নেই');

  // ২. ??= – null হলে তবেই সেট করো
  b ??= 'নতুন মান';

  // ৩. ?. – null না হলে তবেই কল করো
  print(b?.length);
  print(b?.toUpperCase() ?? 'খালি');

  // ৪. ! – bang operator, "আমি নিশ্চিত null নয়"
  String c = b!; // b null হলে ক্র্যাশ করবে

  // ৫. late – পরে সেট করব, কিন্তু ব্যবহারের আগে অবশ্যই
  late String description;
  description = 'পরে দিলাম';
  print(description);
}
```

late কখন ব্যবহার করবেন

```
class MyWidget extends StatefulWidget { /* ... */ }

class _MyWidgetState extends State<MyWidget> {
  // initState-এ সেট করব, কিন্তু nullable বানাতে চাই না
  late final TextEditingController controller;
  late Future<List<Product>> productsFuture;

  @override
  void initState() {
    super.initState();
    controller = TextEditingController();
    productsFuture = ApiService.getProducts();
  }
}
```

```
}  
}
```

late-এর বিপদ

সেট করার আগে ব্যবহার করলে **LateInitializationError** — অ্যাপ ক্র্যাশ। `late` ব্যবহারের আগে নিশ্চিত হোন যে সেট করার কোড অবশ্যই আগে চলবে।

৩.৪ String নিয়ে কাজ

```
void main() {  
    String name = 'রহিম উদ্দিন';  
  
    // Interpolation  
    print('নাম: $name');  
    print('অক্ষর সংখ্যা: ${name.length}');  
  
    // দরকারি মেথড  
    print(name.toUpperCase());  
    print(name.trim());           // আগে-পরের স্পেস বাদ  
    print(name.contains('উদ্দিন')); // true  
    print(name.replaceAll('রহিম', 'করিম'));  
    print(name.split(' '));       // [রহিম, উদ্দিন]  
    print(name.substring(0, 4));  
    print(name.startsWith('র'));  
    print(name.isEmpty());  
    print(name.padLeft(15, '*'));  
  
    // একাধিক লাইন  
    String multi = '''  
প্রথম লাইন  
দ্বিতীয় লাইন  
''';  
  
    // Raw string - \n কে সাধারণ অক্ষর হিসেবে ধরবে  
    String path = r'C:\Users\HP\file.txt';  
  
    // রূপান্তর  
    int n = int.parse('42');  
    double d = double.parse('3.14');  
    int? safe = int.tryParse('abc'); // null, ক্র্যাশ করবে না ★  
    String s = 42.toString();  
    String f = 3.14159.toStringAsFixed(2); // "3.14"  
}
```

সবসময় `tryParse` ব্যবহার করুন

ব্যবহারকারীর ইনপুট থেকে সংখ্যা নিতে `int.parse()` ব্যবহার করলে ভুল ইনপুটে অ্যাপ ক্র্যাশ করবে।
`int.tryParse()` ব্যর্থ হলে `null` দেয় — নিরাপদ।

অধ্যায় ৪ — অপারেটর, কন্ডিশন ও লুপ

৪.১ সব অপারেটর

ধরন	অপারেটর
গাণিতিক	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>~/</code> <code>%</code> <code>—</code> <code>~/</code> পূর্ণসংখ্যা ভাগ, <code>%</code> ভাগশেষ
বৃদ্ধি/হ্রাস	<code>++</code> <code>--</code> (আগে বা পরে বসানো যায়)
তুলনা	<code>==</code> <code>!=</code> <code>></code> <code><</code> <code>>=</code> <code><=</code>
লজিক্যাল	<code>&&</code> <code> </code> <code>!</code>
অ্যাসাইন	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>??=</code>
টাইপ	<code>is</code> <code>·</code> <code>is!</code> <code>·</code> <code>as</code>
null	<code>??</code> <code>?.</code> <code>!</code>
ক্যাসকেড	<code>..</code> — একই অবজেক্টে পরপর কাজ

```
void main() {
  // টাইপ চেক ও কাস্ট
  Object value = 'আমি একটি String';
  if (value is String) {
    print(value.length);          // Dart নিজেই String ধরে নেয় (smart cast)
  }

  // ক্যাসকেড – একই অবজেক্টে পরপর কাজ
  var list = <int>[]
    ..add(1)
    ..add(2)
    ..add(3);

  // এটি না লিখে
  var p = Paint();
  p.color = Colors.red;
  p.strokeWidth = 4;
  // এভাবে লিখুন
  var p2 = Paint()
    ..color = Colors.red
    ..strokeWidth = 4;
}
```


৪.২ কন্ডিশন

```
void main() {
    int marks = 78;

    if (marks >= 80) {
        print('A+');
    } else if (marks >= 70) {
        print('A');
    } else {
        print('আরও চেষ্টা');
    }

    // টার্নারি
    String r = marks >= 33 ? 'পাস' : 'ফেল';

    // নেস্টেড টার্নারি (পড়তে কঠিন, এড়িয়ে চলুন)
    String grade = marks >= 80 ? 'A+' : marks >= 70 ? 'A' : 'B';

    // switch
    String role = 'admin';
    switch (role) {
        case 'admin':
            print('সব অনুমতি');
            break;
        case 'editor':
        case 'author': // একাধিক কেস একসাথে
            print('লেখার অনুমতি');
            break;
        default:
            print('শুধু পড়ার অনুমতি');
    }

    // Dart 3-এর নতুন switch expression (অনেক পরিষ্কার)
    final message = switch (role) {
        'admin' => 'সব অনুমতি',
        'editor' || 'author' => 'লেখার অনুমতি',
        _ => 'শুধু পড়ার অনুমতি',
    };
    print(message);
}
```

৪.৩ সব ধরনের লুপ

```
void main() {
    // ১. সাধারণ for
    for (int i = 0; i < 5; i++) print(i);

    // ২. for-in
    final fruits = ['আম', 'জাম', 'কাঁঠাল'];
    for (final f in fruits) print(f);
}
```

```
// ৩. ইন্ডেক্সসহ for-in
for (final (i, f) in fruits.indexed) { // Dart 3
  print('$i → $f');
}

// ৪. forEach
fruits.forEach(print);

// ৫. while
int n = 3;
while (n > 0) { print(n); n--; }

// ৬. do-while – অন্তত একবার চলবে
do { print('একবার হলেও'); } while (false);

// break ও continue
for (int i = 1; i <= 10; i++) {
  if (i % 2 == 0) continue; // জোড় বাদ
  if (i > 7) break; // ৭-এর পরে থামো
  print(i); // 1 3 5 7
}

// লেবেলযুক্ত break – ন্যেস্টেড লুপ থেকে বের হতে
outer:
for (int i = 0; i < 3; i++) {
  for (int j = 0; j < 3; j++) {
    if (j == 2) break outer;
    print('$i-$j');
  }
}
}
```

Flutter-এ collection-if ও collection-for

UI বানানোর সময় এটি প্রতিদিন লাগবে:

```
Column(
  children: [
    const Text('শিরোনাম'),
    if (isLoggedIn) const Text('স্বাগতম'),
    if (items.isEmpty)
      const Text('খালি')
    else
      for (final item in items) ListTile(title: Text(item)),
    ...extraWidgets, // spread operator
    ...?maybeNullList, // null-aware spread
  ],
)
```

অধ্যায় ৫ — ফাংশন ও ক্লোজার

৫.১ ফাংশনের সব রূপ

```
// সাধারণ
int add(int a, int b) { return a + b; }

// অ্যারো (এক এক্সপ্রেশন হলে)
int multiply(int a, int b) => a * b;

// কিছু ফেরত দেয় না
void log(String msg) => print('[LOG] $msg');

// Optional positional - [] দিয়ে
String greet(String name, [String? title]) {
  return title == null ? 'হ্যালো $name' : 'হ্যালো $title $name';
}

// Named parameter - {} দিয়ে (Flutter-এর স্ট্যান্ডার্ড)
void createUser({
  required String name,           // অবশ্যই দিতে হবে
  int age = 18,                   // ডিফল্ট মান
  String? email,                  // ঐচ্ছিক, null হতে পারে
}) {
  print('$name, $age, ${email ?? "ইমেইল নেই"}');
}

void main() {
  print(add(2, 3));
  print(greet('রহিম', 'জনাব'));
  createUser(name: 'করিম', age: 25);
  createUser(name: 'সালমা');      // age ডিফল্ট ১৮
}
```

৫.২ ফাংশনও একটি মান (First-class function)

```
void main() {
  // ফাংশনকে ভ্যারিয়েবলে রাখা
  int Function(int, int) operation = (a, b) => a + b;
  print(operation(3, 4));

  // ফাংশনকে আর্গুমেন্ট হিসেবে পাঠানো
  void calculate(int a, int b, int Function(int, int) op) {
    print('ফলাফল: ${op(a, b)}');
  }
  calculate(6, 3, (a, b) => a - b);

  // ফাংশন থেকে ফাংশন ফেরত (ক্লোজার)
  Function makeMultiplier(int factor) {
```

```

    return (int x) => x * factor;    // factor মনে রাখে
}
final triple = makeMultiplier(3);
print(triple(5));                  // 15

// typedef দিয়ে পড়তে সহজ করা
// typedef Validator = String? Function(String? value);
}

```

Flutter-এ callback কীভাবে কাজে লাগে

```

class ProductCard extends StatelessWidget {
  final Product product;
  final VoidCallback onTap;          // void Function()
  final ValueChanged<int> onQuantityChange; // void Function(int)

  const ProductCard({
    super.key,
    required this.product,
    required this.onTap,
    required this.onQuantityChange,
  });

  @override
  Widget build(BuildContext context) {
    return InkWell(onTap: onTap, child: /* ... */);
  }
}

```

এভাবে child widget প্যারেন্টকে "জানিয়ে দেয়" কী ঘটেছে — এটি Flutter-এর সবচেয়ে গুরুত্বপূর্ণ প্যাটার্নগুলোর একটি।

৫.৩ Anonymous ফাংশন ও Arrow

```

final numbers = [1, 2, 3, 4, 5];

// পূর্ণ রূপ
numbers.forEach((int n) {
  print(n * 2);
});

// সংক্ষিপ্ত
numbers.forEach((n) => print(n * 2));

// আরও সংক্ষিপ্ত (tear-off)
numbers.map((n) => n * 2).forEach(print);

```

অধ্যায় ৬ — Collection — List, Set, Map

৬.১ List সম্পূর্ণ

```
void main() {
    final nums = [5, 2, 8, 1, 9];

    // পড়া
    print(nums[0]); print(nums.first); print(nums.last);
    print(nums.length); print(nums.isEmpty); print(nums.isNotEmpty);
    print(nums.reversed.toList());

    // পরিবর্তন
    nums.add(10);
    nums.addAll([11, 12]);
    nums.insert(0, 0);
    nums.remove(8);           // মান দিয়ে
    nums.removeAt(0);         // ইনডেক্স দিয়ে
    nums.removeWhere((n) => n > 10);
    nums.clear();
}
```

রূপান্তর ও ফিল্টার — সবচেয়ে বেশি ব্যবহৃত

```
final products = [
    Product('ঘড়ি', 2500), Product('ব্যাগ', 1200), Product('জুতা', 3500),
];

// map – প্রতিটিকে বদলে নতুন লিস্ট
final names = products.map((p) => p.name).toList();

// where – শর্তে মেলা আইটেম
final expensive = products.where((p) => p.price > 2000).toList();

// firstWhere – প্রথম মিল (না পেলে এরর!)
final found = products.firstWhere(
    (p) => p.name == 'ব্যাগ',
    orElse: () => Product('নেই', 0), // ★ সবসময় দিন
);

// any / every
print(products.any((p) => p.price > 3000)); // একটিও আছে?
print(products.every((p) => p.price > 1000)); // সবগুলোই?

// fold / reduce – সব মিলিয়ে একটি মান
final total = products.fold<double>(0, (sum, p) => sum + p.price);
final maxPrice = products.map((p) => p.price).reduce((a, b) => a > b ? a : b);

// sort
```

```

products.sort((a, b) => a.price.compareTo(b.price)); // ছোট → বড়
products.sort((a, b) => b.price.compareTo(a.price)); // বড় → ছোট
products.sort((a, b) => a.name.compareTo(b.name)); // নাম অনুযায়ী

// অন্যান্য
print(products.take(2).toList()); // প্রথম ২টি
print(products.skip(1).toList()); // প্রথম ১টি বাদ
print(products.expand((p) => [p, p]).toList()); // সমতল করা

// গ্রুপ করা
final Map<String, List<Product>> byCategory = {};
for (final p in products) {
  byCategory.putIfAbsent(p.category, () => []).add(p);
}

```

firstWhere-এর ফাঁদ

`firstWhere` কিছু না পেলে **StateError** ছুড়ে অ্যাপ ক্র্যাশ করায়। সবসময় `orElse` দিন, অথবা `collection` প্যাকেজের `firstWhereOrNull` ব্যবহার করুন যা null দেয়।

৬.২ Set — অনন্য মানের সংগ্রহ

```

void main() {
  final a = {1, 2, 3, 4};
  final b = {3, 4, 5, 6};

  print(a.union(b)); // {1,2,3,4,5,6}
  print(a.intersection(b)); // {3,4}
  print(a.difference(b)); // {1,2}
  print(a.contains(2)); // true – List এর চেয়ে অনেক দ্রুত

  // List থেকে ডুপ্লিকেট বাদ দেওয়ার সহজ উপায়
  final withDup = [1, 2, 2, 3, 3, 3];
  final unique = withDup.toSet().toList(); // [1, 2, 3]
}

```

৬.৩ Map সম্পূর্ণ

```

void main() {
  final user = <String, dynamic>{
    'name': 'রহিম', 'age': 25, 'city': 'ঢাকা',
  };

  print(user['name']);
  print(user['unknown']); // null (এরর নয়)

  user['email'] = 'r@mail.com';
  user.remove('city');
}

```

```

user.putIfAbsent('role', () => 'user');
user.update('age', (v) => v + 1);
user.addAll({'phone': '017...'});

print(user.keys.toList());
print(user.values.toList());
print(user.containsKey('name'));
print(user.length);

// লুপ
user.forEach((k, v) => print('$k: $v'));
for (final entry in user.entries) {
  print('${entry.key} = ${entry.value}');
}

// রূপান্তর
final upper = user.map((k, v) => MapEntry(k.toUpperCase(), v));
final filtered = Map.fromEntries(
  user.entries.where((e) => e.value != null),
);
}

```

List বনাম Set বনাম Map — কোনটি কখন

List — ক্রম গুরুত্বপূর্ণ, ডুপ্লিকেট চলবে (পণ্যের তালিকা)।

Set — ডুপ্লিকেট চলবে না, "আছে কিনা" দ্রুত জানা দরকার (নির্বাচিত ID)।

Map — কী দিয়ে খুঁজতে হবে (id → পণ্য, সেটিংস)।

অধ্যায় ৭ — OOP — Class থেকে Mixin

৭.১ Class ও Constructor-এর সব রূপ

```
class Person {
    // ফিল্ড
    final String name;
    final int age;
    String? email;
    static int totalPersons = 0;    // ক্লাসের নিজস্ব, অবজেক্টের নয়

    // ১. সাধারণ কনস্ট্রাক্টর
    Person(this.name, this.age) {
        totalPersons++;
    }

    // ২. Named কনস্ট্রাক্টর
    Person.guest() : name = 'অতিথি', age = 0;

    Person.fromMap(Map<String, dynamic> map)
        : name = map['name'] ?? '',
          age = map['age'] ?? 0;

    // ৩. const কনস্ট্রাক্টর (সব ফিল্ড final হতে হবে)
    // const Person(this.name, this.age);

    // ৪. Factory – নতুন অবজেক্ট না বানিয়েও ফেরত দিতে পারে
    factory Person.fromJson(Map<String, dynamic> json) {
        return Person(json['name'], json['age']);
    }

    // মেথড
    void introduce() => print('আমি $name');

    // Getter – হিসাব করা প্রপার্টি
    bool get isAdult => age >= 18;
    String get initial => name.isEmpty ? '?' : name[0];

    // Setter
    set updateEmail(String value) {
        if (value.contains('@')) email = value;
    }

    // Static মেথড – অবজেক্ট ছাড়াই কল করা যায়
    static Person createDefault() => Person('অজানা', 0);

    // toString override – print করলে সুন্দর দেখাবে
    @override
    String toString() => 'Person(name: $name, age: $age)';
}
```


৭.২ Inheritance (উত্তরাধিকার)

```
class Vehicle {
    final String brand;
    int speed = 0;

    Vehicle(this.brand);

    void start() => print('$brand চালু হলো');
    void accelerate(int by) { speed += by; }
}

class Car extends Vehicle {
    final int doors;

    Car(String brand, this.doors) : super(brand); // বাবার কনস্ট্রাক্টর

    @override
    void start() {
        super.start(); // বাবার কাজও করাও
        print('$doors দরজার গাড়ি প্রস্তুত');
    }

    void openTrunk() => print('ডালা খুলল');
}

void main() {
    final c = Car('টয়োটা', 4);
    c.start();
    c.accelerate(60);
    print(c.speed);

    // পলিমরফিজম – Vehicle টাইপে Car রাখা যায়
    Vehicle v = Car('হোন্ডা', 2);
    v.start(); // Car এর start() চলবে
}
```

৭.৩ Abstract Class

```
// abstract ক্লাসের অবজেক্ট বানানো যায় না – শুধু extend করা যায়
abstract class Shape {
    String get name; // বাস্তবায়ন নেই – সন্তানকে দিতে হবে
    double area(); // বাস্তবায়ন নেই

    void describe() { // এটির বাস্তবায়ন আছে
        print('$name এর ক্ষেত্রফল ${area().toStringAsFixed(2)}');
    }
}

class Circle extends Shape {
    final double radius;
```

```

    Circle(this.radius);

    @override
    String get name => 'বৃত্ত';

    @override
    double area() => 3.1416 * radius * radius;
}

class Rectangle extends Shape {
    final double w, h;
    Rectangle(this.w, this.h);

    @override
    String get name => 'আয়তক্ষেত্র';

    @override
    double area() => w * h;
}

void main() {
    final shapes = <Shape>[Circle(5), Rectangle(4, 6)];
    for (final s in shapes) s.describe();
}

```

৭.৪ Interface ও implements

```

// Dart-এ আলাদা interface কীওয়ার্ড নেই – যেকোনো ক্লাসই interface
abstract class Storage {
    Future<void> save(String key, String value);
    Future<String?> read(String key);
}

// implements – সব মেথড নিজে লিখতেই হবে, কিছুই উত্তরাধিকারে পাওয়া যায় না
class LocalStorage implements Storage {
    final Map<String, String> _data = {};

    @override
    Future<void> save(String key, String value) async => _data[key] = value;

    @override
    Future<String?> read(String key) async => _data[key];
}

class CloudStorage implements Storage {
    @override
    Future<void> save(String key, String value) async { /* API কল */ }

    @override
    Future<String?> read(String key) async => null;
}

```

extends বনাম implements — ইন্টারভিউ প্রশ্ন

extends — একটিমাত্র ক্লাস থেকে; বাবার কোড উত্তরাধিকারে পাওয়া যায়; শুধু দরকারিটুকু override করলেই চলে।

implements — একাধিক ক্লাস থেকে সম্ভব; কোনো কোড পাওয়া যায় না; প্রতিটি সদস্য নিজে লিখতে হয়। শুধু চুক্তি (contract) মানার জন্য।

৭.৫ Mixin — কোড পুনঃব্যবহারের তৃতীয় উপায়

```

mixin Swimmer {
  void swim() => print('সাঁতার কাটছি');
}

mixin Flyer {
  double altitude = 0;
  void fly() => print('উড়ছি');
}

// একাধিক mixin একসাথে ব্যবহার করা যায় – extends এ যা সম্ভব নয়
class Duck with Swimmer, Flyer {
  void quack() => print('প্যাঁক প্যাঁক');
}

// নির্দিষ্ট ক্লাসেই কেবল ব্যবহারযোগ্য mixin
mixin Validator on Shape {
  bool get isValid => area() > 0;
}

void main() {
  final d = Duck()..swim()..fly()..quack();
}
```

Flutter-এ mixin-এর বাস্তব ব্যবহার

with SingleTickerProviderStateMixin — অ্যানিমেশন কন্ট্রোলারের জন্য।
with WidgetsBindingObserver — অ্যাপ ব্যাকগ্রাউন্ডে গেল কিনা জানার জন্য।
with AutomaticKeepAliveClientMixin — ট্যাব বদলালেও পেজের state ধরে রাখতে।

৭.৬ পূর্ণাঙ্গ মডেল ক্লাস — পেশাদার মান

```

class User {
  final int id;
  final String name;
  final String email;
  final String? avatar;
```

```

final DateTime createdAt;

const User({
  required this.id,
  required this.name,
  required this.email,
  this.avatar,
  required this.createdAt,
});

// JSON → অবজেক্ট
factory User.fromJson(Map<String, dynamic> json) => User(
  id: json['id'] as int,
  name: json['name'] as String? ?? '',
  email: json['email'] as String? ?? '',
  avatar: json['avatar'] as String?,
  createdAt: DateTime.tryParse(json['created_at'] ?? '') ?? DateTime.now(),
);

// অবজেক্ট → JSON
Map<String, dynamic> toJson() => {
  'id': id, 'name': name, 'email': email,
  'avatar': avatar, 'created_at': createdAt.toIso8601String(),
};

// copyWith – অপরিবর্তনীয় অবজেক্ট আপডেট করার আদর্শ উপায়
User copyWith({int? id, String? name, String? email, String? avatar}) => User(
  id: id ?? this.id,
  name: name ?? this.name,
  email: email ?? this.email,
  avatar: avatar ?? this.avatar,
  createdAt: createdAt,
);

// সমতা যাচাই – দুটি অবজেক্ট এক কিনা
@override
bool operator ==(Object other) =>
  identical(this, other) ||
  (other is User && other.id == id && other.email == email);

@override
int get hashCode => Object.hash(id, email);

@override
String toString() => 'User(id: $id, name: $name)';
}

```

== override করলে hashCode-ও করতে হবে

না করলে `Set` ও `Map` -এ অবজেক্টটি ঠিকভাবে কাজ করবে না — একই মানের দুটি অবজেক্ট আলাদা বলে গণ্য হবে। এটি ইন্টারভিউতে খুব জিজ্ঞেস করা হয়।

অধ্যায় ৮ — Generic, Enum, Extension ও Exception

৮.১ Generic — টাইপ-নিরাপদ পুনঃব্যবহার

```
// T = যেকোনো টাইপ, কল করার সময় ঠিক হবে
class Box<T> {
  final T value;
  Box(this.value);
  T get item => value;
}

void main() {
  final intBox = Box<int>(5);
  final strBox = Box<String>('হ্যালো');
  print(intBox.item + 1);          // Dart জানে এটি int
}

// জেনেরিক ফাংশন
T firstOrDefault<T>(List<T> list, T fallback) =>
  list.isEmpty ? fallback : list.first;

// সীমাবদ্ধ জেনেরিক – শুধু num বা তার সন্তান
T largest<T extends num>(List<T> items) =>
  items.reduce((a, b) => a > b ? a : b);
```

বাস্তব ব্যবহার — API রেসপন্স মোড়ক

```
class ApiResult<T> {
  final T? data;
  final String? error;
  final bool isLoading;

  const ApiResult.success(this.data) : error = null, isLoading = false;
  const ApiResult.failure(this.error) : data = null, isLoading = false;
  const ApiResult.loading() : data = null, error = null, isLoading = true;

  bool get hasError => error != null;
}

// ব্যবহার
ApiResult<List<Product>> result = const ApiResult.loading();
```

৮.২ Enum — নির্দিষ্ট কয়েকটি মান

```
// সাধারণ enum
enum Status { pending, approved, rejected }
```

```
// উন্নত enum (Dart 2.17+) – ফিন্ড ও মেথডসহ
enum OrderStatus {
  pending('অপেক্ষমাণ', 0xFFFFA000),
  shipped('পাঠানো হয়েছে', 0xFF1976D2),
  delivered('পৌঁছে গেছে', 0xFF388E3C),
  cancelled('বাতিল', 0xFFD32F2F);

  final String label;
  final int colorValue;
  const OrderStatus(this.label, this.colorValue);

  bool get isFinal => this == delivered || this == cancelled;
}

void main() {
  final s = OrderStatus.shipped;
  print(s.label);          // পাঠানো হয়েছে
  print(s.name);           // shipped
  print(s.index);          // 1
  print(OrderStatus.values); // সব মানের তালিকা

  // switch-এ enum – Dart সব কেস আছে কিনা যাচাই করে ★
  switch (s) {
    case OrderStatus.pending: print('অপেক্ষা করুন'); break;
    case OrderStatus.shipped: print('পথে আছে'); break;
    case OrderStatus.delivered: print('সম্পন্ন'); break;
    case OrderStatus.cancelled: print('বাতিল হয়েছে'); break;
  }
}
```

enum কেন String-এর চেয়ে ভালো

`status == 'shipped'` লিখলে বানান ভুল কম্পাইলার ধরবে না — বাগ থেকে যাবে। enum ব্যবহার করলে ভুল নাম কম্পাইল টাইমেই ধরা পড়ে, এবং switch-এ কোনো কেস বাদ পড়লে সতর্কবার্তা আসে।

৮.৩ Extension — বিদ্যমান ক্লাসে নতুন ক্ষমতা যোগ

```
extension StringExtension on String {
  String get capitalized =>
    isEmpty ? this : '${this[0].toUpperCase()}${substring(1)}';

  bool get isValidEmail =>
    RegExp(r'^[\w.+-]+@[ \w-]+\.[\w.]+$').hasMatch(this);

  bool get isValidBDPhone =>
    RegExp(r'^(01)[3-9]\d{8}$').hasMatch(this);
}

extension ContextExtension on BuildContext {
  double get width => MediaQuery.of(this).size.width;
```

```

double get height => MediaQuery.of(this).size.height;
ThemeData get theme => Theme.of(this);
bool get isMobile => width < 600;

void showSnack(String msg) => ScaffoldMessenger.of(this)
    .showSnackBar(SnackBar(content: Text(msg)));
}

extension DateExtension on DateTime {
  String get formatted => '$day/$month/$year';
  bool get isToday {
    final now = DateTime.now();
    return year == now.year && month == now.month && day == now.day;
  }
}

// ব্যবহার – অসাধারণ পরিচ্ছন্ন কোড
void main() {
  print('rahim@mail.com'.isValidEmail); // true
  // context.showSnack('সেভ হয়েছে');
  // if (context.isMobile) ...
}

```

৮.৪ Exception ও Error হ্যান্ডলিং

```

// নিজের Exception ক্লাস
class NetworkException implements Exception {
  final String message;
  final int? statusCode;
  NetworkException(this.message, [this.statusCode]);

  @override
  String toString() => 'NetworkException: $message';
}

class ValidationException implements Exception {
  final Map<String, String> errors;
  ValidationException(this.errors);
}

Future<void> loadData() async {
  try {
    final data = await fetchFromServer();
    process(data);
  } on NetworkException catch (e) {
    // নির্দিষ্ট ধরনের এরর
    print('নেটওয়ার্ক সমস্যা: ${e.message}');
  } on FormatException catch (e) {
    print('ডেটার গঠন ভুল: $e');
  }
}

```

```

    } catch (e, stackTrace) {
        // অন্য সব এরর
        print('অজানা সমস্যা: $e');
        print(stackTrace);

    } finally {
        // এরর হোক বা না হোক – সবসময় চলবে
        print('লোডিং শেষ');
    }
}

// এরর ছুড়ে দেওয়া
void withdraw(double amount, double balance) {
    if (amount <= 0) {
        throw ArgumentError('পরিমাণ ধনাত্মক হতে হবে');
    }
    if (amount > balance) {
        throw NetworkException('পর্যাপ্ত ব্যালেন্স নেই');
    }
}

```

Exception বনাম Error

Exception — প্রত্যাশিত সমস্যা যা ধরে সামলানো উচিত (নেট নেই, ফাইল নেই, ভুল ইনপুট)।

Error — প্রোগ্রামারের ভুল যা ধরা উচিত নয়, ঠিক করা উচিত (null check ব্যর্থ, ইনডেক্স সীমার বাইরে)।

নিজের ক্লাস বানাতে সবসময় `implements Exception` ব্যবহার করুন।

অধ্যায় ৯ — Async, Future, Stream ও Isolate

৯.১ Dart-এর Event Loop

Dart একটি থ্রেডে চলে। তাহলে একসাথে অনেক কাজ কীভাবে হয়? উত্তর — Event Loop। দীর্ঘ কাজ (নেটওয়ার্ক, ফাইল) OS-কে দিয়ে দেওয়া হয়, আর Dart অন্য কাজ চালিয়ে যায়। কাজ শেষ হলে ফলাফল একটি সারিতে (queue) এসে জমা হয় এবং Dart সেটি তুলে নেয়।

```
Event Loop
১. Microtask Queue (আগে চলে)
   scheduleMicrotask, Future.value
২. Event Queue (পরে চলে)
   I/O, টাইমার, ব্যবহারকারীর ট্যাপ
```

৯.২ Future — ভবিষ্যতের একটি মান

```
Future<String> fetchUser() async {
  await Future.delayed(const Duration(seconds: 2));
  return 'রহিম';
}

void main() async {
  // পদ্ধতি ১ - async/await (পড়তে সহজ, এটিই ব্যবহার করুন)
  final name = await fetchUser();
  print(name);

  // পদ্ধতি ২ - then/catchError (কলব্যাক স্টাইল)
  fetchUser()
    .then((value) => print(value))
    .catchError((e) => print('এরর: $e'))
    .whenComplete(() => print('শেষ'));

  // একাধিক Future একসাথে চালানো - ★ খুব গুরুত্বপূর্ণ
  final results = await Future.wait([
    fetchUser(),
    fetchSettings(),
    fetchNotifications(),
  ]);
  // ৩টি আলাদা করে await করলে ৬ সেকেন্ড লাগত, এখানে ২ সেকেন্ড

  // যেটি আগে শেষ হবে
  final fastest = await Future.any([serverA(), serverB()]);

  // টাইমআউট
  try {
    await fetchUser().timeout(const Duration(seconds: 5));
  } catch (e) {
    print('টাইমআউট: $e');
  }
}
```

```

    } on TimeoutException {
        print('সময় শেষ');
    }
}

```

সবচেয়ে সাধারণ পারফরম্যান্স ভুল

```

// ✗ ধীর – একটির পর একটি (মোট ৬ সেকেন্ড)
final a = await fetchA();
final b = await fetchB();
final c = await fetchC();

// ✓ দ্রুত – একসাথে (মোট ২ সেকেন্ড)
final results = await Future.wait([fetchA(), fetchB(), fetchC()]);

```

একটির ফলাফল অন্যটিতে লাগলেই কেবল পরপর await করুন।

৯.৩ Stream — একাধিক মানের ধারা

Future = একটি মান, একবার। **Stream** = অনেক মান, সময়ের সাথে সাথে (চ্যাট বার্তা, লোকেশন আপডেট, সার্চ টাইপিং)।

```

// Stream তৈরি – async* ও yield
Stream<int> countStream(int max) async* {
  for (int i = 1; i <= max; i++) {
    await Future.delayed(const Duration(seconds: 1));
    yield i; // একটি করে মান পাঠাও
  }
}

void main() async {
  // শোনা – পদ্ধতি ১
  await for (final value in countStream(5)) {
    print(value);
  }

  // শোনা – পদ্ধতি ২ (বেশি নিয়ন্ত্রণ)
  final sub = countStream(5).listen(
    (data) => print('পেলাম: $data'),
    onError: (e) => print('এরর: $e'),
    onDone: () => print('শেষ'),
    cancelOnError: false,
  );

  await Future.delayed(const Duration(seconds: 3));
  await sub.cancel(); // ★ অবশ্যই বাতিল করুন
}

```

StreamController — নিজের Stream বানানো

```
class CartService {
  final _controller = StreamController<List<Product>>.broadcast();
  final List<Product> _items = [];

  // বাইরে থেকে শোনার জন্য
  Stream<List<Product>> get stream => _controller.stream;

  void add(Product p) {
    _items.add(p);
    _controller.add(List.from(_items)); // নতুন অবস্থা পাঠাও
  }

  void dispose() => _controller.close(); // ★ মেমোরি লিক এড়াতে
}
```

single-subscription বনাম broadcast

সাধারণ Stream-এ **একজনই** শুনতে পারে; দ্বিতীয়বার listen করলে এরর। `.broadcast()` ব্যবহার করলে **একাধিক** শ্রোতা শুনতে পারে — Flutter-এ সাধারণত এটিই লাগে।

Stream রূপান্তর — সার্চ বক্সে খুব কাজে লাগে

```
searchController.stream
  .where((q) => q.length >= 2) // ২ অক্ষরের কম হলে বাদ
  .distinct() // একই লেখা বারবার নয়
  .debounceTime(const Duration(milliseconds: 400)) // rxdart লাগে
  .asyncMap((q) => api.search(q)) // API কল
  .listen((results) => setState(() => _results = results));
```

৯.৪ Isolate — সত্যিকারের সমান্তরাল কাজ

`async/await` ভারী হিসাব দ্রুত করে না — কারণ সব একই থ্রেডে চলে। বড় JSON পার্স করা, ছবি প্রসেস করা বা জটিল গণনায় UI আটকে যায় (jank)। সমাধান — Isolate, যা আলাদা মেমোরিতে আলাদা থ্রেডে চলে।

```
import 'dart:convert';
import 'package:flutter/foundation.dart';

// ভারী কাজের ফাংশন – অবশ্যই top-level বা static হতে হবে
List<Product> parseProducts(String responseBody) {
  final List parsed = jsonDecode(responseBody);
  return parsed.map((json) => Product.fromJson(json)).toList();
}

Future<List<Product>> fetchProducts() async {
  final response = await http.get(Uri.parse(url));
```

```
// ★ compute() – সহজে আলাদা isolate-এ চালানো
return compute(parseProducts, response.body);
}
```

কখন Isolate লাগবে

১০০+ আইটেমের JSON পার্স · ছবি রিসাইজ বা ফিল্টার · এনক্রিপশন · বড় ফাইল পড়া · জটিল গাণিতিক হিসাব। সাধারণ API কল বা ডেটাবেজ কোয়েরিতে **লাগবে না** — সেগুলো এমনিতেই I/O, UI আটকায় না।

৯.৫ async ব্যবহারের নিয়মাবলি

```
// ✗ ভুল – build() কখনো async হয় না
// Widget build(BuildContext context) async { }
```



```
// ✗ ভুল – initState async হতে পারে না
// void initState() async { }
```



```
// ✓ সঠিক – ভিতরে আলাদা ফাংশন ডাকুন
@override
void initState() {
  super.initState();
  _loadData();
}
```



```
Future<void> _loadData() async {
  final data = await api.getData();
  if (!mounted) return; // ★ অপরিহার্য
  setState(() => _data = data);
}
```



```
// ✓ Future ফেরত না দিলে unawaited ব্যবহার করুন
// unawaited(analytics.logEvent('opened'));
```

পর্ব ৩

ফ্লটার মৌলিক

Widget আসলে কী, ভিতরে কী ঘটে, এবং কেন এই জ্ঞান আপনাকে
বাকিদের থেকে এগিয়ে রাখবে।

অধ্যায় ১০ — Widget, Element ও RenderObject

১০.১ তিনটি গাছ — Flutter-এর আসল রহস্য

বেশিরভাগ ডেভেলপার শুধু Widget নিয়ে ভাবে। কিন্তু Flutter ভিতরে **তিনটি আলাদা গাছ** রাখে। এটি বুঝলে অনেক অদ্ভুত আচরণের ব্যাখ্যা পাবেন এবং ইন্টারভিউতে আলাদা করে চোখে পড়বেন।

গাছ	কাজ	আয়ু
Widget Tree	শুধু বর্ণনা — "আমি কেমন দেখতে চাই"। হালকা ও immutable।	প্রতি rebuild-এ নতুন করে তৈরি হয় (সেকেন্ডে ৬০ বার হতে পারে)
Element Tree	Widget ও RenderObject-এর মধ্যে সেতু। কোন widget কোথায় আছে তা মনে রাখে। State এখানেই থাকে।	যতক্ষণ সম্ভব পুনঃব্যবহার হয়
RenderObject Tree	আসল মাপ ও আঁকার কাজ — layout ও paint। ভারী।	দীর্ঘস্থায়ী, খুব কমই নতুন হয়

কেন Widget নতুন করে বানানো সম্ভ্য

Widget হলো শুধু একটি "রেসিপি" — কয়েকটি ফিল্ডসহ হালকা অবজেক্ট। rebuild-এ Flutter নতুন widget-এর সাথে পুরনোটি তুলনা করে (runtimeType ও key মিলিয়ে) এবং **যা বদলায়নি সেই RenderObject পুনঃব্যবহার করে**। তাই ১০০ বার rebuild হলেও আসল আঁকার কাজ সামান্যই হয়।

১০.২ rebuild-এ ঠিক কী ঘটে

```
setState() ডাকা হলো
↓
Element-টিকে "dirty" চিহ্নিত করা হলো
↓
পরের ফ্রেমে build() আবার চলল → নতুন Widget tree
↓
Flutter পুরনো ও নতুন Widget মেলাল:
• একই runtimeType ও key? → Element পুনঃব্যবহার,
  শুধু RenderObject আপডেট (সম্ভ্য)
• আলাদা? → পুরনো Element বাদ,
  নতুন Element ও RenderObject (ব্যবহৃত)
↓
layout → paint → স্ক্রিনে
```

১০.৩ Key — কখন লাগে, কেন লাগে

সাধারণত key লাগে না। কিন্তু একই ধরনের widget-এর তালিকায় ক্রম বদলালে Flutter বিভ্রান্ত হয়ে যায়।

```
// সমস্যা: key ছাড়া লিস্টের আইটেম সরালে state ভুল জায়গায় যায়
ListView(
  children: items.map((item) => TodoTile(item)).toList(),
)

// সমাধান: প্রতিটির অনন্য পরিচয় দিন
ListView(
  children: items.map((item) => TodoTile(
    key: ValueKey(item.id),      // ★
    item,
  )).toList(),
)
```

Key-এর ধরন	ব্যবহার
<code>ValueKey(id)</code>	একটি অনন্য মান দিয়ে চেনানো — সবচেয়ে বেশি ব্যবহৃত
<code>ObjectKey(obj)</code>	পুরো অবজেক্ট দিয়ে চেনানো
<code>UniqueKey()</code>	প্রতিবার নতুন — জোর করে rebuild করাতে
<code>GlobalKey()</code>	অন্য জায়গা থেকে widget-এর state ধরতে (Form, Scaffold)। ব্যয়বহুল, কম ব্যবহার করুন।

GlobalKey অপব্যবহার করবেন না

`GlobalKey` পুরো অ্যাপে অনন্য হতে হয় এবং তুলনামূলক ব্যয়বহুল। শুধু `GlobalKey<FormState>` -এর মতো প্রয়োজনীয় জায়গায় ব্যবহার করুন। ডেটা আদান-প্রদানের জন্য callback বা state management ব্যবহার করুন, `GlobalKey` নয়।

১০.৪ BuildContext — এটি আসলে কী

`BuildContext` আসলে সেই widget-এর **Element**। এটি গাছের মধ্যে widget-টির অবস্থান জানে, তাই এর মাধ্যমে উপরের দিকে খোঁজা যায়।

```
Theme.of(context)           // উপরে গিয়ে নিকটতম Theme খোঁজে
MediaQuery.of(context)      // স্ক্রিনের মাপ
Navigator.of(context)       // নিকটতম Navigator
ScaffoldMessenger.of(context) // Snackbar দেখাতে
Provider.of<Cart>(context)   // নিকটতম Provider
```

ক্লাসিক এরর: "No Scaffold widget found"

```
// X ভুল – এই context-এ Scaffold তৈরিই হয়নি
Widget build(BuildContext context) {
  return Scaffold(
    body: ElevatedButton(
      onPressed: () => ScaffoldMessenger.of(context) // এরর!
        .showSnackBar(...),
    ),
  );
}

// ✓ সমাধান ১ – Builder দিয়ে নতুন context
body: Builder(
  builder: (context) => ElevatedButton(
    onPressed: () => ScaffoldMessenger.of(context).showSnackBar(...),
  ),
)

// ✓ সমাধান ২ – আলাদা widget-এ সরিয়ে নিন (উত্তম)
```

কারণ: `of(context)` সবসময় **উপরের দিকে** খোঁজে। build মেথডের context-টি Scaffold-এর **উপরে**, ভিতরে নয়।

অধ্যায় ১১ — Stateless, Stateful ও Lifecycle

১১.১ StatelessWidget

```
class ProductCard extends StatelessWidget {
  final String title;
  final double price;
  final VoidCallback onTap;

  const ProductCard({
    super.key,
    required this.title,
    required this.price,
    required this.onTap,
  });

  @override
  Widget build(BuildContext context) {
    return Card(
      child: ListTile(
        title: Text(title),
        subtitle: Text('ট $price'),
        onTap: onTap,
      ),
    );
  }
}
```

নিজের কোনো পরিবর্তনশীল অবস্থা নেই। বাইরে থেকে পাওয়া ডেটা বদলালে প্যারেন্ট নতুন widget বানিয়ে দেয়। সব ফিল্ড final হতে হবে।

১১.২ StatefulWidget — দুটি ক্লাস কেন

```
class Counter extends StatefulWidget { // ১ম ক্লাস: immutable কনফিগ
  final int initialValue;
  const Counter({super.key, this.initialValue = 0});

  @override
  State<Counter> createState() => _CounterState();
}

class _CounterState extends State<Counter> { // ২য় ক্লাস: পরিবর্তনশীল state
  late int _count;

  @override
  void initState() {
    super.initState();
    _count = widget.initialValue; // widget. দিয়ে কনফিগ পড়া
  }
}
```

```

}

@override
Widget build(BuildContext context) {
  return Column(children: [
    Text('$_count'),
    ElevatedButton(
      onPressed: () => setState(() => _count++),
      child: const Text('বাড়াও'),
    ),
  ]);
}
}

```

দুটি ক্লাস কেন — গুরুত্বপূর্ণ ধারণা

Widget প্রতি rebuild-এ ধ্বংস হয়ে নতুন হয়। কিন্তু State অবজেক্টটি Element tree-তে বেঁচে থাকে। তাই কাউন্টারের মান বা ফর্মের ডেটা হারায় না। এই আলাদা করাটাই Flutter-কে দ্রুত ও নিরাপদ রাখে।

১১.৩ সম্পূর্ণ Lifecycle

```

class _MyState extends State<MyWidget> {

  @override
  void initState() {
    super.initState();
    // ★ একবারই চলে, widget তৈরি হওয়ার সময়
    // এখানে করুন: controller তৈরি, API কল, listener যোগ, animation setup
    // এখানে করবেন না: context ব্যবহার (Theme.of/MediaQuery.of) – এখনো প্রস্তুত নয়
  }

  @override
  void didChangeDependencies() {
    super.didChangeDependencies();
    // initState-এর ঠিক পরে, এবং নির্ভরশীল InheritedWidget বদলালে
    // এখানে context নিরাপদে ব্যবহার করা যায়
  }

  @override
  Widget build(BuildContext context) {
    // প্রতিটি rebuild-এ চলে। শুদ্ধ থাকতে হবে – কোনো পার্শ্বপ্রতিক্রিয়া নয়।
    // ✗ এখানে setState, API কল, বা ভারী হিসাব করবেন না
    return const SizedBox();
  }

  @override
  void didUpdateWidget(covariant MyWidget oldWidget) {
    super.didUpdateWidget(oldWidget);
    // প্যারেন্ট নতুন কনফিগ পাঠালে চলে
    if (oldWidget.userId != widget.userId) {

```

```

        _reloadData(); // নতুন id হলে ডেটা আবার আনুন
    }
}

@override
void deactivate() {
    // widget গাছ থেকে সাময়িকভাবে সরানো হলো
    super.deactivate();
}

@override
void dispose() {
    // ★ স্থায়ীভাবে ধ্বংস – এখানে সব পরিষ্কার করুন
    _controller.dispose();
    _subscription.cancel();
    _timer?.cancel();
    _focusNode.dispose();
    super.dispose(); // সবার শেষে
}
}

```

dispose না করলে যা হয়

মেমোরি লিক — অ্যাপ ধীরে ধীরে ভারী হয়ে ক্র্যাশ করে। এবং **"setState() called after dispose()"**

এরর। নিয়ম: যা কিছু `initState` -এ তৈরি করেছেন (Controller, StreamSubscription, Timer, FocusNode, AnimationController), তার সবই `dispose` -এ বন্ধ করুন।

১১.৪ অ্যাপ-স্তরের Lifecycle

```

class _MyAppState extends State<MyApp> with WidgetsBindingObserver {

    @override
    void initState() {
        super.initState();
        WidgetsBinding.instance.addObserver(this);
    }

    @override
    void didChangeAppLifecycleState(AppLifecycleState state) {
        switch (state) {
            case AppLifecycleState.resumed: print('অ্যাপ সামনে এলো'); break;
            case AppLifecycleState.inactive: print('আংশিক ঢাকা'); break;
            case AppLifecycleState.paused: print('ব্যাকগ্রাউন্ডে গেল'); break;
            case AppLifecycleState.detached: print('বন্ধ হচ্ছে'); break;
            case AppLifecycleState.hidden: break;
        }
    }

    @override
    void dispose() {

```

```
WidgetsBinding.instance.removeObserver(this);
super.dispose();
}
}
```

ব্যবহার: ব্যাকগ্রাউন্ডে গেলে ভিডিও থামানো, ফিরে এলে ডেটা রিফ্রেশ করা, বা নিরাপত্তার জন্য স্ক্রিন বাপসা করা।

১১.৫ setState-এর নিয়ম

```
// ✓ সঠিক – শুধু মান বদলানোর কাজটুকু ভিতরে
setState(() {
  _count++;
});

// ✗ ভুল – ভিতরে async কাজ
setState() async {
  final data = await api.get();    // কাজ করবে না
});

// ✓ সঠিক – আগে await, পরে setState
final data = await api.get();
if (!mounted) return;
setState(() => _data = data);

// ✗ ভুল – build() এর ভিতরে setState → অসীম লুপ

// ✓ যদি build এর পরে কিছু করতেই হয়
WidgetsBinding.instance.addPostFrameCallback(_) {
  setState(() => _ready = true);
});
```

অধ্যায় ১২ — Constraint — লেআউটের আসল নিয়ম

১২.১ Flutter-এর সোনালি নিয়ম

মুখস্থ করে ফেলুন

"Constraints go down. Sizes go up. Parent sets position."

১. প্যারেন্ট চাইল্ডকে **সীমা** (সর্বনিম্ন/সর্বোচ্চ প্রস্থ ও উচ্চতা) দেয়।
২. চাইল্ড সেই সীমার মধ্যে নিজের **মাপ** ঠিক করে উপরে জানায়।
৩. প্যারেন্ট চাইল্ডকে **কোথায় বসবে** তা ঠিক করে।

এই তিন লাইনেই Flutter-এর পুরো লেআউট ব্যবস্থা।

১২.২ কেন আপনার Container ২০০ পিক্সেল হচ্ছে না

```
// আপনি লিখলেন
Container(width: 200, height: 200, color: Colors.red)

// কিন্তু পুরো স্ক্রিন লাল হয়ে গেল! কেন?
// কারণ প্যারেন্ট (যেমন Scaffold body) "tight constraint" দিয়েছে –
// অর্থাৎ "তোমাকে ঠিক এই মাপই হতে হবে", পছন্দের সুযোগ নেই।

// ✓ সমাধান – Center বা Align দিয়ে মুড়ুন
Center(
  child: Container(width: 200, height: 200, color: Colors.red),
)
// Center তার চাইল্ডকে "loose constraint" দেয় – "সর্বোচ্চ এতটুকু,
// তবে কম হলেও চলবে"। তাই এখন ২০০x২০০ কাজ করবে।
```

১২.৩ Tight বনাম Loose constraint

ধরন	অর্থ	কারা দেয়
Tight	$\min = \max$ । চাইল্ডের কোনো পছন্দ নেই।	Scaffold body, Expanded, SizedBox.expand
Loose	$\min = 0$, $\max =$ কিছু একটা। চাইল্ড ছোট হতে পারে।	Center, Align, Padding, ListView
Unbounded	$\max =$ অসীম। বিপজ্জনক।	Column-এর ভিতরে, ScrollView-এর ভিতরে

১২.৪ Unbounded — সবচেয়ে সাধারণ লেআউট এরর

```
// X এরর: "Vertical viewport was given unbounded height"
Column(
  children: [
    Text('শিরোনাম'),
    ListView(children: [...]), // Column অসীম উচ্চতা দেয়
  ],
)

// ✓ সমাধান ১ – Expanded (সাধারণত সেরা)
Column(
  children: [
    Text('শিরোনাম'),
    Expanded(child: ListView(children: [...])),
  ],
)

// ✓ সমাধান ২ – নির্দিষ্ট উচ্চতা
SizedBox(height: 300, child: ListView(...))

// ✓ সমাধান ৩ – ListView নিজে স্ক্রল করবে না
ListView(
  shrinkWrap: true,
  physics: const NeverScrollableScrollPhysics(),
  children: [...],
)
```

shrinkWrap কেন সবসময় ভালো নয়

`shrinkWrap: true` দিলে ListView সব আইটেম একসাথে তৈরি করে উচ্চতা মাপে — ফলে builder-এর সুবিধা নষ্ট হয় এবং ১০০০ আইটেম থাকলে অ্যাপ আটকে যায়। অল্প আইটেমে ঠিক আছে, বেশি হলে Expanded বা CustomScrollView ব্যবহার করুন।

১২.৫ LayoutBuilder ও IntrinsicHeight

```
// প্যারেন্ট কত জায়গা দিচ্ছে তা জেনে সিদ্ধান্ত নেওয়া
LayoutBuilder(
  builder: (context, constraints) {
    if (constraints.maxWidth > 600) {
      return const TabletLayout();
    }
    return const MobileLayout();
  },
)

// Row-এর সব চাইল্ডকে সমান উচ্চতা দিতে (ব্যয়বহুল, কম ব্যবহার করুন)
IntrinsicHeight(
  child: Row(
    crossAxisAlignment: CrossAxisAlignment.stretch,
    children: [...],
  ),
)
```

পর্ব ৪

UI নির্মাণ

Widget, লেআউট, স্ক্রলিং, নেভিগেশন, ফর্ম, থিম ও অ্যানিমেশন —
সুন্দর অ্যাপের সবকিছু।

অধ্যায় ১৩ — মৌলিক Widget সমূহ

১৩.১ Text ও RichText

```
Text(  
  'ফ্লাটার শিখছি',  
  style: TextStyle(  
    fontSize: 20, fontWeight: FontWeight.w600,  
    color: Colors.teal, letterSpacing: 0.5, height: 1.5,  
    decoration: TextDecoration.underline,  
    decorationStyle: TextDecorationStyle.dotted,  
    shadows: const [Shadow(blurRadius: 2, color: Colors.black26)],  
  ),  
  textAlign: TextAlign.center,  
  maxLines: 2,  
  overflow: TextOverflow.ellipsis,  
  softWrap: true,  
)  
  
// থিম থেকে স্টাইল নেওয়া – পেশাদার উপায়  
Text('শিরোনাম', style: Theme.of(context).textTheme.titleLarge)  
  
// এক লাইনে একাধিক স্টাইল  
RichText(  
  text: TextSpan(  
    style: DefaultTextStyle.of(context).style,  
    children: const [  
      TextSpan(text: 'মোট দাম: '),  
      TextSpan(text: 'ট ২,৪৯৯',  
        style: TextStyle(fontWeight: FontWeight.bold, color: Colors.red)),  
      TextSpan(text: ' (ভ্যাটসহ)', style: TextStyle(fontSize: 11)),  
    ],  
  ),  
)  
  
// লেখার ভিতরে widget বসানো  
Text.rich(TextSpan(children: [  
  const TextSpan(text: 'রেটিং '),  
  const WidgetSpan(child: Icon(Icons.star, size: 16, color: Colors.amber)),  
  const TextSpan(text: ' ৪.৫'),  
]))
```

১৩.২ Image — সব উৎস থেকে

```
Image.asset('assets/images/logo.png', width: 120)  
Image.network('https://example.com/pic.jpg')  
Image.file(File(path))  
Image.memory(uint8ListData)
```

```
// পূর্ণ নিয়ন্ত্রণসহ
Image.network(
  url,
  width: double.infinity, height: 200,
  fit: BoxFit.cover,
  loadingBuilder: (context, child, progress) {
    if (progress == null) return child;
    return Center(
      child: CircularProgressIndicator(
        value: progress.expectedTotalBytes != null
          ? progress.cumulativeBytesLoaded / progress.expectedTotalBytes!
          : null,
      ),
    );
  },
  errorBuilder: (context, error, stack) =>
    const Icon(Icons.broken_image, size: 48, color: Colors.grey),
)

// ★ প্রোডাকশনে সবসময় ক্যাশ ব্যবহার করুন
CachedNetworkImage(
  imageUrl: url,
  placeholder: (c, u) => const CircularProgressIndicator(),
  errorWidget: (c, u, e) => const Icon(Icons.error),
  fit: BoxFit.cover,
)
```

BoxFit	আচরণ
cover	পুরো জায়গা ভরে, দরকারে ছবি কেটে যায় — সবচেয়ে বেশি ব্যবহৃত
contain	পুরো ছবি দেখায়, ফাঁকা জায়গা থাকতে পারে
fill	জোর করে ভরে, ছবি বিকৃত হয়
fitWidth / fitHeight	শুধু প্রস্থ বা উচ্চতা মেলায়
none / scaleDown	আসল মাপে / দরকার হলে ছোট করে

১৩.৩ Button-এর সব ধরন

```
ElevatedButton(onPressed: () {}, child: const Text('প্রধান'))
FilledButton(onPressed: () {}, child: const Text('ভার্ট')) // M3
FilledButton.tonal(onPressed: () {}, child: const Text('হালকা'))
OutlinedButton(onPressed: () {}, child: const Text('বর্ডার'))
TextButton(onPressed: () {}, child: const Text('শুধু লেখা'))
IconButton(onPressed: () {}, icon: const Icon(Icons.share))
```

```

FloatingActionButton(onPressed: () {}, child: const Icon(Icons.add))
FloatingActionButton.extended(
  onPressed: () {}, icon: const Icon(Icons.add), label: const Text('নতুন'),
)

// সম্পূর্ণ স্টাইলসহ
ElevatedButton(
  onPressed: _isLoading ? null : _submit, // null = নিষ্ক্রিয়
  style: ElevatedButton.styleFrom(
    backgroundColor: Colors.teal,
    foregroundColor: Colors.white,
    disabledBackgroundColor: Colors.grey.shade300,
    padding: const EdgeInsets.symmetric(horizontal: 28, vertical: 14),
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
    elevation: 2,
    minimumSize: const Size(double.infinity, 50),
  ),
  child: _isLoading
    ? const SizedBox(
        width: 20, height: 20,
        child: CircularProgressIndicator(strokeWidth: 2, color: Colors.white),
      )
    : const Text('সাবমিট করুন'),
)

```

১৩.৪ ট্যাপ ও অঙ্গভঙ্গি

```

// InkWell – টেউয়ের টেউ প্রভাবসহ (Material অনুভূতি)
InkWell(
  onTap: () {},
  onLongPress: () {},
  borderRadius: BorderRadius.circular(12),
  splashColor: Colors.teal.withOpacity(0.2),
  child: Container(padding: const EdgeInsets.all(16), child: const Text('চাপুন')),
)

// GestureDetector – প্রভাব ছাড়া, বেশি অঙ্গভঙ্গি
GestureDetector(
  onTap: () {},
  onDoubleTap: () {},
  onLongPress: () {},
  onPanUpdate: (details) => print(details.delta),
  onScaleUpdate: (details) => print(details.scale),
  onHorizontalDragEnd: (d) {
    if (d.primaryVelocity! > 0) print('ডানে সোয়াইপ');
  },
  child: Container(color: Colors.transparent, child: const Text('টানুন')),
)

```

InkWell-এ ডেউ দেখা যাচ্ছে না?

InkWell-এর ডেউ Material পৃষ্ঠের উপর আঁকা হয়। উপরে যদি রঙিন Container থাকে, ডেউ ঢাকা পড়ে যায়। সমাধান — `Material(color: Colors.transparent, child: InkWell(...))` ব্যবহার করুন, অথবা `Ink(decoration: ..., child: InkWell(...))`।

১৩.৫ Card, ListTile ও অন্যান্য

```
Card(
  elevation: 3,
  clipBehavior: Clip.antiAlias,
  shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
  child: Column(children: [...]),
)

ListTile(
  leading: const CircleAvatar(child: Icon(Icons.person)),
  title: const Text('রহিম উদ্দিন'),
  subtitle: const Text('সফটওয়্যার ইঞ্জিনিয়ার'),
  trailing: const Icon(Icons.chevron_right),
  onTap: () {},
  contentPadding: const EdgeInsets.symmetric(horizontal: 16),
)

Chip(label: const Text('ফ্লাটার'), onDelete: () {})
Badge(label: const Text('৩'), child: const Icon(Icons.notifications))
const Divider(height: 1, thickness: 1, indent: 16, endIndent: 16)
const CircularProgressIndicator()
const LinearProgressIndicator(value: 0.7)
const Tooltip(message: 'মুছে ফেলুন', child: Icon(Icons.delete))
```

অধ্যায় ১৪ — লেআউট Widget সমূহ

১৪.১ Container বনাম SizedBox বনাম Padding

```
// Container – সব একসাথে (কিন্তু ভারী)
Container(
  width: 200, height: 100,
  padding: const EdgeInsets.all(16),
  margin: const EdgeInsets.symmetric(vertical: 8),
  alignment: Alignment.center,
  decoration: BoxDecoration(
    color: Colors.white,
    borderRadius: BorderRadius.circular(16),
    border: Border.all(color: Colors.grey.shade300),
    boxShadow: [
      BoxShadow(color: Colors.black.withOpacity(.08), blurRadius: 10,
        offset: const Offset(0, 4)),
    ],
    gradient: const LinearGradient(
      colors: [Color(0xFF0E7490), Color(0xFF4CC9F0)],
      begin: Alignment.topLeft, end: Alignment.bottomRight,
    ),
  ),
  child: const Text('বিষয়বস্তু'),
)
```

```
// শুধু মাপ দরকার? SizedBox ব্যবহার করুন – হালকা ও const হতে পারে
const SizedBox(height: 16)
const SizedBox(width: 100, height: 100)
const SizedBox.shrink() // কিছুই দেখাবে না
const SizedBox.expand() // যত জায়গা আছে সব
```

```
// শুধু ফাঁকা দরকার? Padding ব্যবহার করুন
const Padding(padding: EdgeInsets.all(16), child: Text('হ্যালো'))
```

পারফরম্যান্স অভ্যাস

শুধু ফাঁকা জায়গার জন্য Container(height: 16) লিখবেন না — const SizedBox(height: 16) লিখুন। Container ভিতরে অনেকগুলো widget-এ ভেঙে যায়, SizedBox একটিমাত্র হালকা RenderObject।

১৪.২ Row ও Column — সম্পূর্ণ

```
Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  crossAxisAlignment: CrossAxisAlignment.center,
  mainAxisAlignment: MainAxisAlignment.max, // max = সব জায়গা নেবে, min = যতটুকু লাগে
  textDirection: TextDirection.ltr,
```

```
children: const [...],
)
```

MainAxisAlignment	ফলাফল
start / center / end	শুরুতে / মাঝে / শেষে চেপে
spaceBetween	দুই প্রান্তে, ফাঁক মাঝে সমান
spaceAround	প্রতিটির দুপাশে সমান (প্রান্তে অর্ধেক)
spaceEvenly	সব ফাঁক একদম সমান

১৪.৩ Expanded, Flexible ও Spacer

```
Row(children: [
  // Expanded – বাকি জায়গা জোর করে দখল করে
  Expanded(flex: 2, child: Container(color: Colors.red, height: 50)),
  Expanded(flex: 1, child: Container(color: Colors.blue, height: 50)),
])

Row(children: [
  // Flexible – বাকি জায়গা নিতে পারে, কিন্তু বাধ্য নয়
  Flexible(fit: FlexFit.loose, child: Text('ছোট লেখা')),
  Flexible(fit: FlexFit.tight, child: Container(color: Colors.green)),
])

Row(children: const [
  Text('বামে'),
  Spacer(), // Expanded(child: SizedBox()) এর শর্টকাট
  Text('ডানে'),
])

// দীর্ঘ টেক্সট Row-এ overflow করলে
Row(children: [
  const Icon(Icons.person),
  const SizedBox(width: 8),
  Expanded( // ★ এটিই সমাধান
    child: Text(longName, overflow: TextOverflow.ellipsis),
  ),
])
```

১৪.৪ Stack, Positioned ও Align

```
Stack(
  alignment: Alignment.center,
  clipBehavior: Clip.none, // সীমার বাইরেও দেখাবে
  fit: StackFit.loose,
```

```

children: [
  Image.network(url, height: 220, width: double.infinity, fit: BoxFit.cover),

  // উপরে গ্রেডিয়েন্ট আবরণ – লেখা পড়তে সুবিধা হয়
  Positioned.fill(
    child: DecoratedBox(
      decoration: BoxDecoration(
        gradient: LinearGradient(
          begin: Alignment.topCenter, end: Alignment.bottomCenter,
          colors: [Colors.transparent, Colors.black.withOpacity(.7)],
        ),
      ),
    ),
  ),
  Positioned(bottom: 16, left: 16,
    child: const Text('সুন্দরবন',
      style: TextStyle(color: Colors.white, fontSize: 20))),

  Positioned(top: 8, right: 8, child: const Badge(label: Text('নতুন'))),

  // Align – Positioned-এর সহজ বিকল্প
  const Align(alignment: Alignment.topLeft, child: Icon(Icons.arrow_back)),
],
)

```

১৪.৫ Wrap — জায়গা শেষ হলে পরের লাইনে

```

Wrap(
  spacing: 8,           // পাশাপাশি ফাঁক
  runSpacing: 8,        // লাইনের মধ্যে ফাঁক
  alignment: WrapAlignment.start,
  children: tags.map((t) => Chip(label: Text(t))).toList(),
)

```

কখন: ট্যাগ, ফিল্টার চিপ, ক্যাটাগরি — যেখানে আইটেমের সংখ্যা ও প্রস্থ অনিশ্চিত। Row ব্যবহার করলে overflow হতো।

১৪.৬ অন্যান্য দরকারি লেআউট

```

SafeArea(child: ...) // নচ ও স্ট্যাটাস বার এড়ানো
AspectRatio(aspectRatio: 16 / 9, child: ...) // অনুপাত ঠিক রাখা
FittedBox(fit: BoxFit.scaleDown, child: ...) // ভিতরের জিনিস মাপে আনা
ConstrainedBox(
  constraints: const BoxConstraints(maxWidth: 400, minHeight: 100),
  child: ...,
)
ClipRRect(borderRadius: BorderRadius.circular(16), child: ...)
ClipOval(child: ...)

```

```
Opacity(opacity: 0.5, child: ...)
Transform.rotate(angle: 0.5, child: ...)
Transform.scale(scale: 1.2, child: ...)
Visibility(visible: false, child: ...)
Offstage(offstage: true, child: ...) // লুকানো কিন্তু state বেঁচে থাকে
Table(children: [TableRow(children: [...])])
```


অধ্যায় ১৫ — স্ক্রলিং, ListView ও Sliver

১৫.১ ListView-এর সব রূপ

```
// ১. স্থির কয়েকটি আইটেম
ListView(children: const [ListTile(title: Text('এক')), ...])

// ২. builder – ★ ডিফল্ট পছন্দ, শুধু দৃশ্যমান আইটেম তৈরি করে
ListView.builder(
  itemCount: items.length,
  padding: const EdgeInsets.all(12),
  physics: const BouncingScrollPhysics(),
  itemBuilder: (context, index) => ProductTile(items[index]),
)

// ৩. separated – মাঝে বিভাজক
ListView.separated(
  itemCount: items.length,
  separatorBuilder: (_, __) => const Divider(height: 1),
  itemBuilder: (c, i) => ListTile(title: Text(items[i])),
)

// ৪. অনুভূমিক লিস্ট
SizedBox(
  height: 120,
  child: ListView.builder(
    scrollDirection: Axis.horizontal,
    itemCount: categories.length,
    itemBuilder: (c, i) => CategoryCard(categories[i]),
  ),
)
```

১৫.২ GridView

```
GridView.builder(
  gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: 2,
    crossAxisSpacing: 12, mainAxisSpacing: 12,
    childAspectRatio: 0.72,
  ),
  itemCount: products.length,
  itemBuilder: (c, i) => ProductCard(products[i]),
)

// প্রস্থ অনুযায়ী স্বয়ংক্রিয় কলাম সংখ্যা – রেসপন্সিভ
GridView.builder(
  gridDelegate: const SliverGridDelegateWithMaxCrossAxisExtent(
    maxCrossAxisExtent: 200, // প্রতিটি আইটেম সর্বোচ্চ ২০০ চওড়া
    crossAxisSpacing: 12, mainAxisSpacing: 12,
```

```
),  
  itemBuilder: ...,  
)
```

GridView-এ RenderFlex overflow?

`childAspectRatio` ঠিক করুন। এটি **প্রস্থ ÷ উচ্চতা**। মান ছোট করলে (যেমন 0.6) আইটেম লম্বা হয়।
ছবি + লেখা + বাটনওয়ালা কার্ডে সাধারণত 0.65–0.75 ভালো কাজ করে।

১৫.৩ CustomScrollView ও Sliver

একই স্ক্রলে ভিন্ন ধরনের অংশ (কোলাপসিং হেডার + গ্রিড + লিস্ট) থাকলে Sliver ব্যবহার করুন।

```
CustomScrollView(  
  slivers: [  
    // স্ক্রল করলে ছোট হয়ে যাওয়া AppBar  
    SliverAppBar(  
      expandedHeight: 220,  
      pinned: true,           // ছোট হয়ে উপরে আটকে থাকবে  
      floating: false,  
      flexibleSpace: FlexibleSpaceBar(  
        title: const Text('পণ্যসমূহ'),  
        background: Image.network(bannerUrl, fit: BoxFit.cover),  
      ),  
    ),  
  
    // সাধারণ widget-কে sliver বানানো  
    const SliverToBoxAdapter(  
      child: Padding(padding: EdgeInsets.all(16), child: Text('জনপ্রিয়')),  
    ),  
  
    // গ্রিড  
    SliverGrid(  
      gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(  
        crossAxisCount: 2, mainAxisSpacing: 12, crossAxisSpacing: 12,  
      ),  
      delegate: SliverChildBuilderDelegate(  
        (context, i) => ProductCard(products[i]),  
        childCount: products.length,  
      ),  
    ),  
  
    // লিস্ট  
    SliverList(  
      delegate: SliverChildBuilderDelegate(  
        (context, i) => ListTile(title: Text(reviews[i])),  
        childCount: reviews.length,  
      ),  
    ),  
  ],  
)
```

```
// ফাঁকা জায়গা
const SliverPadding(padding: EdgeInsets.only(bottom: 40)),
],
)
```

১৫.৪ Pagination — অসীম স্ক্রল

```
class _ProductListState extends State<ProductList> {
  final _scrollController = ScrollController();
  final List<Product> _items = [];
  int _page = 1;
  bool _isLoading = false;
  bool _hasMore = true;

  @override
  void initState() {
    super.initState();
    _loadMore();
    _scrollController.addListener(() {
      // নিচ থেকে ২০০ পিক্সেল আগেই পরের পাতা আনা শুরু
      if (_scrollController.position.pixels >=
        _scrollController.position.maxScrollExtent - 200) {
        _loadMore();
      }
    });
  }

  Future<void> _loadMore() async {
    if (_isLoading || !_hasMore) return;
    setState(() => _isLoading = true);

    final newItems = await api.getProducts(page: _page);
    if (!mounted) return;

    setState(() {
      _items.addAll(newItems);
      _hasMore = newItems.isNotEmpty;
      _page++;
      _isLoading = false;
    });
  }

  @override
  void dispose() {
    _scrollController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return ListView.builder(
      controller: _scrollController,
```

```

        itemCount: _items.length + (_hasMore ? 1 : 0),
        itemBuilder: (context, i) {
          if (i >= _items.length) {
            return const Padding(
              padding: EdgeInsets.all(16),
              child: Center(child: CircularProgressIndicator()),
            );
          }
          return ProductTile(_items[i]);
        },
      );
    }
  }
}

```

🔄 Refresh & Swipe

```

RefreshIndicator(
  onRefresh: () async {
    _page = 1; _items.clear();
    await _loadMore();
  },
  child: ListView.builder(...),
)

Dismissible(
  key: ValueKey(item.id),
  direction: DismissDirection.endToStart,
  confirmDismiss: (dir) async => await showDialog<bool>(
    context: context,
    builder: (c) => AlertDialog(
      title: const Text('মুছে ফেলবেন?'),
      actions: [
        TextButton(onPressed: () => Navigator.pop(c, false),
          child: const Text('না')),
        TextButton(onPressed: () => Navigator.pop(c, true),
          child: const Text('হ্যাঁ')),
      ],
    ),
  ),
  background: Container(
    color: Colors.red, alignment: Alignment.centerRight,
    padding: const EdgeInsets.only(right: 20),
    child: const Icon(Icons.delete, color: Colors.white),
  ),
  onDismissed: (_) => setState(() => items.remove(item)),
  child: ListTile(title: Text(item.title)),
)

```

অধ্যায় ১৬ — নেভিগেশন ও রাউটিং

১৬.১ Navigator 1.0 — স্ট্যাক ভিত্তিক

```
// নতুন পেজে যাওয়া
Navigator.push(context,
  MaterialPageRoute(builder: (_) => const DetailPage()));

// ফিরে আসা
Navigator.pop(context);

// বর্তমান পেজ প্রতিস্থাপন (স্প্যাশ → হোম)
Navigator.pushReplacement(context,
  MaterialPageRoute(builder: (_) => const HomePage()));

// সব মুছে নতুন (লগআউট)
Navigator.pushAndRemoveUntil(
  context,
  MaterialPageRoute(builder: (_) => const LoginPage()),
  (route) => false,
);

// নির্দিষ্ট পেজ পর্যন্ত ফিরে যাওয়া
Navigator.popUntil(context, (route) => route.isFirst);

// ফিরে যাওয়া সম্ভব কিনা
if (Navigator.canPop(context)) Navigator.pop(context);
```

১৬.২ ডেটা পাঠানো ও ফেরত আনা

```
// পাঠানো – constructor দিয়ে (সবচেয়ে নিরাপদ)
Navigator.push(context, MaterialPageRoute(
  builder: (_) => ProductDetail(product: product),
));

// ফেরত আনা
final result = await Navigator.push<String>(
  context,
  MaterialPageRoute(builder: (_) => const CityPicker()),
);
if (result != null && mounted) {
  setState(() => _city = result);
}

// দ্বিতীয় পেজে ফেরত পাঠানো
Navigator.pop(context, 'চট্টগ্রাম');
```

১৬.৩ Named Routes

```
MaterialApp(  
  initialRoute: '/',  
  routes: {  
    '/': (context) => const HomePage(),  
    '/cart': (context) => const CartPage(),  
    '/profile': (context) => const ProfilePage(),  
  },  
  
  // আর্গুমেন্টসহ রাউট বা ডাইনামিক রাউট  
  onGenerateRoute: (settings) {  
    if (settings.name == '/detail') {  
      final product = settings.arguments as Product;  
      return MaterialPageRoute(  
        builder: (_) => ProductDetail(product: product),  
      );  
    }  
    return null;  
  },  
  
  // অজানা রাউট  
  onUnknownRoute: (settings) => MaterialPageRoute(  
    builder: (_) => const NotFoundPage(),  
  ),  
)  
  
// ব্যবহার  
Navigator.pushNamed(context, '/detail', arguments: product);
```

১৬.৪ go_router — আধুনিক পদ্ধতি

```
flutter pub add go_router
```

```
final router = GoRouter(  
  initialLocation: '/',  
  routes: [  
    GoRoute(  
      path: '/',  
      builder: (context, state) => const HomePage(),  
      routes: [  
        // নেষ্টেড রাউট → /product/5  
        GoRoute(  
          path: 'product/:id',  
          builder: (context, state) {  
            final id = state.pathParameters['id']!;  
            return ProductDetail(id: int.parse(id));  
          },  
        ),  
      ],  
    ),  
  ],  
)
```

```

    ),
    GoRoute(path: '/login', builder: (c, s) => const LoginPage()),
  ],

  // লগইন না থাকলে স্বয়ংক্রিয়ভাবে সরিয়ে দেওয়া ★
  redirect: (context, state) {
    final loggedIn = AuthService.isLoggedIn;
    final goingToLogin = state.matchedLocation == '/login';

    if (!loggedIn && !goingToLogin) return '/login';
    if (loggedIn && goingToLogin) return '/';
    return null;
  },

  errorBuilder: (context, state) => const NotFoundPage(),
);

// অ্যাপে যুক্ত করা
MaterialApp.router(routerConfig: router)

// ব্যবহার
context.go('/product/5'); // প্রতিস্থাপন
context.push('/product/5'); // স্ট্যাকে যোগ
context.pop();

```

কখন go_router ব্যবহার করবেন

ওয়েব সাপোর্ট, deep linking (নোটিফিকেশনে ক্লিক করে নির্দিষ্ট পেজে যাওয়া), লগইন-ভিত্তিক redirect, বা নেস্টেড নেভিগেশন দরকার হলে। ছোট অ্যাপে সাধারণ `Navigator.push` -ই যথেষ্ট।

১৬.৫ Tab, Drawer ও Bottom Navigation

```

// IndexedStack – ট্যাব বদলালেও প্রতিটি পেজের state বেঁচে থাকে ★
class MainShell extends StatefulWidget {
  const MainShell({super.key});
  @override State<MainShell> createState() => _MainShellState();
}

class _MainShellState extends State<MainShell> {
  int _index = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: IndexedStack( // _pages[_index] লিখলে state হারাত
        index: _index,
        children: const [HomePage(), SearchPage(), CartPage(), ProfilePage()],
      ),
      bottomNavigationBar: NavigationBar( // Material 3
        selectedIndex: _index,

```

```

onDestinationSelected: (i) => setState(() => _index = i),
destinations: const [
  NavigationDestination(icon: Icon(Icons.home), label: 'হোম'),
  NavigationDestination(icon: Icon(Icons.search), label: 'খুঁজুন'),
  NavigationDestination(icon: Icon(Icons.shopping_cart), label: 'কার্ট'),
  NavigationDestination(icon: Icon(Icons.person), label: 'প্রোফাইল'),
],
),
);
}
}

// TabBar – উপরে ট্যাব
DefaultTabController(
  length: 3,
  child: Scaffold(
    appBar: AppBar(
      title: const Text('খবর'),
      bottom: const TabBar(tabs: [
        Tab(text: 'সব'), Tab(text: 'খেলা'), Tab(text: 'প্রযুক্তি'),
      ]),
    ),
    body: const TabBarView(children: [AllNews(), SportsNews(), TechNews()]),
  ),
)

```

১৬.৬ Dialog, Sheet ও ফিরে যাওয়া নিয়ন্ত্রণ

```

// নিশ্চিতকরণ ডায়ালগ থেকে ফলাফল
final confirmed = await showDialog<bool>(
  context: context,
  barrierDismissible: false,
  builder: (ctx) => AlertDialog(
    title: const Text('নিশ্চিত?'),
    content: const Text('এই কাজটি ফেরানো যাবে না।'),
    actions: [
      TextButton(onPressed: () => Navigator.pop(ctx, false),
        child: const Text('বাতিল')),
      FilledButton(onPressed: () => Navigator.pop(ctx, true),
        child: const Text('মুছুন')),
    ],
  ),
);
if (confirmed == true) _delete();

// নিচ থেকে শিট
showModalBottomSheet(
  context: context,
  isScrollControlled: true, // কিবোর্ড এলে বড় হবে
  shape: const RoundedRectangleBorder(
    borderRadius: BorderRadius.vertical(top: Radius.circular(20)),
  ),
)

```



```
builder: (_) => Padding(
  padding: EdgeInsets.only(
    bottom: MediaQuery.of(context).viewInsets.bottom, // কিবোর্ড
  ),
  child: const FilterSheet(),
),
);

// ব্যাক বাটন আটকানো – অসংরক্ষিত পরিবর্তন থাকলে
PopScope(
  canPop: !_hasUnsavedChanges,
  onPopInvoked: (didPop) async {
    if (didPop) return;
    final leave = await _confirmLeave();
    if (leave && context.mounted) Navigator.pop(context);
  },
  child: Scaffold(...),
)
```

অধ্যায় ১৭ — ফর্ম, ইনপুট ও ভ্যালিডেশন

১৭.১ TextField-এর সম্পূর্ণ কনফিগারেশন

```
TextField(  
  controller: _controller,  
  focusNode: _focusNode,  
  keyboardType: TextInputType.emailAddress,  
  textInputAction: TextInputAction.next,  
  textCapitalization: TextCapitalization.words,  
  obscureText: false,  
  maxLength: 50,  
  maxLines: 1,  
  minLines: 1,  
  enabled: true,  
  readOnly: false,  
  autofocus: false,  
  inputFormatters: [  
    FilteringTextInputFormatter.digitsOnly,           // শুধু সংখ্যা  
    LengthLimitingTextInputFormatter(11),  
  ],  
  onChanged: (v) => print(v),  
  onSubmitted: (v) => _search(v),  
  decoration: InputDecoration(  
    labelText: 'ইমেইল',  
    hintText: 'example@mail.com',  
    helperText: 'আমরা কখনো শেয়ার করব না',  
    errorText: _error,  
    prefixIcon: const Icon(Icons.email),  
    suffixIcon: IconButton(  
      icon: const Icon(Icons.clear),  
      onPressed: _controller.clear,  
    ),  
    counterText: '',                                // maxLength কাউন্টার লুকানো  
    filled: true,  
    fillColor: Colors.grey.shade100,  
    border: OutlineInputBorder(borderRadius: BorderRadius.circular(12)),  
    focusedBorder: OutlineInputBorder(  
      borderRadius: BorderRadius.circular(12),  
      borderSide: const BorderSide(color: Colors.teal, width: 2),  
    ),  
    contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),  
  ),  
)
```

১৭.২ পূর্ণাঙ্গ ফর্ম — পেশাদার মান

```
class RegisterPage extends StatefulWidget {  
  const RegisterPage({super.key});
```

```

@override State<RegisterPage> createState() => _RegisterPageState();
}

class _RegisterPageState extends State<RegisterPage> {
  final _formKey = GlobalKey<FormState>();
  final _name = TextEditingController();
  final _email = TextEditingController();
  final _phone = TextEditingController();
  final _pass = TextEditingController();
  final _confirm = TextEditingController();

  bool _obscure = true;
  bool _agreed = false;
  bool _loading = false;

  @override
  void dispose() {
    for (final c in [_name, _email, _phone, _pass, _confirm]) {
      c.dispose();
    }
    super.dispose();
  }

  Future<void> _submit() async {
    // ১. ফিল্ড যাচাই
    if (!_formKey.currentState!.validate()) return;

    // ২. চেকবক্সের মতো ফিল্ড-বহির্ভূত যাচাই
    if (!_agreed) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('শর্তাবলি মেনে নিন')),
      );
      return;
    }

    setState(() => _loading = true);
    try {
      await AuthService.register(
        name: _name.text.trim(),
        email: _email.text.trim(),
        password: _pass.text,
      );
      if (!mounted) return;
      Navigator.pushReplacementNamed(context, '/home');
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('ব্যর্থ: $e'), backgroundColor: Colors.red),
      );
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  @override

```

```
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('নিবন্ধন')),
    body: GestureDetector(
      onTap: () => FocusScope.of(context).unfocus(), // কিবোর্ড বন্ধ
      child: SingleChildScrollView(
        padding: const EdgeInsets.all(20),
        child: Form(
          key: _formKey,
          autovalidateMode: AutovalidateMode.onUserInteraction,
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              TextFormField(
                controller: _name,
                textInputAction: TextInputAction.next,
                decoration: const InputDecoration(
                  labelText: 'পূর্ণ নাম', border: OutlineInputBorder()),
                validator: (v) => (v == null || v.trim().length < 3)
                  ? 'কমপক্ষে ৩ অক্ষরের নাম দিন' : null,
              ),
              const SizedBox(height: 16),
              TextFormField(
                controller: _email,
                keyboardType: TextInputType.emailAddress,
                decoration: const InputDecoration(
                  labelText: 'ইমেইল', border: OutlineInputBorder()),
                validator: (v) {
                  if (v == null || v.isEmpty) return 'ইমেইল দিন';
                  final ok = RegExp(r'^[\w.+-]+@[ \w-]+\.[\w.]+$').hasMatch(v);
                  return ok ? null : 'সঠিক ইমেইল দিন';
                },
              ),
              const SizedBox(height: 16),
              TextFormField(
                controller: _phone,
                keyboardType: TextInputType.phone,
                inputFormatters: [FilteringTextInputFormatter.digitsOnly],
                decoration: const InputDecoration(
                  labelText: 'মোবাইল', hintText: '01XXXXXXXX',
                  border: OutlineInputBorder()),
                validator: (v) =>
                  RegExp(r'^(01)[3-9]\d{8}$').hasMatch(v ?? '')
                  ? null : 'সঠিক মোবাইল নম্বর দিন',
              ),
              const SizedBox(height: 16),
              TextFormField(
                controller: _pass,
                obscureText: _obscure,
                decoration: InputDecoration(
                  labelText: 'পাসওয়ার্ড',
                  border: const OutlineInputBorder(),
```

```

        suffixIcon: IconButton(
          icon: Icon(_obscure
            ? Icons.visibility : Icons.visibility_off),
          onPressed: () => setState(() => _obscure = !_obscure),
        ),
      ),
      validator: (v) {
        if (v == null || v.length < 8) return 'কমপক্ষে ৮ অক্ষর';
        if (!RegExp(r'[A-Z]').hasMatch(v)) return 'একটি বড় হাতের অক্ষর দিন';
        if (!RegExp(r'\d').hasMatch(v)) return 'একটি সংখ্যা দিন';
        return null;
      },
    ),
    const SizedBox(height: 16),

    TextFormField(
      controller: _confirm,
      obscureText: true,
      decoration: const InputDecoration(
        labelText: 'পাসওয়ার্ড নিশ্চিত করুন',
        border: OutlineInputBorder(),
      ),
      validator: (v) =>
        v != _pass.text ? 'পাসওয়ার্ড মিলছে না' : null,
    ),
    const SizedBox(height: 8),

    CheckboxListTile(
      value: _agreed,
      onChanged: (v) => setState(() => _agreed = v ?? false),
      title: const Text('আমি শর্তাবলি মেনে নিচ্ছি'),
      controlAffinity: ListTileControlAffinity.leading,
      contentPadding: EdgeInsets.zero,
    ),
    const SizedBox(height: 16),

    FilledButton(
      onPressed: _loading ? null : _submit,
      style: FilledButton.styleFrom(
        minimumSize: const Size.fromHeight(50)),
      child: _loading
        ? const SizedBox(width: 22, height: 22,
            child: CircularProgressIndicator(
              strokeWidth: 2, color: Colors.white))
        : const Text('নিবন্ধন করুন'),
    ),
  ],
),
),
),
),
),
);
}
}

```

১৭.৩ পুনঃব্যবহারযোগ্য Validator

```
class Validators {
  static String? required(String? v, [String field = 'এই ঘর']) =>
    (v == null || v.trim().isEmpty) ? '$field পূরণ করুন' : null;

  static String? email(String? v) {
    if (v == null || v.isEmpty) return 'ইমেইল দিন';
    return RegExp(r'^[\w.+-]+@[ \w-]+\.[\w.]+$').hasMatch(v)
      ? null : 'সঠিক ইমেইল দিন';
  }

  static String? phoneBD(String? v) =>
    RegExp(r'^(\01)[3-9]\d{8}$').hasMatch(v ?? '')
      ? null : 'সঠিক মোবাইল নম্বর দিন';

  static String? minLength(String? v, int n) =>
    (v == null || v.length < n) ? 'কমপক্ষে $n অক্ষর দিন' : null;

  // একাধিক নিয়ম একসাথে
  static String? Function(String?) combine(
    List<String? Function(String?)> validators) {
    return (value) {
      for (final v in validators) {
        final result = v(value);
        if (result != null) return result;
      }
      return null;
    };
  }
}

// ব্যবহার
TextFormField(
  validator: Validators.combine([
    (v) => Validators.required(v, 'নাম'),
    (v) => Validators.minLength(v, 3),
  ]),
)
```

১৭.৪ অন্যান্য ইনপুট

```
SwitchListTile(value: _dark, onChanged: (v) {}, title: const Text('ডার্ক মোড'))
RadioListTile<String>(value: 'পুরুষ', groupValue: _gender, onChanged: (v) {},
  title: const Text('পুরুষ'))
DropDownButtonFormField<String>(
  value: _city, items: cities.map((c) =>
    DropdownMenuItem(value: c, child: Text(c))).toList(),
  onChanged: (v) => setState(() => _city = v),
  validator: (v) => v == null ? 'শহর বাছুন' : null,
)
```

```
Slider(value: _val, min: 0, max: 100, divisions: 10, onChanged: (v) {})  
RangeSlider(values: _range, min: 0, max: 5000, onChanged: (v) {})
```

```
// তারিখ ও সময়
```

```
final date = await showDatePicker(context: context,  
    initialDate: DateTime.now(),  
    firstDate: DateTime(1950), lastDate: DateTime(2100));  
final time = await showTimePicker(context: context,  
    initialTime: TimeOfDay.now());  
final range = await showDateRangePicker(context: context,  
    firstDate: DateTime(2020), lastDate: DateTime(2030));
```

অধ্যায় ১৮ — থিম, স্টাইল ও রেসপন্সিভ ডিজাইন

১৮.১ সম্পূর্ণ থিম কনফিগারেশন

```
ThemeData buildLightTheme() {
  const seed = Color(0xFF0E7490);

  return ThemeData(
    useMaterial3: true,
    colorScheme: ColorScheme.fromSeed(seedColor: seed),
    fontFamily: 'HindSiliguri',
    scaffoldBackgroundColor: const Color(0xFFF7F9FB),

    appBarTheme: const AppBarTheme(
      centerTitle: true, elevation: 0, scrolledUnderElevation: 2,
      titleTextStyle: TextStyle(fontSize: 18, fontWeight: FontWeight.w600),
    ),

    cardTheme: CardTheme(
      elevation: 2,
      margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
    ),

    filledButtonTheme: FilledButtonThemeData(
      style: FilledButton.styleFrom(
        minimumSize: const Size.fromHeight(50),
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
      ),
    ),

    inputDecorationTheme: InputDecorationTheme(
      filled: true, fillColor: Colors.white,
      border: OutlineInputBorder(borderRadius: BorderRadius.circular(12)),
      contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
    ),

    textTheme: const TextTheme(
      displayLarge: TextStyle(fontSize: 32, fontWeight: FontWeight.bold),
      titleLarge: TextStyle(fontSize: 20, fontWeight: FontWeight.w600),
      bodyMedium: TextStyle(fontSize: 14, height: 1.6),
      labelSmall: TextStyle(fontSize: 11, color: Colors.grey),
    ),

    snackBarTheme: const SnackBarThemeData(
      behavior: SnackBarBehavior.floating,
    ),
  );
}

MaterialApp(
```



```
theme: buildLightTheme(),
darkTheme: ThemeData.dark(useMaterial3: true),
themeMode: themeProvider.mode, // system / light / dark
)
```

থিম ব্যবহারের সঠিক নিয়ম

কোডে সরাসরি `Colors.teal` লিখবেন না। লিখুন `Theme.of(context).colorScheme.primary`। তাহলে ডার্ক মোড ও ব্র্যান্ড পরিবর্তন এক জায়গা থেকেই হবে। `ColorScheme`-এর দরকারি নাম: `primary`, `onPrimary`, `secondary`, `surface`, `error`, `outline`।

১৮.২ নিজস্ব রঙ ও ধ্রুবক

```
class AppColors {
  static const primary = Color(0xFF0E7490);
  static const accent = Color(0xFF4CC9F0);
  static const success = Color(0xFF2E9E5B);
  static const danger = Color(0xFFD32F2F);
  static const textDark = Color(0xFF1C2430);
}

class AppSizes {
  static const double paddingS = 8;
  static const double paddingM = 16;
  static const double paddingL = 24;
  static const double radius = 12;
}

class AppStrings {
  static const appName = 'আমার অ্যাপ';
  static const noInternet = 'ইন্টারনেট সংযোগ নেই';
}
```

১৮.৩ রেসপন্সিভ ডিজাইন

```
class Responsive extends StatelessWidget {
  final Widget mobile;
  final Widget? tablet;
  final Widget? desktop;

  const Responsive({super.key, required this.mobile, this.tablet, this.desktop});

  static bool isMobile(BuildContext c) =>
    MediaQuery.of(c).size.width < 650;
  static bool isTablet(BuildContext c) =>
    MediaQuery.of(c).size.width >= 650 &&
    MediaQuery.of(c).size.width < 1100;
```

```

@override
Widget build(BuildContext context) {
  return LayoutBuilder(builder: (context, constraints) {
    if (constraints.maxWidth >= 1100) return desktop ?? tablet ?? mobile;
    if (constraints.maxWidth >= 650) return tablet ?? mobile;
    return mobile;
  });
}

// ব্যবহার
Responsive(
  mobile: const ProductGrid(columns: 2),
  tablet: const ProductGrid(columns: 3),
  desktop: const ProductGrid(columns: 5),
)

```

স্ক্রিনের তথ্য

```

final mq = MediaQuery.of(context);
mq.size.width           // প্রস্থ
mq.size.height          // উচ্চতা
mq.padding.top          // স্ট্যাটাস বারের উচ্চতা
mq.viewInsets.bottom    // কিবোর্ডের উচ্চতা ★
mq.orientation          // portrait / landscape
mq.textScaler            // ব্যবহারকারীর ফন্ট সাইজ সেটিং
mq.platformBrightness   // ডার্ক মোড চালু কিনা

```

MediaQuery.of(context) পারফরম্যান্স সমস্যা

`MediaQuery.of(context)` ব্যবহার করলে কিবোর্ড ওঠানামা করলেও পুরো widget rebuild হয়। শুধু মাপ দরকার হলে `MediaQuery.sizeOf(context)` ব্যবহার করুন — এটি কেবল মাপ বদলালেই rebuild করে।

অধ্যায় ১৯ — অ্যানিমেশন ও CustomPainter

১৯.১ Implicit Animation — সহজ ও যথেষ্ট

```
// মান বদলালেই নিজে থেকে মসৃণভাবে পরিবর্তন হয়
AnimatedContainer(
  duration: const Duration(milliseconds: 400),
  curve: Curves.easeOutCubic,
  width: _expanded ? 300 : 150,
  height: _expanded ? 200 : 100,
  decoration: BoxDecoration(
    color: _expanded ? Colors.teal : Colors.orange,
    borderRadius: BorderRadius.circular(_expanded ? 24 : 8),
  ),
)

AnimatedOpacity(duration: d, opacity: _visible ? 1 : 0, child: ...)
AnimatedPadding(duration: d, padding: _pad, child: ...)
AnimatedAlign(duration: d, alignment: _align, child: ...)
AnimatedPositioned(duration: d, top: _top, child: ...)
AnimatedDefaultTextStyle(duration: d, style: _style, child: ...)
AnimatedScale(duration: d, scale: _scale, child: ...)
AnimatedRotation(duration: d, turns: _turns, child: ...)

// widget বদলানোর সময় ট্রানজিশন
AnimatedSwitcher(
  duration: const Duration(milliseconds: 300),
  transitionBuilder: (child, animation) =>
    FadeTransition(opacity: animation, child: child),
  child: _loading
    ? const CircularProgressIndicator(key: ValueKey('loading'))
    : Text('সম্পন্ন', key: const ValueKey('done')),
)

// লিস্ট আইটেম যোগ/বাদ হলে অ্যানিমেশন
AnimatedList(
  key: _listKey,
  itemCount: _items.length,
  itemBuilder: (context, i, animation) => SizeTransition(
    sizeFactor: animation,
    child: ListTile(title: Text(_items[i])),
  ),
)

// যেকোনো মানের জন্য সাধারণ অ্যানিমেশন
TweenAnimationBuilder<double>(
  tween: Tween(begin: 0, end: _progress),
  duration: const Duration(milliseconds: 600),
  builder: (context, value, child) =>
```

```
LinearProgressIndicator(value: value),
```

```
)
```

১৯.২ Explicit Animation — পূর্ণ নিয়ন্ত্রণ

```
class PulseIcon extends StatefulWidget {
  const PulseIcon({super.key});
  @override State<PulseIcon> createState() => _PulseIconState();
}

class _PulseIconState extends State<PulseIcon>
  with SingleTickerProviderStateMixin { // ★ mixin লাগবে

  late final AnimationController _controller;
  late final Animation<double> _scale;
  late final Animation<Color?> _color;

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(
      vsync: this,
      duration: const Duration(milliseconds: 900),
    )..repeat(reverse: true); // বারবার আগে-পিছে

    _scale = Tween<double>(begin: 1.0, end: 1.35).animate(
      CurvedAnimation(parent: _controller, curve: Curves.easeInOut),
    );

    _color = ColorTween(begin: Colors.red, end: Colors.pink)
      .animate(_controller);
  }

  @override
  void dispose() {
    _controller.dispose(); // ★ অপরিহার্য
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    // AnimatedBuilder – শুধু ভিতরের অংশ rebuild হয় (দক্ষ)
    return AnimatedBuilder(
      animation: _controller,
      builder: (context, child) => Transform.scale(
        scale: _scale.value,
        child: Icon(Icons.favorite, color: _color.value, size: 48),
      ),
    );
  }
}
```

কন্ট্রোলার মেথড	কাজ
<code>forward()</code>	$0 \rightarrow 1$
<code>reverse()</code>	$1 \rightarrow 0$
<code>repeat(reverse: true)</code>	বারবার আগে-পিছে
<code>stop()</code> / <code>reset()</code>	থামানো / শুরুতে ফেরানো
<code>.value</code>	বর্তমান অবস্থান (0-1)

১৯.৩ Hero — পেজের মধ্যে উড়ন্ত ট্রানজিশন

```
// লিস্ট পেজে
Hero(
  tag: 'product-${product.id}',
  child: Image.network(product.image, height: 100),
)

// ডিটেইল পেজে – একই tag
Hero(
  tag: 'product-${product.id}',
  child: Image.network(product.image, height: 300),
)
```

Hero-এর নিয়ম

একটি স্ক্রিনে একই `tag` দুবার থাকলে এরর হবে। লিস্টে সবসময় অনন্য কিছু (যেমন id) ব্যবহার করুন। এবং Hero-এর ভিতরে `Material` widget (যেমন Text সরাসরি) থাকলে ট্রানজিশনের সময় অদ্ভুত দেখাতে পারে — `Material(type: MaterialType.transparency)` দিয়ে মুড়ে নিন।

১৯.8 CustomPainter — নিজে আঁকা

```
class RingPainter extends CustomPainter {
  final double progress; // ০ থেকে ১
  final Color color;

  RingPainter({required this.progress, required this.color});

  @override
  void paint(Canvas canvas, Size size) {
    final center = Offset(size.width / 2, size.height / 2);
```

```

final radius = size.width / 2 - 8;

// পটভূমির বৃত্ত
final bg = Paint()
  ..color = Colors.grey.shade300
  ..style = PaintingStyle.stroke
  ..strokeWidth = 12;
canvas.drawCircle(center, radius, bg);

// অগ্রগতির চাপ
final fg = Paint()
  ..color = color
  ..style = PaintingStyle.stroke
  ..strokeWidth = 12
  ..strokeCap = StrokeCap.round;

canvas.drawArc(
  Rect.fromCircle(center: center, radius: radius),
  -1.5708, // উপর থেকে শুরু (-90°)
  6.2832 * progress, // 2π × progress
  false, fg,
);
}

@override
bool shouldRepaint(RingPainter old) =>
  old.progress != progress || old.color != color;
}

// ব্যবহার
CustomPaint(
  size: const Size(140, 140),
  painter: RingPainter(progress: 0.72, color: Colors.teal),
  child: const Center(child: Text('৭২%')),
)

```

অ্যানিমেশনের পারফরম্যান্স নিয়ম

১. `AnimatedBuilder` -এর `child` প্যারামিটার ব্যবহার করুন — যে অংশ বদলায় না তা বারবার তৈরি হবে না।
২. অ্যানিমেশনে `Opacity` -র বদলে `FadeTransition`, `Transform` -এর বদলে `ScaleTransition` ব্যবহার করুন — এগুলো বেশি দক্ষ।
৩. `shouldRepaint` -এ সবসময় `true` ফেরত দেবেন না।

পর্ব ৫

ডেটা ও স্টেট

API, স্টেট ম্যানেজমেন্ট, ডেটাবেজ, Firebase ও ডিভাইস ফিচার —
অ্যাপকে সত্যিকারের কাজের উপযোগী করা।

অধ্যায় ২০ — নেটওয়ার্কিং, REST API ও JSON

২০.১ HTTP-এর মৌলিক ধারণা

মেথড	কাজ	উদাহরণ
GET	ডেটা আনা	পণ্যের তালিকা
POST	নতুন তৈরি	নিবন্ধন, অর্ডার
PUT	পুরোটা আপডেট	প্রোফাইল সম্পাদনা
PATCH	অংশবিশেষ আপডেট	শুধু ছবি বদলানো
DELETE	মুছে ফেলা	নোট ডিলিট

কোড	অর্থ ও করণীয়
200 / 201	সফল / তৈরি হয়েছে
204	সফল, কিন্তু কোনো কনটেন্ট নেই (DELETE-এর পরে সাধারণ)
400	রিকোয়েস্ট ভুল — পাঠানো ডেটা যাচাই করুন
401	টোকেন নেই বা মেয়াদ শেষ — লগইনে পাঠান
403	অনুমতি নেই — ভূমিকা (role) যাচাই করুন
404	URL বা রিসোর্স নেই
422	ভ্যালিডেশন ব্যর্থ — সার্ভারের এরর মেসেজ দেখান
500 / 502 / 503	সার্ভারের সমস্যা — "পরে চেষ্টা করুন" দেখান

২০.২ http প্যাকেজ — সব মেথড

```
import 'dart:convert';
import 'package:http/http.dart' as http;

const baseUrl = 'https://api.example.com';

// GET
final res = await http.get(
```



```

Uri.parse('$baseUrl/products?page=1&limit=20'),
headers: {'Authorization': 'Bearer $token'},
);

// POST
final res2 = await http.post(
  Uri.parse('$baseUrl/orders'),
  headers: {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'Authorization': 'Bearer $token',
  },
  body: jsonEncode({'productId': 5, 'quantity': 2}),
);

// PUT / PATCH / DELETE
await http.put(Uri.parse('$baseUrl/users/1'), body: jsonEncode(data));
await http.patch(Uri.parse('$baseUrl/users/1'), body: jsonEncode({'name': 'x'}));
await http.delete(Uri.parse('$baseUrl/notes/3'));

// ফাইল আপলোড (multipart)
final request = http.MultipartRequest('POST', Uri.parse('$baseUrl/upload'))
  ..headers['Authorization'] = 'Bearer $token'
  ..fields['title'] = 'আমার ছবি'
  ..files.add(await http.MultipartFile.fromPath('image', file.path));

final streamed = await request.send();
final response = await http.Response.fromStream(streamed);

```

২০.৩ পেশাদার API Service

```

import 'dart:async';
import 'dart:io';

class ApiException implements Exception {
  final String message;
  final int? statusCode;
  ApiException(this.message, [this.statusCode]);
  @override String toString() => message;
}

class ApiClient {
  static const String _base = 'https://api.example.com';
  static const Duration _timeout = Duration(seconds: 20);
  static String? _token;

  static set token(String? t) => _token = t;

  static Map<String, String> get _headers => {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    if (_token != null) 'Authorization': 'Bearer $_token',
  }
}

```

```

};

static Future<dynamic> get(String path, {Map<String, dynamic>? query}) async {
  final uri = Uri.parse('${_base$path}').replace(
    queryParameters: query?.map((k, v) => MapEntry(k, v.toString())),
  );
  return _send(() => http.get(uri, headers: _headers));
}

static Future<dynamic> post(String path, Map<String, dynamic> body) async {
  return _send(() => http.post(
    Uri.parse('${_base$path}'),
    headers: _headers,
    body: jsonEncode(body),
  ));
}

// সব রিকোয়েস্টের কেন্দ্রীয় হ্যান্ডলার
static Future<dynamic> _send(Future<http.Response> Function() request) async {
  try {
    final res = await request().timeout(_timeout);

    final body = res.body.isEmpty ? null : jsonDecode(res.body);

    if (res.statusCode >= 200 && res.statusCode < 300) {
      return body;
    }

    // সার্ভারের এরর মেসেজ থাকলে সেটিই দেখান
    final msg = (body is Map && body['message'] != null)
      ? body['message'].toString()
      : _defaultMessage(res.statusCode);

    throw ApiException(msg, res.statusCode);

  } on SocketException {
    throw ApiException('ইন্টারনেট সংযোগ নেই');
  } on TimeoutException {
    throw ApiException('সার্ভার সাড়া দিচ্ছে না, আবার চেষ্টা করুন');
  } on FormatException {
    throw ApiException('সার্ভার থেকে ভুল ডেটা এসেছে');
  }
}

static String _defaultMessage(int code) => switch (code) {
  400 => 'অনুরোধটি সঠিক নয়',
  401 => 'আবার লগইন করুন',
  403 => 'আপনার অনুমতি নেই',
  404 => 'কিছু পাওয়া যায়নি',
  500 => 'সার্ভারে সমস্যা হয়েছে',
  _ => 'কিছু একটা ভুল হয়েছে ($code)',
};
}

```

২০.৪ Repository স্তর

```
class ProductRepository {
    Future<List<Product>> getProducts({int page = 1, String? search}) async {
        final data = await ApiClient.get('/products', query: {
            'page': page,
            if (search != null) 'q': search,
        });

        return (data['data'] as List)
            .map((json) => Product.fromJson(json))
            .toList();
    }

    Future<Product> getById(int id) async {
        final data = await ApiClient.get('/products/$id');
        return Product.fromJson(data);
    }

    Future<void> createOrder(int productId, int qty) async {
        await ApiClient.post('/orders', {'product_id': productId, 'quantity': qty});
    }
}
```

কেন এত স্তর

ApiClient শুধু HTTP জানে · **Repository** ব্যবসায়িক ডেটা জানে · **UI** শুধু Repository ডাকে। এতে API বদলালে (বা মক ডেটা দিয়ে টেস্ট করতে চাইলে) শুধু একটি ফাইল বদলালেই হয়, ৫০টা স্ক্রিন ছুঁতে হয় না।

২০.৫ JSON পার্সিং — জটিল কাঠামো

```
// নেস্টেড অবজেক্ট ও লিস্টসহ
class Order {
    final int id;
    final String status;
    final Customer customer;           // নেস্টেড অবজেক্ট
    final List<OrderItem> items;       // অবজেক্টের লিস্ট
    final DateTime createdAt;
    final double? discount;           // থাকতেও পারে, নাও পারে

    Order({
        required this.id, required this.status, required this.customer,
        required this.items, required this.createdAt, this.discount,
    });

    factory Order.fromJson(Map<String, dynamic> json) => Order(
        id: json['id'] as int,
        status: json['status'] as String? ?? 'unknown',
```

```

// নেস্টেড অবজেক্ট
customer: Customer.fromJson(json['customer'] ?? {}),

// অবজেক্টের লিস্ট
items: (json['items'] as List? ?? [])
    .map((e) => OrderItem.fromJson(e as Map<String, dynamic>))
    .toList(),

// তারিখ – সবসময় tryParse
createdAt: DateTime.tryParse(json['created_at'] ?? '') ?? DateTime.now(),

// সংখ্যা – int নাকি double অনিশ্চিত হলে num দিয়ে নিন
discount: (json['discount'] as num?)?.toDouble(),
);

double get total =>
    items.fold(0.0, (sum, i) => sum + i.price * i.quantity) - (discount ?? 0);
}

```

JSON পার্সিং-এ নতুনদের ৫টি ভুল

১. `json['price']` সার্ভারে কখনো `100` (int) কখনো `100.0` (double) — সবসময় `(json['price'] as num).toDouble()` লিখুন।
২. ফিল্ড না থাকলে `null` আসে — `??` ডিফল্ট দিন।
৩. তারিখে `DateTime.parse` ব্যবহার করলে ভুল ফরম্যাটে ক্র্যাশ — `tryParse` ব্যবহার করুন।
৪. খালি লিস্ট নয়, `null` আসতে পারে — `as List? ?? []`।
৫. বড় JSON মূল থেড়ে পার্স করলে UI আটকায় — `compute()` ব্যবহার করুন।

২০.৬ FutureBuilder ও পূর্ণ অবস্থা ব্যবস্থাপনা

```

FutureBuilder<List<Product>>(
    future: _future, // ★ build() এ কল করবেন না, initState এ রাখুন
    builder: (context, snapshot) {
        switch (snapshot.connectionState) {
            case ConnectionState.waiting:
                return const Center(child: CircularProgressIndicator());

            case ConnectionState.done:
                if (snapshot.hasError) {
                    return ErrorView(
                        message: snapshot.error.toString(),
                        onRetry: () => setState(() => _future = _repo.getProducts()),
                    );
                }
                final items = snapshot.data ?? [];
                if (items.isEmpty) return const EmptyView('কোনো পণ্য নেই');
                return ProductList(items: items);
        }
    }
)

```

```
    default:  
      return const SizedBox.shrink();  
  }  
},  
)
```

অধ্যায় ২১ — স্টেট ম্যানেজমেন্ট সম্পূর্ণ

২১.১ কোনটি কখন

পদ্ধতি	শেখার কষ্ট	উপযুক্ত
setState	খুব সহজ	একটি widget-এর নিজস্ব state (চেকবক্স, ফর্ম)
InheritedWidget	মাঝারি	ভিত্তি বোঝার জন্য; সরাসরি কম ব্যবহৃত
Provider	সহজ	★ নতুনদের জন্য সেরা, ছোট-মাঝারি অ্যাপ
Riverpod	মাঝারি	Provider-এর উন্নত রূপ; compile-safe, টেস্ট সহজ
Bloc	কঠিন	বড় টিম, কঠোর কাঠামো, এন্টারপ্রাইজ
GetX	সহজ	দ্রুত কাজ, কিন্তু খারাপ অভ্যাস তৈরি করে — এড়িয়ে চলুন

২১.২ InheritedWidget — সবার ভিত্তি

```
class CounterScope extends InheritedWidget {
  final int count;
  final VoidCallback increment;

  const CounterScope({
    super.key, required this.count,
    required this.increment, required super.child,
  });

  static CounterScope of(BuildContext context) {
    final result = context.dependOnInheritedWidgetOfExactType<CounterScope>();
    assert(result != null, 'CounterScope পাওয়া যায়নি');
    return result!;
  }

  @override
  bool updateShouldNotify(CounterScope old) => old.count != count;
}

// ব্যবহার - CounterScope.of(context).count
```

Provider, Riverpod, Bloc — সবাই ভিতরে এটিই ব্যবহার করে। এটি বোঝা থাকলে বাকিগুলো সহজ লাগবে।

২১.৩ Provider — সম্পূর্ণ ব্যবহারিক

```
flutter pub add provider
```

```
class CartProvider extends ChangeNotifier {
  final List<CartItem> _items = [];
  bool _isLoading = false;
  String? _error;

  // শুধু পড়ার getter
  List<CartItem> get items => List.unmodifiable(_items);
  bool get isLoading => _isLoading;
  String? get error => _error;
  int get count => _items.fold(0, (s, i) => s + i.quantity);
  double get subtotal => _items.fold(0.0, (s, i) => s + i.product.price * i.quantity);
  double get total => subtotal > 1000 ? subtotal : subtotal + 60; // ডেলিভারি
  bool get isEmpty => _items.isEmpty;

  void add(Product p, [int qty = 1]) {
    final index = _items.indexWhere((i) => i.product.id == p.id);
    if (index >= 0) {
      _items[index].quantity += qty;
    } else {
      _items.add(CartItem(product: p, quantity: qty));
    }
    notifyListeners(); // ★ ভুলবেন না
  }

  void updateQuantity(int productId, int qty) {
    final i = _items.indexWhere((e) => e.product.id == productId);
    if (i < 0) return;
    if (qty <= 0) {
      _items.removeAt(i);
    } else {
      _items[i].quantity = qty;
    }
    notifyListeners();
  }

  void remove(int productId) {
    _items.removeWhere((i) => i.product.id == productId);
    notifyListeners();
  }

  void clear() { _items.clear(); notifyListeners(); }

  // async কাজ – লোডিং ও এররসহ
  Future<bool> checkout() async {
    _isLoading = true; _error = null;
    notifyListeners();

    try {
```

```

        await OrderRepository().placeOrder(_items);
        _items.clear();
        return true;
    } catch (e) {
        _error = e.toString();
        return false;
    } finally {
        _isLoading = false;
        notifyListeners();
    }
}
}
}

```

অ্যাপে যুক্ত করা

```

void main() {
  runApp(
    MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (_) => CartProvider()),
        ChangeNotifierProvider(create: (_) => AuthProvider()),
        ChangeNotifierProvider(create: (_) => ThemeProvider()),

        // সাধারণ অবজেক্ট (ChangeNotifier নয়)
        Provider(create: (_) => ProductRepository()),

        // অন্য provider-এর উপর নির্ভরশীল
        ChangeNotifierProxyProvider<AuthProvider, OrderProvider>(
          create: (_) => OrderProvider(),
          update: (_, auth, orders) => orders!..updateToken(auth.token),
        ),
      ],
      child: const MyApp(),
    ),
  );
}

```

UI-তে ব্যবহার — চারটি উপায়

```

// ১. watch – মান পড়া + বদলালে rebuild
final cart = context.watch<CartProvider>();
Text('${cart.count} টি পণ্য');

// ২. read – শুধু কাজ করানো, rebuild নয় (onPressed-এ ★)
onPressed: () => context.read<CartProvider>().add(product),

// ৩. select – শুধু একটি মান দেখা (সবচেয়ে দক্ষ ★)
final count = context.select<CartProvider, int>((c) => c.count);

// ৪. Consumer – গাছের একটি অংশ মাত্র rebuild
Consumer<CartProvider>(
  builder: (context, cart, child) => Badge(

```



```

    label: Text('${cart.count}'),
    child: child, // এই অংশ rebuild হবে না ★
  ),
  child: const Icon(Icons.shopping_cart),
)

```

Provider-এর ৫টি সাধারণ ভুল

১. `notifyListeners()` ভুলে যাওয়া → UI বদলায় না।
২. `onPressed` -এ `watch` ব্যবহার → অপয়োজনীয় rebuild। `read` ব্যবহার করুন।
৩. `initState` -এ `watch` → এরর। `read` বা `listen: false` ব্যবহার করুন।
৪. Provider-কে ব্যবহারকারী widget-এর নিচে বসানো → "Could not find the correct Provider"।
৫. লিস্টের ভিতরের অবজেক্ট সরাসরি বদলানো (`_items[0].qty++`) কিন্তু `notify` না করা।

২১.৪ Riverpod — পরবর্তী ধাপ

```
flutter pub add flutter_riverpod
```

```

// Provider সংজ্ঞা – global কিন্তু নিরাপদ
final cartProvider = ChangeNotifierProvider((ref) => CartNotifier());

final productsProvider = FutureProvider<List<Product>>((ref) async {
  return ref.watch(repositoryProvider).getProducts();
});

final filteredProductsProvider = Provider<List<Product>>((ref) {
  final products = ref.watch(productsProvider).value ?? [];
  final query = ref.watch(searchQueryProvider);
  return products.where((p) => p.title.contains(query)).toList();
});

// অ্যাপ মোড়ানো
void main() => runApp(const ProviderScope(child: MyApp()));

// ব্যবহার – ConsumerWidget
class ProductPage extends ConsumerWidget {
  const ProductPage({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final async = ref.watch(productsProvider);

    return async.when(
      loading: () => const Center(child: CircularProgressIndicator()),
      error: (e, s) => Center(child: Text('এরর: $e')),
      data: (products) => ListView.builder(
        itemCount: products.length,
        itemBuilder: (c, i) => ProductTile(products[i]),
      ),
    );
  }
}

```

```

    ),
  );
}
}

```

Riverpod-এর সুবিধা: `BuildContext` লাগে না · compile টাইমেই ভুল ধরা পড়ে · `.when()` দিয়ে লোডিং/এরর/ডেটা একসাথে সামলানো যায় · টেস্ট করা সহজ।

২১.৫ Bloc — সংক্ষিপ্ত পরিচয়

```
flutter pub add flutter_bloc
```

```

// Event – কী ঘটল
sealed class CartEvent {}
class AddToCart extends CartEvent { final Product p; AddToCart(this.p); }
class RemoveFromCart extends CartEvent { final int id; RemoveFromCart(this.id); }

// State – এখন অবস্থা কী
class CartState {
  final List<CartItem> items;
  final bool isLoading;
  const CartState({this.items = const [], this.isLoading = false});

  CartState copyWith({List<CartItem>? items, bool? isLoading}) => CartState(
    items: items ?? this.items,
    isLoading: isLoading ?? this.isLoading,
  );
}

// Bloc – Event নিয়ে State বানায়
class CartBloc extends Bloc<CartEvent, CartState> {
  CartBloc() : super(const CartState()) {
    on<AddToCart>((event, emit) {
      final updated = [...state.items, CartItem(product: event.p, quantity: 1)];
      emit(state.copyWith(items: updated));
    });

    on<RemoveFromCart>((event, emit) {
      emit(state.copyWith(
        items: state.items.where((i) => i.product.id != event.id).toList(),
      ));
    });
  }
}

// UI
BlocBuilder<CartBloc, CartState>(
  builder: (context, state) => Text('${state.items.length}'),
)
context.read<CartBloc>().add(AddToCart(product));

```

আমার পরামর্শ

Provider দিয়ে শুরু করুন — ২টি প্রজেক্ট বানিয়ে ভালোভাবে বুঝে নিন। তারপর **Riverpod** শিখুন (Provider জানা থাকলে ১ দিনেই হবে)। **Bloc** শিখুন যখন এমন চাকরি পাবেন যেখানে এটি ব্যবহার হয় — আগে নয়। তিনটি একসাথে শেখার চেষ্টা করলে কোনোটিই ভালো হবে না।

অধ্যায় ২২ — লোকাল স্টোরেজ ও ডেটাবেজ

২২.১ কোনটি কখন

প্যাকেজ	উপযুক্ত
shared_preferences	ছোট key-value — সেটিংস, টোকেন, ফ্ল্যাগ
flutter_secure_storage	সংবেদনশীল ডেটা — পাসওয়ার্ড, রিফ্রেশ টোকেন (এনক্রিপ্টেড)
sqflite	সম্পর্কযুক্ত ডেটা, জটিল কোয়েরি, হাজারো রেকর্ড
hive / isar	দ্রুত NoSQL, SQL না জেনেই অবজেক্ট সেভ
path_provider	ফাইল সেভ করার ফোল্ডার খুঁজে পাওয়া

২২.২ shared_preferences — মোড়কসহ

```
class LocalStorage {
  static late SharedPreferences _prefs;

  // main() এ একবার কল করুন
  static Future<void> init() async {
    _prefs = await SharedPreferences.getInstance();
  }

  // টোকেন
  static String? get token => _prefs.getString('token');
  static Future<void> setToken(String v) => _prefs.setString('token', v);

  // থিম
  static bool get isDark => _prefs.getBool('isDark') ?? false;
  static Future<void> setDark(bool v) => _prefs.setBool('isDark', v);

  // অবজেক্ট সেভ (JSON করে)
  static Future<void> saveUser(User u) =>
    _prefs.setString('user', jsonEncode(u.toJson()));

  static User? get user {
    final s = _prefs.getString('user');
    return s == null ? null : User.fromJson(jsonDecode(s));
  }

  static Future<void> logout() async {
    await _prefs.remove('token');
    await _prefs.remove('user');
  }
}
```

```
// main.dart
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await LocalStorage.init();
  runApp(const MyApp());
}
```

২২.৩ sqflite — সম্পূর্ণ CRUD

```
flutter pub add sqflite path
```

```
class DatabaseHelper {
  static final DatabaseHelper instance = DatabaseHelper._();
  DatabaseHelper._();
  static Database? _db;

  Future<Database> get database async {
    _db ??= await _init();
    return _db!;
  }

  Future<Database> _init() async {
    final path = join(await getDatabasesPath(), 'app.db');

    return openDatabase(
      path,
      version: 2,
      onCreate: (db, v) async {
        await db.execute('''
          CREATE TABLE notes (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT NOT NULL,
            body TEXT,
            category TEXT DEFAULT 'general',
            isPinned INTEGER DEFAULT 0,
            createdAt TEXT NOT NULL
          )
        ''');
        await db.execute('CREATE INDEX idx_category ON notes(category)');
      },

      // অ্যাপ আপডেটে ডেটাবেজের গঠন বদলানো ★
      onUpgrade: (db, oldV, newV) async {
        if (oldV < 2) {
          await db.execute('ALTER TABLE notes ADD COLUMN isPinned INTEGER DEFAULT 0');
        }
      },
    );
  }
}
```

```

// CREATE
Future<int> insert(Note note) async {
    final db = await database;
    return db.insert('notes', note.toMap(),
        conflictAlgorithm: ConflictAlgorithm.replace);
}

// READ সব
Future<List<Note>> getAll({String? category}) async {
    final db = await database;
    final maps = await db.query(
        'notes',
        where: category != null ? 'category = ?' : null,
        whereArgs: category != null ? [category] : null,
        orderBy: 'isPinned DESC, createdAt DESC',
    );
    return maps.map(Note.fromMap).toList();
}

// READ একটি
Future<Note?> getById(int id) async {
    final db = await database;
    final maps = await db.query('notes', where: 'id = ?', whereArgs: [id],
        limit: 1);
    return maps.isEmpty ? null : Note.fromMap(maps.first);
}

// UPDATE
Future<int> update(Note note) async {
    final db = await database;
    return db.update('notes', note.toMap(),
        where: 'id = ?', whereArgs: [note.id]);
}

// DELETE
Future<int> delete(int id) async {
    final db = await database;
    return db.delete('notes', where: 'id = ?', whereArgs: [id]);
}

// SEARCH
Future<List<Note>> search(String q) async {
    final db = await database;
    final maps = await db.query('notes',
        where: 'title LIKE ? OR body LIKE ?', whereArgs: ['%$q%', '%$q%']);
    return maps.map(Note.fromMap).toList();
}

// কাঁচা SQL – জটিল কোয়েরির জন্য
Future<List<Map<String, dynamic>>> countByCategory() async {
    final db = await database;
    return db.rawQuery(
        'SELECT category, COUNT(*) as total FROM notes GROUP BY category');
}

```

```
// একসাথে অনেক কাজ – অনেক দ্রুত ★
Future<void> insertMany(List<Note> notes) async {
  final db = await database;
  final batch = db.batch();
  for (final n in notes) {
    batch.insert('notes', n.toMap());
  }
  await batch.commit(noResult: true);
}
}
```

মডেলে toMap ও fromMap

```
class Note {
  final int? id;
  final String title;
  final String? body;
  final bool isPinned;
  final DateTime createdAt;

  Note({this.id, required this.title, this.body,
    this.isPinned = false, required this.createdAt});

  // SQLite-এ bool ও DateTime নেই – রূপান্তর করতে হয়
  Map<String, dynamic> toMap() => {
    if (id != null) 'id': id,
    'title': title,
    'body': body,
    'isPinned': isPinned ? 1 : 0,
    'createdAt': createdAt.toIso8601String(),
  };

  factory Note.fromMap(Map<String, dynamic> m) => Note(
    id: m['id'] as int?,
    title: m['title'] as String,
    body: m['body'] as String?,
    isPinned: (m['isPinned'] as int? ?? 0) == 1,
    createdAt: DateTime.parse(m['createdAt'] as String),
  );
}
```

২২.৪ Hive — SQL ছাড়াই

```
flutter pub add hive hive_flutter
```

```
void main() async {
  await Hive.initFlutter();
  await Hive.openBox('settings');
  runApp(const MyApp());
}
```

```

final box = Hive.box('settings');
box.put('theme', 'dark');
final theme = box.get('theme', defaultValue: 'light');
box.delete('theme');

// রিয়েলটাইম আপডেট
ValueListenableBuilder(
  valueListenable: box.listenable(),
  builder: (context, Box box, _) => Text(box.get('theme') ?? 'light'),
)

```

২২.৫ ফাইল সেভ করা

```

import 'package:path_provider/path_provider.dart';

Future<File> _localFile(String name) async {
  final dir = await getApplicationDocumentsDirectory();
  return File('${dir.path}/${name}');
}

// লেখা
final file = await _localFile('data.json');
await file.writeAsString(jsonEncode(data));

// পড়া
if (await file.exists()) {
  final content = await file.readAsString();
  final data = jsonDecode(content);
}

```

ফোল্ডারের ধরন: `getApplicationDocumentsDirectory()` — স্থায়ী ডেটা (ব্যাকআপ হয়) · `getTemporaryDirectory()` — ক্যাশ (OS মুছে দিতে পারে) · `getExternalStorageDirectory()` — Android SD কার্ড।

অধ্যায় ২৩ — Firebase সম্পূর্ণ

২৩.১ সেটআপ

```
# ১. Firebase CLI
npm install -g firebase-tools
firebase login

# ২. FlutterFire CLI
dart pub global activate flutterfire_cli

# ৩. প্রজেক্টে কনফিগার (firebase_options.dart তৈরি হবে)
flutterfire configure

# ৪. প্যাকেজ
flutter pub add firebase_core firebase_auth cloud_firestore firebase_storage
```

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const MyApp());
}
```

২৩.২ Authentication

```
class AuthService {
  final _auth = FirebaseAuth.instance;

  User? get currentUser => _auth.currentUser;
  Stream<User?> get authState => _auth.authStateChanges(); // ★

  Future<User?> register(String email, String pass, String name) async {
    try {
      final cred = await _auth.createUserWithEmailAndPassword(
        email: email, password: pass);
      await cred.user?.updateDisplayName(name);

      // Firestore-এ প্রোফাইল তৈরি
      await FirebaseFirestore.instance
        .collection('users').doc(cred.user!.uid).set({
          'name': name, 'email': email,
          'createdAt': FieldValue.serverTimestamp(),
        });
      return cred.user;
    } catch (e) {
      // ...
    }
  }
}
```

```

    } on FirebaseAuthException catch (e) {
      throw _mapError(e.code);
    }
  }

Future<User?> login(String email, String pass) async {
  try {
    final cred = await _auth.signInWithEmailAndPassword(
      email: email, password: pass);
    return cred.user;
  } on FirebaseAuthException catch (e) {
    throw _mapError(e.code);
  }
}

Future<void> logout() => _auth.signOut();
Future<void> resetPassword(String email) =>
  _auth.sendPasswordResetEmail(email: email);

String _mapError(String code) => switch (code) {
  'user-not-found'      => 'এই ইমেইলে কোনো অ্যাকাউন্ট নেই',
  'wrong-password'      => 'পাসওয়ার্ড ভুল',
  'email-already-in-use'=> 'এই ইমেইল আগেই ব্যবহৃত',
  'weak-password'       => 'পাসওয়ার্ড খুব দুর্বল',
  'invalid-email'       => 'ইমেইল সঠিক নয়',
  'network-request-failed' => 'ইন্টারনেট সংযোগ নেই',
  _ => 'সমস্যা হয়েছে, আবার চেষ্টা করুন',
};
}

// লগইন অবস্থা অনুযায়ী স্বয়ংক্রিয় স্ক্রিন ★
StreamBuilder<User?> (
  stream: FirebaseAuth.instance.authStateChanges(),
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const SplashScreen();
    }
    return snapshot.hasData ? const HomePage() : const LoginPage();
  },
);
)

```

২৩.৩ Firestore — CRUD ও রিয়েলটাইম

```

final db = FirebaseFirestore.instance;

// CREATE – স্বয়ংক্রিয় id
final ref = await db.collection('notes').add({
  'title': 'আমার নোট',
  'body': 'বিস্তারিত',
  'userId': uid,
  'createdAt': FieldValue.serverTimestamp(),
});

```

```

// CREATE – নিজের id
await db.collection('users').doc(uid).set({'name': 'রহিম'});

// READ একবার
final doc = await db.collection('notes').doc(id).get();
if (doc.exists) print(doc.data());

// READ রিয়েলটাইম ★ – ডেটা বদলালেই UI আপডেট
StreamBuilder<QuerySnapshot>(
  stream: db.collection('notes')
    .where('userId', isEqualTo: uid)
    .orderBy('createdAt', descending: true)
    .limit(50)
    .snapshots(),
  builder: (context, snapshot) {
    if (snapshot.hasError) return Text('এরর: ${snapshot.error}');
    if (!snapshot.hasData) return const CircularProgressIndicator();

    final docs = snapshot.data!.docs;
    if (docs.isEmpty) return const Text('কোনো নোট নেই');

    return ListView.builder(
      itemCount: docs.length,
      itemBuilder: (c, i) {
        final data = docs[i].data() as Map<String, dynamic>;
        return ListTile(
          title: Text(data['title'] ?? ''),
          subtitle: Text(data['body'] ?? ''),
          onTap: () => _edit(docs[i].id),
        );
      },
    );
  },
);

// UPDATE
await db.collection('notes').doc(id).update({'title': 'নতুন শিরোনাম'});
await db.collection('posts').doc(id).update({
  'likes': FieldValue.increment(1),
  'likedBy': FieldValue.arrayUnion([uid]),
});

// DELETE
await db.collection('notes').doc(id).delete();

// একসাথে অনেক কাজ (সব হবে বা কিছুই হবে না)
final batch = db.batch();
batch.update(db.collection('users').doc(uid), {'points': 100});
batch.delete(db.collection('carts').doc(uid));
await batch.commit();

```

Firestore নিরাপত্তা নিয়ম — অবশ্যই সেট করুন

ডিফল্ট test mode-এ যে কেউ আপনার সব ডেটা পড়তে ও মুছতে পারে। Firebase Console → Firestore → Rules-এ যান:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /notes/{noteId} {
      allow read, write: if request.auth != null
                          && request.auth.uid == resource.data.userId;
      allow create: if request.auth != null;
    }
  }
}
```

২৩.৪ Storage ও Messaging

```
// ছবি আপলোড
Future<String> uploadImage(File file, String uid) async {
  final ref = FirebaseStorage.instance
    .ref('avatars/${uid}/${DateTime.now().millisecondsSinceEpoch}.jpg');

  final task = ref.putFile(file);

  task.snapshotEvents.listen((s) {
    final progress = s.bytesTransferred / s.totalBytes;
    print('আপলোড: ${(progress * 100).toStringAsFixed(0)}%');
  });

  await task;
  return ref.getDownloadURL();
}

// পুশ নোটিফিকেশন
final messaging = FirebaseMessaging.instance;
await messaging.requestPermission();
final token = await messaging.getToken(); // সার্ভারে পাঠান

// অ্যাপ খোলা অবস্থায়
FirebaseMessaging.onMessage.listen((message) {
  print('বার্তা: ${message.notification?.title}');
});

// নোটিফিকেশনে ক্লিক করে অ্যাপ খুললে
FirebaseMessaging.onMessageOpenedApp.listen((message) {
  Navigator.pushNamed(context, '/detail',
```

```
arguments: message.data['id']);  
});
```

অধ্যায় ২৪ — ডিভাইস ফিচার ও পারমিশন

২৪.১ পারমিশন ঘোষণা

Android — `android/app/src/main/AndroidManifest.xml` :

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.CAMERA" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name="android.permission.READ_MEDIA_IMAGES" />
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
```

iOS — `ios/Runner/Info.plist` (কারণ লিখতেই হবে, না হলে App Store বাতিল করবে):

```
<key>NSCameraUsageDescription</key>
<string>প্রোফাইল ছবি তুলতে ক্যামেরা প্রয়োজন</string>
<key>NSLocationWhenInUseUsageDescription</key>
<string>কাছের দোকান দেখাতে অবস্থান প্রয়োজন</string>
<key>NSPhotoLibraryUsageDescription</key>
<string>ছবি বাছাই করতে গ্যালারি প্রয়োজন</string>
```

২৪.২ পারমিশন চাওয়ার সঠিক পদ্ধতি

```
import 'package:permission_handler/permission_handler.dart';

Future<bool> ensurePermission(Permission permission, BuildContext context,
  String reason) async {
  var status = await permission.status;

  if (status.isGranted) return true;

  if (status.isDenied) {
    // চাওয়ার আগে কেন লাগবে তা বুঝিয়ে বলুন – অনুমোদনের হার বাড়ে
    final proceed = await showDialog<bool>(
      context: context,
      builder: (c) => AlertDialog(
        title: const Text('অনুমতি প্রয়োজন'),
        content: Text(reason),
        actions: [
          TextButton(onPressed: () => Navigator.pop(c, false),
            child: const Text('পরে')),
          FilledButton(onPressed: () => Navigator.pop(c, true),
            child: const Text('ঠিক আছে')),
        ],
      ),
    );
  }
}
```

```

    if (proceed != true) return false;

    status = await permission.request();
    return status.isGranted;
  }

  if (status.isPermanentlyDenied) {
    // ব্যবহারকারী "আর জিজেস কোরো না" দিয়েছে – সেটিংসে পাঠান
    await showDialog(
      context: context,
      builder: (c) => AlertDialog(
        title: const Text('সেটিংস থেকে চালু করুন'),
        content: Text(reason),
        actions: [
          TextButton(onPressed: () => Navigator.pop(c),
            child: const Text('বাতিল')),
          FilledButton(
            onPressed: () { Navigator.pop(c); openAppSettings(); },
            child: const Text('সেটিংস'),
          ),
        ],
      ),
    );
    return false;
  }

  return false;
}

```

২৪.৩ ক্যামেরা ও গ্যালারি

```

import 'package:image_picker/image_picker.dart';

Future<File?> pickImage(ImageSource source) async {
  final picked = await ImagePicker().pickImage(
    source: source,
    imageQuality: 75, // আপলোড সাইজ কমাতে
    maxWidth: 1200,
    preferredCameraDevice: CameraDevice.front,
  );
  return picked == null ? null : File(picked.path);
}

// একাধিক ছবি
final images = await ImagePicker().pickMultiImage();

// ভিডিও
final video = await ImagePicker().pickVideo(source: ImageSource.camera);

// বাছাইয়ের শিট
void showPickerSheet(BuildContext context) {
  showModalBottomSheet(

```

```

context: context,
builder: (_) => SafeArea(
  child: Column(mainAxisSize: MainAxisSize.min, children: [
    ListTile(
      leading: const Icon(Icons.camera_alt),
      title: const Text('ক্যামেরা'),
      onTap: () { Navigator.pop(context); pickImage(ImageSource.camera); },
    ),
    ListTile(
      leading: const Icon(Icons.photo_library),
      title: const Text('গ্যালারি'),
      onTap: () { Navigator.pop(context); pickImage(ImageSource.gallery); },
    ),
  ]),
),
);
}

```

২৪.৪ অবস্থান (GPS)

```

import 'package:geolocator/geolocator.dart';

Future<Position> getCurrentLocation() async {
  if (!await Geolocator.isLocationServiceEnabled()) {
    throw Exception('অনুগ্রহ করে GPS চালু করুন');
  }

  var permission = await Geolocator.checkPermission();
  if (permission == LocationPermission.denied) {
    permission = await Geolocator.requestPermission();
    if (permission == LocationPermission.denied) {
      throw Exception('অবস্থানের অনুমতি দেওয়া হয়নি');
    }
  }
  if (permission == LocationPermission.deniedForever) {
    throw Exception('সেটিংস থেকে অবস্থানের অনুমতি দিন');
  }

  return Geolocator.getCurrentPosition(
    desiredAccuracy: LocationAccuracy.high,
    timeLimit: const Duration(seconds: 15),
  );
}

// দুই বিন্দুর দূরত্ব (মিটারে)
final distance = Geolocator.distanceBetween(lat1, lon1, lat2, lon2);

// লাইভ ট্র্যাকিং
final sub = Geolocator.getPositionStream(
  locationSettings: const LocationSettings(
    accuracy: LocationAccuracy.high,
    distanceFilter: 10, // ১০ মিটার সরলে আপডেট
  ),
);

```



```
),  
) .listen((position) => print('${position.latitude}, ${position.longitude}'));
```

২৪.৫ অন্যান্য দরকারি ফিচার

```
// লিংক, ফোন কল, ইমেইল খোলা  
import 'package:url_launcher/url_launcher.dart';  
await launchUrl(Uri.parse('https://flutter.dev'),  
  mode: LaunchMode.externalApplication);  
await launchUrl(Uri.parse('tel:+8801711111111'));  
await launchUrl(Uri.parse('mailto:a@b.com?subject=হ্যালো'));  
await launchUrl(Uri.parse('https://wa.me/8801711111111'));  
  
// ইন্টারনেট আছে কিনা  
import 'package:connectivity_plus/connectivity_plus.dart';  
final result = await Connectivity().checkConnectivity();  
final hasNet = !result.contains(ConnectivityResult.none);  
  
Connectivity().onConnectivityChanged.listen((result) {  
  if (result.contains(ConnectivityResult.none)) {  
    // "ইন্টারনেট নেই" ব্যানার দেখান  
  }  
});  
  
// লোকাল নোটিফিকেশন  
import 'package:flutter_local_notifications/flutter_local_notifications.dart';  
final plugin = FlutterLocalNotificationsPlugin();  
await plugin.initialize(const InitializationSettings(  
  android: AndroidInitializationSettings('@mipmap/ic_launcher'),  
));  
await plugin.show(0, 'শিরোনাম', 'বার্তা',  
  const NotificationDetails(  
    android: AndroidNotificationDetails('main', 'প্রধান',  
      importance: Importance.high),  
  ));  
  
// ডিভাইসের তথ্য  
import 'package:device_info_plus/device_info_plus.dart';  
final android = await DeviceInfoPlugin().androidInfo;  
print('${android.model}, Android ${android.version.release}');  
  
// অ্যাপের সংস্করণ  
import 'package:package_info_plus/package_info_plus.dart';  
final info = await PackageInfo.fromPlatform();  
print('সংস্করণ ${info.version} (${info.buildNumber})');  
  
// শেয়ার  
import 'package:share_plus/share_plus.dart';  
await Share.share('এই অ্যাপটি দেখুন: https://...');
```

পর্ব ৬

পেশাদার ডেভেলপমেন্ট

আর্কিটেকচার, টেস্টিং, পারফরম্যান্স ও রিলিজ — শখের কোড
থেকে চাকরির মানের কোডে যাওয়ার পথ।

অধ্যায় ২৫ — আর্কিটেকচার ও ক্লিন কোড

২৫.১ ফোল্ডার কাঠামো — দুটি পদ্ধতি

```
// পদ্ধতি ১ – স্তর অনুযায়ী (ছোট অ্যাপে ভালো)
lib/
├─ main.dart
├─ models/           product.dart, user.dart, order.dart
├─ screens/          home_screen.dart, login_screen.dart
├─ widgets/          product_card.dart, custom_button.dart
├─ services/         api_client.dart, auth_service.dart
├─ providers/        cart_provider.dart, theme_provider.dart
├─ utils/            validators.dart, formatters.dart
└─ constants/        colors.dart, strings.dart, sizes.dart

// পদ্ধতি ২ – ফিচার অনুযায়ী (★ বড় অ্যাপে অনেক ভালো)
lib/
├─ main.dart
├─ core/
│   ├── constants/
│   ├── network/     api_client.dart, interceptors.dart
│   ├── theme/
│   ├── utils/
│   └── widgets/     সব ফিচারে ব্যবহৃত widget
└─ features/
    ├── auth/
    │   ├── data/     auth_repository.dart, auth_api.dart
    │   ├── domain/   user_model.dart
    │   └── presentation/ login_screen.dart, auth_provider.dart
    ├── products/
    │   ├── data/
    │   ├── domain/
    │   └── presentation/
    └─ cart/
```

কেন ফিচার-ভিত্তিক ভালো

"কার্টে ছাড় যোগ করো" — এই কাজে ফিচার-ভিত্তিক কাঠামোতে আপনি শুধু `features/cart/` খুলবেন, সব একসাথে পাবেন। স্তর-ভিত্তিক কাঠামোতে ৫টি ভিন্ন ফোল্ডারে ঘুরতে হবে। ২০+ স্ক্রিনের অ্যাপে পার্থক্যটা বিশাল।

২৫.২ তিন স্তরের আর্কিটেকচার

```
Presentation (UI)
Screen • Widget • Provider/Bloc
```

→ শুধু দেখায় ও ব্যবহারকারীর ইনপুট নেয়

↓ ডাকে

Domain (ব্যবসায়িক নিয়ম)

Model · Repository interface · UseCase

→ Flutter-নিরপেক্ষ, বিশুদ্ধ Dart

↓ ডাকে

Data

Repository impl · API · Database · Cache

→ ডেটা কোথা থেকে আসে তা জানে

বাস্তব উদাহরণ

```
// domain/repositories/product_repository.dart - শুধু চুক্তি
abstract class ProductRepository {
  Future<List<Product>> getProducts({int page});
  Future<Product> getById(int id);
}

// data/repositories/product_repository_impl.dart - আসল বাস্তবায়ন
class ProductRepositoryImpl implements ProductRepository {
  final ApiClient api;
  final DatabaseHelper db;

  ProductRepositoryImpl(this.api, this.db);

  @override
  Future<List<Product>> getProducts({int page = 1}) async {
    try {
      final data = await api.get('/products', query: {'page': page});
      final products = (data as List).map(Product.fromJson).toList();
      await db.cacheProducts(products); // অফলাইনের জন্য
      return products;
    } on ApiException {
      final cached = await db.getCachedProducts(); // নেট না থাকলে
      if (cached.isNotEmpty) return cached;
      rethrow;
    }
  }

  @override
  Future<Product> getById(int id) async =>
    Product.fromJson(await api.get('/products/$id'));
}
```

interface কেন — ইন্টারভিউতে ডিজেন্স করা হয়

UI শুধু `ProductRepository` চেনে, আসল বাস্তবায়ন নয়। ফলে টেস্টে `FakeProductRepository` বসিয়ে দেওয়া যায়, বা REST থেকে GraphQL-এ যেতে চাইলে শুধু নতুন `Impl` লিখলেই হয় — UI-এর একটি লাইনও বদলাতে হয় না। একেই বলে **Dependency Inversion**।

২৫.৩ Dependency Injection (get_it)

```
flutter pub add get_it
```

```
final sl = GetIt.instance;      // sl = service locator

Future<void> setupLocator() async {
  // একটিই ইনস্ট্যান্স, সাথে সাথে তৈরি
  final prefs = await SharedPreferences.getInstance();
  sl.registerSingleton<SharedPreferences>(prefs);

  // একটিই, কিন্তু প্রথম ব্যবহারে তৈরি (দ্রুত অ্যাপ চালু হয়)
  sl.registerLazySingleton<ApiClient>(() => ApiClient());
  sl.registerLazySingleton<ProductRepository>(
    () => ProductRepositoryImpl(sl(), sl());

  // প্রতিবার নতুন
  sl.registerFactory<CartProvider>(() => CartProvider(sl()));
}

// main.dart
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await setupLocator();
  runApp(const MyApp());
}

// যেকোনো জায়গায়
final repo = sl<ProductRepository>();
```

২৫.৪ ক্লিন কোডের নিয়ম

নিয়ম	ব্যাখ্যা
একটি ফাইল = একটি দায়িত্ব	৩০০ লাইনের বেশি হলে ভাগ করার কথা ভাবুন
build() ছোট রাখুন	৫০ লাইনের বেশি হলে আলাদা widget-এ ভাগ করুন
জাদুর সংখ্যা নয়	16 নয়, AppSizes.paddingM
নাম অর্থবহ হোক	d নয়, deliveryCharge
UI-তে লজিক নয়	দাম গণনা Provider বা model-এ, widget-এ নয়
const যেখানে পারেন	পারফরম্যান্স ও পঠনযোগ্যতা দুটোই বাড়ে
private রাখুন	বাইরে লাগবে না এমন সবকিছুতে _ দিন

২৫.৫ Widget নাকি ফাংশন?

```
// ✗ কম ভালো – ফাংশন দিয়ে UI ভাগ করা
Widget _buildHeader() => Container(...);

// ✓ ভালো – আলাদা widget ক্লাস
class _Header extends StatelessWidget {
  const _Header();
  @override
  Widget build(BuildContext context) => Container(...);
}
```

কারণ: আলাদা widget ক্লাসের নিজস্ব Element থাকে, তাই Flutter শুধু সেটিই rebuild করতে পারে এবং `const` ব্যবহার করা যায়। ফাংশন হলে পুরো প্যারেন্ট rebuild হয়। ছোট প্রজেক্টে পার্থক্য কম, বড় প্রজেক্টে বেশ বড়।

অধ্যায় ২৬ — টেস্টিং

২৬.১ তিন ধরনের টেস্ট

ধরন	কী পরীক্ষা করে	গতি
Unit	একটি ফাংশন বা ক্লাসের লজিক	খুব দ্রুত
Widget	একটি widget ঠিকভাবে দেখাচ্ছে ও সাড়া দিচ্ছে কিনা	দ্রুত
Integration	পুরো অ্যাপের প্রবাহ, আসল ডিভাইসে	ধীর

২৬.২ Unit Test

```
// test/cart_provider_test.dart
import 'package:flutter_test/flutter_test.dart';

void main() {
  group('CartProvider', () {
    late CartProvider cart;
    final product = Product(id: 1, title: 'ঘড়ি', price: 1000);

    setUp(() {
      cart = CartProvider(); // প্রতিটি টেস্টের আগে নতুন
    });

    test('শুরুতে কার্ট খালি থাকবে', () {
      expect(cart.items, isEmpty);
      expect(cart.count, 0);
    });

    test('পণ্য যোগ করলে count বাড়বে', () {
      cart.add(product);
      expect(cart.count, 1);
      expect(cart.subtotal, 1000);
    });

    test('একই পণ্য দুবার দিলে quantity বাড়বে, নতুন আইটেম নয়', () {
      cart.add(product);
      cart.add(product);
      expect(cart.items.length, 1);
      expect(cart.count, 2);
    });

    test('quantity শূন্য করলে আইটেম মুছে যাবে', () {
      cart.add(product);
      cart.updateQuantity(1, 0);
      expect(cart.items, isEmpty);
    });
  });
}
```

```

});

test('১০০০ টাকার বেশি হলে ডেলিভারি ফ্রি', () {
  cart.add(Product(id: 2, title: 'ব্যাগ', price: 1500));
  expect(cart.total, 1500); // ৬০ টাকা যোগ হবে না
});

});

group('Validators', () {
  test('সঠিক ইমেইল গ্রহণ করবে', () {
    expect(Validators.email('a@b.com'), isNull);
  });
  test('ভুল ইমেইল বাতিল করবে', () {
    expect(Validators.email('abc'), isNotNull);
  });
});

});
}

```

২৬.৩ Widget Test

```

testWidgets('কাউন্টার + চাপলে বাড়ে', (WidgetTester tester) async {
  // widget বসান
  await tester.pumpWidget(const MaterialApp(home: CounterPage()));

  // শুরুর অবস্থা যাচাই
  expect(find.text('0'), findsOneWidget);
  expect(find.text('1'), findsNothing);

  // ট্যাপ করুন
  await tester.tap(find.byIcon(Icons.add));
  await tester.pump(); // একটি ফ্রেম আঁকুন

  // ফলাফল যাচাই
  expect(find.text('1'), findsOneWidget);
});

testWidgets('ফর্ম খালি থাকলে এরর দেখায়', (tester) async {
  await tester.pumpWidget(const MaterialApp(home: LoginPage()));

  await tester.tap(find.text('লগইন করুন'));
  await tester.pumpAndSettle(); // সব অ্যানিমেশন শেষ হওয়া পর্যন্ত

  expect(find.text('ইমেইল দিন'), findsOneWidget);
});

testWidgets('টেক্সট লিখতে পারা যায়', (tester) async {
  await tester.pumpWidget(const MaterialApp(home: SearchPage()));

  await tester.enterText(find.byType(TextField), 'ঘড়ি');
  await tester.pump();
}

```



```
expect(find.text('ঘড়ি'), findsOneWidget);
});
```

Finder-এর ধরন

```
find.text('লগইন')
find.byType(ElevatedButton)
find.byIcon(Icons.add)
find.byKey(const ValueKey('submit'))
find.widgetWithText(ElevatedButton, 'সাবমিট')
find.descendant(of: find.byType(Card), matching: find.text('দাম'))
```

২৬.৪ Mock দিয়ে টেস্ট

```
flutter pub add --dev mocktail
```

```
class MockProductRepository extends Mock implements ProductRepository {}

void main() {
  late MockProductRepository repo;
  late ProductProvider provider;

  setUp(() {
    repo = MockProductRepository();
    provider = ProductProvider(repo);
  });

  test('সফল হলে পণ্য লোড হবে', () async {
    // নকল উত্তর ঠিক করে দিন
    when(() => repo.getProducts(page: 1))
      .thenAnswer((_) async => [Product(id: 1, title: 'ঘড়ি', price: 100)]);

    await provider.load();

    expect(provider.products.length, 1);
    expect(provider.error, isNull);
    verify(() => repo.getProducts(page: 1)).called(1);
  });

  test('ব্যর্থ হলে এরর সেট হবে', () async {
    when(() => repo.getProducts(page: 1))
      .thenThrow(ApiException('ইন্টারনেট নেই'));

    await provider.load();

    expect(provider.products, isEmpty);
    expect(provider.error, contains('ইন্টারনেট'));
  });
}
```

২৬.৫ চালানো

```
flutter test                                # সব টেস্ট
flutter test test/cart_test.dart           # নির্দিষ্ট ফাইল
flutter test --coverage                     # কতটুকু কোড টেস্ট হলো
flutter test --name "কার্ট"                # নাম মিলিয়ে
```

নতুন হিসেবে কোথায় শুরু করবেন

সব টেস্ট করার চেষ্টা করবেন না। শুরু করুন **ব্যবসায়িক লজিকের unit test** দিয়ে — দাম গণনা, ভ্যালিডেশন, ছাড়ের হিসাব। এগুলোতেই বাগ বেশি হয় এবং টেস্ট লেখা সবচেয়ে সহজ। UI টেস্ট পরে।

অধ্যায় ২৭ — পারফরম্যান্স ও ডিবাগিং

২৭.১ পারফরম্যান্সের সোনালি নিয়ম

নিয়ম	কেন
const ব্যবহার করুন	widget পুনঃব্যবহার হয়, rebuild হয় না
ListView.builder ব্যবহার করুন	শুধু দৃশ্যমান আইটেম তৈরি হয়
rebuild-এর পরিসর ছোট করুন	Consumer/Selector দিয়ে শুধু দরকারি অংশ
build()-এ ভারী কাজ নয়	build সেকেন্ডে ৬০ বার চলতে পারে
ছবি ক্যাশ করুন	cached_network_image ব্যবহার করুন
বড় হিসাব isolate-এ	compute() দিয়ে UI আটকানো এড়ান
Opacity এড়িয়ে চলুন	ব্যয়বহুল; FadeTransition বা রঙের alpha ব্যবহার করুন

২৭.২ rebuild কমানোর কৌশল

```
// X খারাপ – পুরো পেজ rebuild হয়
@override
Widget build(BuildContext context) {
  final cart = context.watch<CartProvider>();
  return Scaffold(
    appBar: AppBar(title: Text('কার্ট (${cart.count})')),
    body: const HeavyProductList(), // এটিও rebuild হচ্ছে!
  );
}

// ✓ ভালো – শুধু সংখ্যাটুকু rebuild হয়
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Consumer<CartProvider>(
        builder: (_, cart, __) => Text('কার্ট (${cart.count})'),
      ),
    ),
    body: const HeavyProductList(), // আর rebuild হবে না
  );
}
```

```
}
```

```
// ✓ আরও ভালো – select দিয়ে শুধু একটি ফিল্ড  
final count = context.select<CartProvider, int>((c) => c.count);
```

২৭.৩ DevTools ব্যবহার

ট্যাব	যা দেখায়
Widget Inspector	widget tree, লেআউট সমস্যা, প্রতিটি widget-এর মাপ
Performance	কোন ফ্রেম ১৬ms-এর বেশি নিচ্ছে (jank)
CPU Profiler	কোন ফাংশনে সবচেয়ে বেশি সময় যাচ্ছে
Memory	মেমোরি লিক ও অবজেক্টের সংখ্যা
Network	সব HTTP রিকোয়েস্ট ও রেসপন্স
Logging	print ও এরর লগ

```
// rebuild গোনা – কোন widget বারবার rebuild হচ্ছে দেখুন  
void main() {  
  debugRepaintRainbowEnabled = true;    // রঙিন বর্ডার দেখাবে  
  debugPrintRebuildDirtyWidgets = true; // কনসোলে লগ  
  runApp(const MyApp());  
}
```

২৭.৪ ডিবাগিং কৌশল

```
print('মান: $value');  
debugPrint('লম্বা লেখা কেটে যাবে না');  
log('বিস্তারিত', name: 'API', error: e, stackTrace: s); // dart:developer  
  
assert(items.isNotEmpty, 'লিস্ট খালি হওয়ার কথা নয়'); // শুধু debug মোডে  
  
debugDumpApp();           // পুরো widget tree  
debugDumpRenderTree();    // render tree ও মাপ  
  
// breakpoint বসিয়ে ডিবাগ করুন – print এর চেয়ে অনেক ভালো  
// VS Code-এ লাইন নম্বরের বাঁয়ে ক্লিক করুন, তারপর F5
```

২৭.৫ Release-এ ক্র্যাশ ধরা

```
// Firebase Crashlytics
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();

  // Flutter-এর সব এরর
  FlutterError.onError = FirebaseCrashlytics.instance.recordFlutterFatalError;

  // Flutter-এর বাইরের এরর
  PlatformDispatcher.instance.onError = (error, stack) {
    FirebaseCrashlytics.instance.recordError(error, stack, fatal: true);
    return true;
  };

  runApp(const MyApp());
}
```

২৭.৬ অ্যাপের সাইজ কমানো

```
flutter build apk --split-per-abi      # প্রতি আর্কিটেকচারে আলাদা (~৪০% ছোট)
flutter build appbundle                # Play Store নিজেই অপ্টিমাইজ করে
flutter build apk --analyze-size       # কী কতটুকু জায়গা নিচ্ছে
```

- অব্যবহৃত প্যাকেজ বাদ দিন (`flutter pub deps` দেখুন)।
- PNG-এর বদলে WebP বা SVG ব্যবহার করুন।
- ছবি আগেই সঠিক মাপে রিসাইজ করে রাখুন।
- একাধিক ফন্ট ওজন যোগ করবেন না যদি ব্যবহার না করেন।

অধ্যায় ২৮ — ডিপ্লয়মেন্ট ও CI/CD

২৮.১ রিলিজের আগে চেকলিস্ট

কাজ	কোথায়
অ্যাপের নাম	AndroidManifest.xml → android:label
প্যাকেজ নাম	build.gradle → applicationId
আইকন	flutter_launcher_icons
স্প্ল্যাশ স্ক্রিন	flutter_native_splash
সংস্করণ	pubspec.yaml → version: 1.0.0+1
সব print সরানো	কোডজুড়ে
API URL প্রোডাকশনের	constants ফাইল
minSdkVersion	build.gradle (২১+)

২৮.২ আইকন ও স্প্ল্যাশ

```
dev_dependencies:  
  flutter_launcher_icons: ^0.13.1  
  flutter_native_splash: ^2.3.0  
  
flutter_launcher_icons:  
  android: true  
  ios: true  
  image_path: "assets/icon/icon.png"          # 1024x1024  
  adaptive_icon_background: "#0E7490"  
  adaptive_icon_foreground: "assets/icon/fg.png"  
  
flutter_native_splash:  
  color: "#0E7490"  
  image: assets/icon/splash.png  
  android_12:  
    image: assets/icon/splash.png  
    color: "#0E7490"
```

```
flutter pub get  
dart run flutter_launcher_icons  
dart run flutter_native_splash:create
```

২৮.৩ অ্যাপ সাইন করা (Android)

```
# ১. keystore তৈরি
keytool -genkey -v -keystore upload-keystore.jks \
-keyalg RSA -keysize 2048 -validity 10000 -alias upload
```

২. `android/key.properties` ফাইল বানান (এটি `.gitignore`-এ যোগ করুন):

```
storePassword=আপনার_পাসওয়ার্ড
keyPassword=আপনার_পাসওয়ার্ড
keyAlias=upload
storeFile=C:/keys/upload-keystore.jks
```

৩. `android/app/build.gradle` -এ:

```
def keystoreProperties = new Properties()
def keystorePropertiesFile = rootProject.file('key.properties')
if (keystorePropertiesFile.exists()) {
    keystoreProperties.load(new FileInputStream(keystorePropertiesFile))
}

android {
    signingConfigs {
        release {
            keyAlias keystoreProperties['keyAlias']
            keyPassword keystoreProperties['keyPassword']
            storeFile file(keystoreProperties['storeFile'])
            storePassword keystoreProperties['storePassword']
        }
    }
    buildTypes {
        release {
            signingConfig signingConfigs.release
            minifyEnabled true
            shrinkResources true
        }
    }
}
```

keystore হারালে সর্বনাশ

এই ফাইল ও পাসওয়ার্ড হারালে আপনি **কখনোই** Play Store-এ সেই অ্যাপের আপডেট দিতে পারবেন না — নতুন প্যাকেজ নামে নতুন অ্যাপ ছাড়তে হবে, সব ইউজার হারাবেন। নিরাপদ জায়গায় ব্যাকআপ রাখুন, কিন্তু **কখনোই git-এ পুশ করবেন না**।

২৮.৪ বিন্ড ও প্রকাশ

```
# Android
flutter build apk --release           # সরাসরি বিতরণের জন্য
flutter build apk --split-per-abi     # ছোট সাইজ
flutter build appbundle --release     # ★ Play Store-এর জন্য

# iOS (macOS লাগবে)
flutter build ios --release
flutter build ipa

# Web
flutter build web --release --web-renderer canvaskit

# Windows / macOS / Linux
flutter build windows --release
```

ফাইলের

অবস্থান:

build/app/outputs/flutter-apk/app-release.apk

এবং

build/app/outputs/bundle/release/app-release.aab

২৮.৫ Play Store-এ প্রকাশের ধাপ

1. Google Play Console-এ অ্যাকাউন্ট খুলুন (একবার \$25)।
2. Create app → নাম, ভাষা, ফ্রি/পেইড নির্বাচন।
3. Store listing — বর্ণনা, ৮টি স্ক্রিনশট, ফিচার গ্রাফিক (1024×500), আইকন (512×512)।
4. Content rating প্রশ্নপত্র পূরণ।
5. **Privacy Policy** URL (বাধ্যতামূলক) — GitHub Pages-এ বিনামূল্যে হোস্ট করতে পারেন।
6. Data safety ফর্ম — কী কী ডেটা সংগ্রহ করেন তা ঘোষণা।
7. Production → Create release → `.aab` আপলোড।
8. রিভিউ — সাধারণত ১-৭ দিন।

২৮.৬ CI/CD — GitHub Actions

```
# .github/workflows/build.yml
name: Build APK

on:
  push:
    branches: [ main ]
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest
```


steps:

- uses: actions/checkout@v4
- uses: subosito/flutter-action@v2
with:
 - flutter-version: '3.24.0'
 - channel: stable
- run: flutter pub get
- run: flutter analyze
- run: flutter test
- run: flutter build apk --release
- uses: actions/upload-artifact@v4
with:
 - name: release-apk
 - path: build/app/outputs/flutter-apk/app-release.apk

flavor — এক কোডবেসে dev ও prod

```
flutter run --flavor dev --dart-define=API_URL=https://dev.api.com
```

কোডে পড়ুন: `const apiUrl = String.fromEnvironment('API_URL');`

এতে ডেভেলপমেন্ট ও প্রোডাকশন সার্ভার আলাদা রাখা যায়, এবং দুটি অ্যাপ একই ফোনে পাশাপাশি ইনস্টল থাকতে পারে।

পর্ব ৭

প্রজেক্ট

৬টি পূর্ণাঙ্গ প্রজেক্ট — কাঠামো, মূল কোড ও ধাপে ধাপে নির্দেশনা।
এগুলোই আপনার পোর্টফোলিও।

প্রজেক্ট ১ — নোট অ্যাপ (SQLite + CRUD)

যা শিখবেন

StatefulWidget · sqflite CRUD · সার্চ · ফর্ম · ডার্ক মোড সেভ · Dismissible · রেসপন্সিভ গ্রিড।

সময়: ২-৩ দিন। কঠিনতা: শুরুর স্তর।

ফোল্ডার কাঠামো

```
lib/  
├─ main.dart  
├─ models/note.dart  
├─ services/database_helper.dart  
├─ providers/note_provider.dart  
├─ screens/  
│   ├── notes_screen.dart  
│   └── note_edit_screen.dart  
└─ widgets/note_card.dart
```

models/note.dart

```
class Note {  
  final int? id;  
  final String title;  
  final String body;  
  final String color;  
  final bool isPinned;  
  final DateTime updatedAt;  
  
  Note({  
    this.id, required this.title, this.body = '',  
    this.color = '#FFFFFF', this.isPinned = false,  
    required this.updatedAt,  
  });  
  
  Map<String, dynamic> toMap() => {  
    if (id != null) 'id': id,  
    'title': title, 'body': body, 'color': color,  
    'isPinned': isPinned ? 1 : 0,  
    'updatedAt': updatedAt.toIso8601String(),  
  };  
  
  factory Note.fromMap(Map<String, dynamic> m) => Note(  
    id: m['id'] as int?,  
    title: m['title'] as String,  
    body: m['body'] as String? ?? '',  
    color: m['color'] as String? ?? '#FFFFFF',  
  );  
}
```

```

        isPinned: (m['isPinned'] as int? ?? 0) == 1,
        updatedAt: DateTime.parse(m['updatedAt'] as String),
    );

    Note copyWith({String? title, String? body, String? color, bool? isPinned}) =>
    Note(
        id: id,
        title: title ?? this.title,
        body: body ?? this.body,
        color: color ?? this.color,
        isPinned: isPinned ?? this.isPinned,
        updatedAt: DateTime.now(),
    );
}

```

providers/note_provider.dart

```

class NoteProvider extends ChangeNotifier {
    final _db = DatabaseHelper.instance;

    List<Note> _notes = [];
    String _query = '';
    bool _loading = false;

    bool get loading => _loading;

    List<Note> get notes {
        if (_query.isEmpty) return _notes;
        final q = _query.toLowerCase();
        return _notes.where((n) =>
            n.title.toLowerCase().contains(q) ||
            n.body.toLowerCase().contains(q)).toList();
    }

    Future<void> load() async {
        _loading = true;
        notifyListeners();
        _notes = await _db.getAll();
        _loading = false;
        notifyListeners();
    }

    Future<void> save(Note note) async {
        if (note.id == null) {
            await _db.insert(note);
        } else {
            await _db.update(note);
        }
        await load();
    }

    Future<void> delete(int id) async {

```

```

    await _db.delete(id);
    _notes.removeWhere((n) => n.id == id);
    notifyListeners();
  }

  Future<void> togglePin(Note note) async {
    await _db.update(note.copyWith(isPinned: !note.isPinned));
    await load();
  }

  void search(String q) {
    _query = q;
    notifyListeners();
  }
}

```

screens/notes_screen.dart (মূল অংশ)

```

class NotesScreen extends StatefulWidget {
  const NotesScreen({super.key});
  @override State<NotesScreen> createState() => _NotesScreenState();
}

class _NotesScreenState extends State<NotesScreen> {
  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addPostFrameCallback((_) {
      context.read<NoteProvider>().load();
    });
  }

  @override
  Widget build(BuildContext context) {
    final provider = context.watch<NoteProvider>();

    return Scaffold(
      appBar: AppBar(
        title: const Text('আমার নোট'),
        actions: [
          IconButton(
            icon: Icon(context.watch<ThemeProvider>().isDark
              ? Icons.light_mode : Icons.dark_mode),
            onPressed: () => context.read<ThemeProvider>().toggle(),
          ),
        ],
      ),
      body: Column(
        children: [
          Padding(
            padding: const EdgeInsets.all(12),

```

```

        child: TextField(
          onChanged: provider.search,
          decoration: const InputDecoration(
            hintText: 'নোট খুঁজুন...',
            prefixIcon: Icon(Icons.search),
            border: OutlineInputBorder(),
          ),
        ),
      ),
    ),
  ),

  Expanded(
    child: provider.loading
      ? const Center(child: CircularProgressIndicator())
      : provider.notes.isEmpty
        ? const _EmptyView()
        : GridView.builder(
            padding: const EdgeInsets.all(12),
            gridDelegate:
              const SliverGridDelegateWithMaxCrossAxisExtent(
                maxCrossAxisExtent: 220,
                mainAxisSpacing: 12, crossAxisSpacing: 12,
                childAspectRatio: 0.9,
              ),
            itemCount: provider.notes.length,
            itemBuilder: (c, i) => NoteCard(
              note: provider.notes[i],
              onTap: () => _openEditor(provider.notes[i]),
              onPin: () => provider.togglePin(provider.notes[i]),
            ),
          ),
    ),
  ],
),

floatingActionButton: FloatingActionButton(
  onPressed: () => _openEditor(null),
  child: const Icon(Icons.add),
),
);
}

void _openEditor(Note? note) => Navigator.push(
  context,
  MaterialPageRoute(builder: (_) => NoteEditScreen(note: note)),
);
}

```

নিজে যোগ করুন

১. নোটে রঙ বাছাইয়ের সুবিধা। ২. ক্যাটাগরি ও ফিল্টার। ৩. নোট শেয়ার করা (`share_plus`)। ৪. রিমাইন্ডার নোটিফিকেশন। ৫. ছবি সংযুক্ত করা।

প্রজেক্ট ২ — ই-কমার্স অ্যাপ (API + Provider + Cart)

যা শিখবেন

REST API · JSON পার্সিং · Provider · কার্ট লজিক · Hero অ্যানিমেশন · সার্চ ও ফিল্টার · pagination · চেকআউট।

সময়: ৪-৫ দিন। কঠিনতা: মাঝারি। API: fakestoreapi.com

এটিই আপনার পোর্টফোলিওর প্রধান প্রজেক্ট হওয়া উচিত।

মডেল

```
class Product {
  final int id;
  final String title, description, category, image;
  final double price, rating;
  final int ratingCount;

  const Product({
    required this.id, required this.title, required this.description,
    required this.category, required this.image,
    required this.price, required this.rating, required this.ratingCount,
  });

  factory Product.fromJson(Map<String, dynamic> j) => Product(
    id: j['id'] as int,
    title: j['title'] as String? ?? '',
    description: j['description'] as String? ?? '',
    category: j['category'] as String? ?? '',
    image: j['image'] as String? ?? '',
    price: (j['price'] as num?)?.toDouble() ?? 0,
    rating: (j['rating']?['rate'] as num?)?.toDouble() ?? 0,
    ratingCount: (j['rating']?['count'] as num?)?.toInt() ?? 0,
  );
}

class CartItem {
  final Product product;
  int quantity;
  CartItem({required this.product, this.quantity = 1});
  double get total => product.price * quantity;
}
```

প্রোডাক্ট প্রোভাইডার — সার্চ, ফিল্টার ও সর্ট

```
enum SortBy { none, priceLow, priceHigh, rating }
```

```

class ProductProvider extends ChangeNotifier {
    final ProductRepository _repo;
    ProductProvider(this._repo);

    List<Product> _all = [];
    String _search = '';
    String? _category;
    SortBy _sort = SortBy.none;
    bool _loading = false;
    String? _error;

    bool get loading => _loading;
    String? get error => _error;
    List<String> get categories =>
        _all.map((p) => p.category).toSet().toList()..sort();

    List<Product> get products {
        var list = _all.where((p) {
            final matchSearch = _search.isEmpty ||
                p.title.toLowerCase().contains(_search.toLowerCase());
            final matchCat = _category == null || p.category == _category;
            return matchSearch && matchCat;
        }).toList();

        switch (_sort) {
            case SortBy.priceLow: list.sort((a, b) => a.price.compareTo(b.price));
            case SortBy.priceHigh: list.sort((a, b) => b.price.compareTo(a.price));
            case SortBy.rating: list.sort((a, b) => b.rating.compareTo(a.rating));
            case SortBy.none: break;
        }
        return list;
    }

    Future<void> load() async {
        _loading = true; _error = null;
        notifyListeners();
        try {
            _all = await _repo.getProducts();
        } catch (e) {
            _error = e.toString();
        } finally {
            _loading = false;
            notifyListeners();
        }
    }

    void setSearch(String v) { _search = v; notifyListeners(); }
    void setCategory(String? v){ _category = v; notifyListeners(); }
    void setSort(SortBy v) { _sort = v; notifyListeners(); }
}

```


প্রোডাক্ট কার্ড widget

```
class ProductCard extends StatelessWidget {
  final Product product;
  final VoidCallback onTap;
  const ProductCard({super.key, required this.product, required this.onTap});

  @override
  Widget build(BuildContext context) {
    return Card(
      clipBehavior: Clip.antiAlias,
      child: InkWell(
        onTap: onTap,
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Expanded(
              child: Hero(
                tag: 'product-${product.id}',
                child: Padding(
                  padding: const EdgeInsets.all(12),
                  child: CachedNetworkImage(
                    imageUrl: product.image,
                    fit: BoxFit.contain,
                    placeholder: (c, u) =>
                      const Center(child: CircularProgressIndicator()),
                    errorWidget: (c, u, e) => const Icon(Icons.image),
                  ),
                ),
              ),
            ),
            Padding(
              padding: const EdgeInsets.fromLTRB(10, 0, 10, 10),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(product.title, maxLines: 2,
                    overflow: TextOverflow.ellipsis,
                    style: const TextStyle(
                      fontSize: 13, fontWeight: FontWeight.w600)),
                  const SizedBox(height: 4),
                  Row(children: [
                    const Icon(Icons.star, size: 14, color: Colors.amber),
                    Text(' ${product.rating}  (${product.ratingCount})',
                      style: const TextStyle(fontSize: 11)),
                  ]),
                  const SizedBox(height: 6),
                  Row(
                    mainAxisAlignment: MainAxisAlignment.spaceBetween,
                    children: [
                      Text('₳ ${product.price.toStringAsFixed(0)}',
                        style: TextStyle(
                          fontWeight: FontWeight.bold,
                          color: Theme.of(context).colorScheme.primary)),
```

```

Consumer<CartProvider>(
  builder: (_, cart, __) => IconButton(
    icon: const Icon(Icons.add_shopping_cart, size: 20),
    onPressed: () {
      cart.add(product);
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: const Text('কার্টে যোগ হয়েছে'),
          duration: const Duration(seconds: 1),
          action: SnackBarAction(
            label: 'দেখুন',
            onPressed: () => Navigator.pushNamed(
              context, '/cart'),
          ),
        ),
      );
    },
  ),
),
],
),
),
],
),
],
),
),
],
),
);
}
}

```

কার্ট স্ক্রিন — মোট হিসাবসহ

```

class CartScreen extends StatelessWidget {
  const CartScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final cart = context.watch<CartProvider>();

    if (cart.isEmpty) {
      return const Scaffold(body: Center(child: Text('কার্ট খালি')));
    }

    return Scaffold(
      appBar: AppBar(title: Text('কার্ট (${cart.count})')),
      body: ListView.builder(
        itemCount: cart.items.length,
        itemBuilder: (c, i) {
          final item = cart.items[i];
          return Dismissible(
            key: ValueKey(item.product.id),

```

```

direction: DismissDirection.endToStart,
background: Container(
  color: Colors.red, alignment: Alignment.centerRight,
  padding: const EdgeInsets.only(right: 20),
  child: const Icon(Icons.delete, color: Colors.white),
),
onDismissed: (_) => cart.remove(item.product.id),
child: ListTile(
  leading: Image.network(item.product.image, width: 50),
  title: Text(item.product.title, maxLines: 1,
    overflow: TextOverflow.ellipsis),
  subtitle: Text('৳ ${item.product.price}'),
  trailing: Row(
    mainAxisAlignment: MainAxisAlignment.min,
    children: [
      IconButton(
        icon: const Icon(Icons.remove_circle_outline),
        onPressed: () => cart.updateQuantity(
          item.product.id, item.quantity - 1),
      ),
      Text('${item.quantity}'),
      IconButton(
        icon: const Icon(Icons.add_circle_outline),
        onPressed: () => cart.updateQuantity(
          item.product.id, item.quantity + 1),
      ),
    ],
  ),
),
);
},
),
),
),

bottomNavigationBar: Container(
  padding: const EdgeInsets.all(16),
  decoration: BoxDecoration(
    color: Theme.of(context).cardColor,
    boxShadow: [BoxShadow(color: Colors.black12, blurRadius: 8)],
  ),
  child: SafeArea(
    child: Column(mainAxisSize: MainAxisSize.min, children: [
      _row('সাবটোটাল', cart.subtotal),
      _row('ডেলিভারি', cart.subtotal > 1000 ? 0 : 60),
      const Divider(),
      _row('মোট', cart.total, bold: true),
      const SizedBox(height: 12),
      FilledButton(
        onPressed: cart.isLoading ? null : () => _checkout(context, cart),
        style: FilledButton.styleFrom(
          minimumSize: const Size.fromHeight(50)),
        child: const Text('অর্ডার করুন'),
      ),
    ]),
  ),
),
),

```

```

    );
}

Widget _row(String label, double value, {bool bold = false}) => Padding(
  padding: const EdgeInsets.symmetric(vertical: 2),
  child: Row(
    mainAxisAlignment: MainAxisAlignment.spaceBetween,
    children: [
      Text(label,
        style: TextStyle(
          fontWeight: bold ? FontWeight.bold : FontWeight.normal)),
      Text('₹ ${value.toStringAsFixed(0)}',
        style: TextStyle(
          fontWeight: bold ? FontWeight.bold : FontWeight.normal)),
    ],
  ),
);
}

```

নিজে যোগ করুন

১. পছন্দের তালিকা (wishlist)। ২. অর্ডার ইতিহাস। ৩. প্রোডাক্ট রিভিউ। ৪. shared_preferences দিয়ে কার্ট সেভ রাখা। ৫. ক্যাটাগরি অনুযায়ী ব্যানার স্লাইডার।

প্রজেক্ট ৩ — আবহাওয়া অ্যাপ (GPS + API)

যা শিখবেন

Geolocator · API কী ব্যবস্থাপনা · সুন্দর UI ও গ্রেডিয়েন্ট · তারিখ ফরম্যাট · এরর হ্যান্ডলিং।

API: openweathermap.org (ফ্রি কী নিন) · **সময়:** ২ দিন।

```
class WeatherService {
  static const _key = String.fromEnvironment('OWM_KEY'); // ★ কোডে লিখবেন না
  static const _base = 'https://api.openweathermap.org/data/2.5';

  Future<Weather> getByLocation(double lat, double lon) async {
    final res = await http.get(Uri.parse(
      '$_base/weather?lat=$lat&lon=$lon&appid=$_key&units=metric&lang=bn'));

    if (res.statusCode != 200) throw Exception('আবহাওয়ার তথ্য পাওয়া যায়নি');
    return Weather.fromJson(jsonDecode(res.body));
  }

  Future<List<Weather>> getForecast(double lat, double lon) async {
    final res = await http.get(Uri.parse(
      '$_base/forecast?lat=$lat&lon=$lon&appid=$_key&units=metric'));
    final list = jsonDecode(res.body)['list'] as List;

    // প্রতিদিন দুপুর ১২টার তথ্য নিন
    return list
      .where((e) => (e['dt_txt'] as String).contains('12:00:00'))
      .map((e) => Weather.fromJson(e))
      .toList();
  }
}

class Weather {
  final String city, condition, icon;
  final double temp, feelsLike, windSpeed;
  final int humidity;
  final DateTime date;

  Weather({required this.city, required this.condition, required this.icon,
    required this.temp, required this.feelsLike, required this.windSpeed,
    required this.humidity, required this.date});

  factory Weather.fromJson(Map<String, dynamic> j) => Weather(
    city: j['name'] as String? ?? '',
    condition: j['weather'][0]['description'] as String? ?? '',
    icon: j['weather'][0]['icon'] as String? ?? '01d',
    temp: (j['main']['temp'] as num).toDouble(),
    feelsLike: (j['main']['feels_like'] as num).toDouble(),
    humidity: (j['main']['humidity'] as num).toInt(),
    windSpeed: (j['wind']['speed'] as num).toDouble(),
  );
}
```

```

        date: DateTime.fromMillisecondsSinceEpoch((j['dt'] as int) * 1000),
    );

String get imageUrl => 'https://openweathermap.org/img/wn/$icon@2x.png';

// সময় অনুযায়ী পটভূমির রঙ
List<Color> get gradient {
    final hour = date.hour;
    if (hour >= 6 && hour < 17) return [Color(0xFF4A90E2), Color(0xFF87CEEB)];
    if (hour >= 17 && hour < 20) return [Color(0xFFFFF7E5F), Color(0xFFFFEB47B)];
    return [Color(0xFF0F2027), Color(0xFF2C5364)];
}
}

```

প্রজেক্ট ৪ — চ্যাট অ্যাপ (Firebase Realtime)

যা শিখবেন

Firebase Auth · Firestore StreamBuilder · রিয়েলটাইম আপডেট · ছবি আপলোড · টাইমস্ট্যাম্প।

সময়: ৩-৪ দিন। কঠিনতা: মাঝারি-উচ্চ।

```

class ChatService {
    final _db = FirebaseFirestore.instance;
    final String currentUid;
    ChatService(this.currentUid);

    // দুজনের জন্য অনন্য ও অপরিবর্তনীয় রুম আইডি
    String roomId(String otherUid) {
        final ids = [currentUid, otherUid]..sort();
        return ids.join('_');
    }

    Stream<List<Message>> messages(String otherUid) => _db
        .collection('chats').doc(roomId(otherUid))
        .collection('messages')
        .orderBy('createdAt', descending: true)
        .limit(100)
        .snapshots()
        .map((snap) => snap.docs.map((d) => Message.fromDoc(d)).toList());

    Future<void> send(String otherUid, String text) async {
        if (text.trim().isEmpty) return;
        final room = roomId(otherUid);

        await _db.collection('chats').doc(room)
            .collection('messages').add({
                'text': text.trim(),
                'senderId': currentUid,
            });
    }
}

```

```

        'createdAt': FieldValue.serverTimestamp(),
        'isRead': false,
    });

    // চ্যাট তালিকায় শেষ বার্তা দেখাতে
    await _db.collection('chats').doc(room).set({
        'participants': [currentUid, otherUid],
        'lastMessage': text.trim(),
        'lastMessageAt': FieldValue.serverTimestamp(),
    }, SetOptions(merge: true));
}
}

// বার্তার বুদবুদ
class MessageBubble extends StatelessWidget {
    final Message message;
    final bool isMine;
    const MessageBubble({super.key, required this.message, required this.isMine});

    @override
    Widget build(BuildContext context) {
        return Align(
            alignment: isMine ? Alignment.centerRight : Alignment.centerLeft,
            child: Container(
                margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 4),
                padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 10),
                constraints: BoxConstraints(
                    maxWidth: MediaQuery.sizeOf(context).width * 0.75),
                decoration: BoxDecoration(
                    color: isMine
                        ? Theme.of(context).colorScheme.primary
                        : Colors.grey.shade200,
                    borderRadius: BorderRadius.only(
                        topLeft: const Radius.circular(16),
                        topRight: const Radius.circular(16),
                        bottomLeft: Radius.circular(isMine ? 16 : 4),
                        bottomRight: Radius.circular(isMine ? 4 : 16),
                    ),
                ),
            child: Column(
                crossAxisAlignment: CrossAxisAlignment.end,
                children: [
                    Text(message.text,
                        style: TextStyle(color: isMine ? Colors.white : Colors.black87)),
                    Text(DateFormat('hh:mm a').format(message.createdAt),
                        style: TextStyle(
                            fontSize: 10,
                            color: isMine ? Colors.white70 : Colors.black45)),
                ],
            ),
        );
    }
}

```

প্রজেক্ট ৫ — খরচের হিসাব (Chart + Local DB)

যা শিখবেন

sqlite দিয়ে হিসাব · fl_chart দিয়ে পাই ও বার চার্ট · তারিখ ফিল্টার · ক্যাটাগরি · মাসিক সারাংশ। সময়: ৩ দিন।

```
flutter pub add fl_chart intl
```

```
// ক্যাটাগরি অনুযায়ী মোট বের করা
Future<Map<String, double>> totalsByCategory(DateTime month) async {
  final db = await database;
  final start = DateTime(month.year, month.month, 1).toIso8601String();
  final end = DateTime(month.year, month.month + 1, 0).toIso8601String();

  final rows = await db.rawQuery('''
    SELECT category, SUM(amount) as total
    FROM transactions
    WHERE type = 'expense' AND date BETWEEN ? AND ?
    GROUP BY category
    ORDER BY total DESC
  ''', [start, end]);

  return {for (final r in rows)
    r['category'] as String: (r['total'] as num).toDouble()};
}

// পাই চার্ট
PieChart(
  PieChartData(
    sectionsSpace: 2,
    centerSpaceRadius: 50,
    sections: totals.entries.map((e) {
      final percent = (e.value / grandTotal * 100);
      return PieChartSectionData(
        value: e.value,
        title: '${percent.toStringAsFixed(0)}%',
        color: categoryColors[e.key],
        radius: 70,
        titleStyle: const TextStyle(
          fontSize: 12, fontWeight: FontWeight.bold, color: Colors.white),
      );
    }).toList(),
  ),
)
```


প্রজেক্ট ৬ — অ্যাটেনডেন্স অ্যাপ (GPS + ক্যামেরা)

যা শিখবেন

GPS যাচাই (geofencing) · সেলফি ক্যাপচার · ছবি আপলোড · সময় হিসাব · অফলাইন সিস্টেম।

সময়: ৪ দিন। কঠিনতা: উচ্চ। এটি চাকরির ইন্টারভিউতে দারুণ কাজে দেয়।

```
class AttendanceService {
    // অফিসের অবস্থান ও অনুমোদিত ব্যাসার্ধ
    static const officeLat = 23.8103;
    static const officeLon = 90.4125;
    static const allowedRadius = 150.0; // মিটার

    Future<CheckInResult> checkIn() async {
        // ১. অবস্থান নিন
        final pos = await LocationService.getCurrentLocation();

        // ২. দূরত্ব যাচাই
        final distance = Geolocator.distanceBetween(
            pos.latitude, pos.longitude, officeLat, officeLon);

        if (distance > allowedRadius) {
            throw Exception(
                'আপনি অফিস থেকে ${distance.toStringAsFixed(0)} মিটার দূরে আছেন');
        }

        // ৩. নকল অবস্থান যাচাই ★ গুরুত্বপূর্ণ
        if (pos.isMocked) {
            throw Exception('নকল অবস্থান শনাক্ত হয়েছে');
        }

        // ৪. সেলফি
        final selfie = await ImagePicker().pickImage(
            source: ImageSource.camera,
            preferredCameraDevice: CameraDevice.front,
            imageQuality: 70,
        );
        if (selfie == null) throw Exception('সেলফি তুলতে হবে');

        // ৫. সার্ভারে পাঠানো (নেট না থাকলে লোকালে জমা)
        try {
            await _api.submitCheckIn(pos, File(selfie.path));
            return CheckInResult.success(DateTime.now());
        } on ApiException {
            await _db.queueOffline(pos, selfie.path);
            return CheckInResult.queued();
        }
    }
}
```

পোর্টফোলিওর জন্য পরামর্শ

৬টি প্রজেক্টই বানানোর দরকার নেই। **২টি প্রজেক্ট চমৎকারভাবে শেষ করুন** — ই-কমার্স (প্রজেক্ট ২) এবং আপনার পছন্দের আরেকটি। প্রতিটির জন্য: GitHub রিপো + ভালো README + স্ক্রিনশট + APK ডাউনলোড লিংক + ১ মিনিটের ডেমো ভিডিও। এটি ১০টি অসম্পূর্ণ প্রজেক্টের চেয়ে অনেক বেশি কাজে দেবে।

পর্ব ৮

ইন্টারভিউ প্রস্তুতি

১১০+ প্রশ্ন ও উত্তর, কোডিং সমস্যা এবং চাকরি পাওয়ার বাস্তব
কৌশল।

অধ্যায় ২৯ — Dart ইন্টারভিউ প্রশ্ন

১. Dart কী এবং কেন Flutter এটি ব্যবহার করে?

Dart হলো Google-এর client-optimized ভাষা। Flutter এটি বেছে নিয়েছে কারণ — JIT কম্পাইল Hot Reload সম্ভব করে, AOT কম্পাইল রিলিজে নেটিভ গতি দেয়, sound null safety কম্পাইল টাইমেই ভুল ধরে, এবং এর দ্রুত মেমোরি বরাদ্দ ও generational GC widget-এর মতো অসংখ্য স্বল্পায়ু অবজেক্টের জন্য আদর্শ।

২. JIT ও AOT কম্পাইলেশনের পার্থক্য কী?

JIT (Just-In-Time) — কোড চলার সময় কম্পাইল হয়। ডেভেলপমেন্টে ব্যবহৃত, Hot Reload সম্ভব করে, কিন্তু ধীর। **AOT** (Ahead-Of-Time) — বিল্ডের সময়েই মেশিন কোডে রূপান্তর। রিলিজে ব্যবহৃত, দ্রুত চালু হয় ও দ্রুত চলে।

৩. var, final ও const-এর পার্থক্য?

`var` — টাইপ অনুমান করে, মান পরে বদলানো যায়। `final` — রানটাইমে একবার সেট, আর বদলানো যায় না (যেমন `DateTime.now()`)। `const` — কম্পাইল টাইমেই মান জানা থাকতে হবে, এবং অবজেক্টটি সম্পূর্ণ immutable ও canonicalized (একই const মানের একটিই কপি মেমোরিতে থাকে)।

৪. Sound Null Safety বলতে কী বোঝায়?

"Sound" মানে — কম্পাইলার গ্যারান্টি দেয় যে non-nullable টাইপ কখনো null হবে না। এর সুবিধা কেবল নিরাপত্তা নয়: কম্পাইলার null চেক বাদ দিয়ে ছোট ও দ্রুত কোড তৈরি করতে পারে।

৫. ?, !, ??, ?., ??= — প্রত্যেকটির কাজ?

`String?` null হতে পারে · `x!` "আমি নিশ্চিত null নয়" (ভুল হলে ক্র্যাশ) · `a ?? b` a null হলে b · `a?.b` a null না হলে তবেই b · `a ??= b` a null হলে তবেই b সেট করো।

৬. late কীওয়ার্ড কখন ব্যবহার করবেন?

যখন ভ্যারিয়েবলটি non-nullable রাখতে চান কিন্তু ঘোষণার সময় মান দিতে পারছেন না — যেমন `initState()` -এ সেট করা controller। এটি lazy initialization-ও করে (প্রথম ব্যবহারে তৈরি)। ঝুঁকি: সেট করার আগে ব্যবহার করলে `LateInitializationError`।

৭. final লিস্টে add() করা যায় কেন?

`final` কেবল রেফারেন্সটি আটকায়, ভিতরের বিষয়বস্তু নয়। তাই `list.add(4)` চলবে কিন্তু `list = [5]` চলবে না। ভিতরের বিষয়বস্তুও আটকাতে হলে `const` বা `List.unmodifiable()` ব্যবহার করুন।

৮. == এবং identical() -এর পার্থক্য?

`==` মান তুলনা করে (override করা যায়)। `identical()` যাচাই করে দুটি রেফারেন্স একই অবজেক্টকে নির্দেশ করে কিনা।

৯. == override করলে hashCode -ও কেন করতে হয়?

কারণ `Set` ও `Map` প্রথমে `hashCode` দিয়ে বালতি (bucket) খুঁজে, তারপর `==` দিয়ে মেলায়। `hashCode` না মিললে সমান দুটি অবজেক্ট আলাদা বালতিতে গিয়ে ডুপ্লিকেট তৈরি করবে। নিয়ম: সমান অবজেক্টের `hashCode` অবশ্যই সমান হতে হবে।

১০. abstract class ও interface-এর পার্থক্য?

Dart-এ আলাদা `interface` কীওয়ার্ড নেই — যেকোনো ক্লাসই `interface` হিসেবে ব্যবহার করা যায়। `extends` (একটিমাত্র) — বাস্তবায়ন উত্তরাধিকারে পাওয়া যায়। `implements` (একাধিক) — কেবল চুক্তি, প্রতিটি সদস্য নিজে লিখতে হয়।

১১. Mixin কী এবং কেন দরকার?

Mixin হলো একাধিক ক্লাসে কোড পুনঃব্যবহারের উপায়, যেখানে inheritance ব্যবহার করা অনুচিত। Dart-এ multiple inheritance নেই, কিন্তু `with A, B, C` দিয়ে একাধিক mixin ব্যবহার করা যায়। Flutter উদাহরণ: `SingleTickerProviderStateMixin` ।

১২. on কীওয়ার্ড mixin-এ কী করে?

`mixin X on Y` মানে — X শুধু Y বা Y-এর সন্তানে ব্যবহার করা যাবে। এতে mixin-এর ভিতর থেকে Y-এর সদস্য নিরাপদে ব্যবহার করা যায়।

১৩. Factory constructor কী?

এমন কনস্ট্রাক্টর যা নতুন অবজেক্ট বানাতে বাধ্য নয় — ক্যাশ থেকে, singleton থেকে, বা সাবক্লাসের অবজেক্ট ফেরত দিতে পারে। সবচেয়ে সাধারণ ব্যবহার: `factory User.fromJson(...)` ।

১৪. Singleton কীভাবে বানাবেন?

```
class ApiClient {
  static final ApiClient _instance = ApiClient._internal();
  factory ApiClient() => _instance;
  ApiClient._internal();
}
```

১৫. Getter ও Setter কেন ব্যবহার করবেন?

হিসাব করা প্রপার্টি দিতে (`double get total => ...`), ভিতরের ফিল্ড লুকিয়ে নিয়ন্ত্রিত অ্যাক্সেস দিতে, এবং সেট করার আগে যাচাই করতে।

১৬. Future ও Stream-এর পার্থক্য?

Future — একটি মান, একবার (API কল)। **Stream** — অনেক মান, সময়ের সাথে (চ্যাট বার্তা, লোকেশন আপডেট, Firestore snapshots)।

১৭. async / await কীভাবে কাজ করে?

`async` ফাংশনকে Future রিটার্ন করতে বাধ্য করে। `await` সেই ফাংশনের বাকি অংশকে একটি callback-এ পরিণত করে যা Future সম্পূর্ণ হলে Event Loop চালাবে। এতে **থ্রেড ব্লক হয় না** — অন্য কাজ চলতে থাকে।

১৮. Microtask Queue ও Event Queue-এর পার্থক্য?

Event Loop প্রথমে **Microtask Queue** সম্পূর্ণ খালি করে, তারপর **Event Queue** থেকে একটি নেয়।

`Future.value()` ও `scheduleMicrotask()` microtask-এ যায়; I/O, টাইমার ও ব্যবহারকারীর ইনপুট event queue-তে। তাই microtask-এ ভারী কাজ দিলে UI আটকে যায়।

১৯. Future.wait() কেন গুরুত্বপূর্ণ?

একাধিক স্বাধীন async কাজ **সমান্তরালে** চালায়। পরপর await করলে সময় যোগ হয় (২+২+২=৬ সেকেন্ড),

`Future.wait` -এ সবচেয়ে ধীরটির সমান (২ সেকেন্ড)।

২০. Isolate কী এবং async থেকে কীভাবে আলাদা?

`async` একই থ্রেডে চলে — কেবল **অপেক্ষার** সময় অন্য কাজ করতে দেয়, কিন্তু ভারী **হিসাব** UI আটকায়।

Isolate সত্যিকারের আলাদা থ্রেড ও আলাদা মেমোরি — মেমোরি শেয়ার করে না, বার্তা পাঠিয়ে যোগাযোগ করে। Flutter-এ `compute()` এর সহজ রূপ।

২১. async* ও yield কী?

`async*` ফাংশন একটি Stream রিটার্ন করে, আর `yield` দিয়ে একটির পর একটি মান পাঠানো যায়।

`yield*` দিয়ে আরেকটি stream-এর সব মান পাঠানো যায়।

২২. single-subscription ও broadcast Stream-এর পার্থক্য?

সাধারণ Stream-এ একজনই listen করতে পারে; দ্বিতীয়বার করলে এরর। `.asBroadcastStream()` বা `StreamController.broadcast()` দিয়ে একাধিক শ্রোতা সম্ভব — তবে broadcast stream পুরনো মান ধরে রাখে না।

২৩. Generic কেন ব্যবহার করবেন?

টাইপ নিরাপত্তা বজায় রেখে কোড পুনঃব্যবহার করতে। `List<String>` ব্যবহার করলে ভুল টাইপ ঢোকানো কম্পাইল টাইমেই ধরা পড়ে এবং কাস্ট করতে হয় না।

২৪. dynamic , Object ও var -এর পার্থক্য?

`var` — টাইপ অনুমান হয় ও স্থির থাকে (টাইপ চেক চালু)। `Object` — সব টাইপের বাবা, তবে ব্যবহারের আগে কাস্ট বা `is` চেক লাগে (নিরাপদ)। `dynamic` — টাইপ চেক সম্পূর্ণ বন্ধ, ভুল রানটাইমে ধরা পড়ে (বিপজ্জনক)।

২৫. Extension method কী?

বিদ্যমান ক্লাসের সোর্স কোড না বদলে তাতে নতুন মেথড যোগ করা — যেমন `String.isValidEmail` । সাবক্লাস বানানোর চেয়ে হালকা ও পরিষ্কার।

২৬. Exception ও Error -এর পার্থক্য?

Exception — প্রত্যাশিত সমস্যা যা ধরে সামলানো উচিত (নেট নেই)। **Error** — প্রোগ্রামারের ভুল যা ধরা নয়, ঠিক করা উচিত (`RangeError` , `NoSuchMethodError`)।

২৭. rethrow কী?

ধরা exception-টিকে মূল stack trace অক্ষত রেখে আবার উপরে ছুড়ে দেয়। `throw e` লিখলে stack trace হারিয়ে যায়, তাই ডিবাগ করা কঠিন হয়।

২৮. Cascade operator (..) কী করে?

একই অবজেক্টে পরপর কয়েকটি কাজ করতে দেয় এবং প্রতিবার অবজেক্টটিই ফেরত দেয় — `Paint().color = red.strokeWidth = 4` । মেথড কী রিটার্ন করছে তা উপেক্ষা করে।

২৯. Spread operator ও null-aware spread?

`[...list1, ...list2]` দিয়ে লিস্ট জোড়া লাগানো। `...?maybeNull` — null হলে কিছুই যোগ হবে না, এরর হবে না। Flutter-এ children লিস্টে খুব কাজে লাগে।

৩০. map , where , fold , reduce -এর পার্থক্য?

`map` — প্রতিটিকে রূপান্তর করে সমান দৈর্ঘ্যের নতুন সংগ্রহ। `where` — শর্তে মেলাগুলো রেখে ফিল্টার।
`reduce` — সব মিলিয়ে একটি মান (খালি লিস্ট এরর)। `fold` — `reduce`-এর মতো কিন্তু প্রাথমিক মান দেওয়া যায়, তাই খালি লিস্টও নিরাপদ এবং ভিন্ন টাইপ ফেরত দিতে পারে।

৩১. List , Set , Map — কোনটি কখন?

List — ক্রমযুক্ত, ডুপ্লিকেট চলে। Set — অনন্য মান, `contains` O(1)। Map — key দিয়ে দ্রুত খোঁজা।

৩২. firstWhere কিছু না পেলে কী হয়?

`StateError` ছুড়ে দেয়। তাই সবসময় `orElse:` দিন, অথবা `collection` প্যাকেজের `firstWhereOrNull` ব্যবহার করুন।

৩৩. Dart-এ garbage collection কীভাবে কাজ করে?

Generational GC — নতুন অবজেক্ট "young generation"-এ যায় যা দ্রুত ও ঘন ঘন পরিষ্কার হয়; যেগুলো টিকে থাকে সেগুলো "old generation"-এ সরে যায়। Flutter-এ widget-এর মতো স্বল্পায়ু অবজেক্ট প্রচুর তৈরি হয় বলে এই কৌশল খুব কার্যকর।

৩৪. typedef কী?

একটি টাইপের (বিশেষত ফাংশন টাইপের) সংক্ষিপ্ত নাম — `typedef Validator = String? Function(String?);`। কোড পড়তে সহজ হয়।

৩৫. Record (Dart 3) কী?

ক্লাস না বানিয়ে একাধিক মান একসাথে ফেরত দেওয়ার উপায় — `(String name, int age) getUser() => ('রহিম', 25);`। নামযুক্তও হতে পারে: `{String name, int age}`।

৩৬. sealed class (Dart 3) কী?

এমন ক্লাস যার সব সাবক্লাস একই ফাইলে থাকতে হয়। ফলে `switch` -এ সব কেস আছে কিনা কম্পাইলার যাচাই করতে পারে — state modelling-এ (Loading/Success/Error) দারুণ কাজে লাগে।

৩৭. Positional ও Named parameter-এর পার্থক্য?

Positional — ক্রম অনুযায়ী (`[]` দিয়ে ঐচ্ছিক)। Named — নাম ধরে (`{}` দিয়ে), ক্রম গুরুত্বপূর্ণ নয়, `required` দেওয়া যায়। Flutter প্রায় সর্বত্র named ব্যবহার করে কারণ পড়তে সহজ।

৩৮. `const` কনস্ট্রাক্টরের শর্ত কী?

ক্লাসের সব ফিল্ড `final` হতে হবে এবং কনস্ট্রাক্টরের `body` থাকতে পারবে না। বিনিময়ে একই মানের অবজেক্ট মেমোরিতে একবারই তৈরি হয় — Flutter widget-এ এটি বড় পারফরম্যান্স সুবিধা।

৩৯. Closure কী?

এমন ফাংশন যা নিজের বাইরের স্কোপের ভ্যারিয়েবল "মনে রাখে" — মূল ফাংশন শেষ হয়ে যাওয়ার পরেও।

`makeMultiplier(3)` যে ফাংশন ফেরত দেয় তা `3` মনে রাখে।

৪০. `covariant` কীওয়ার্ড কেন লাগে?

`override` করার সময় প্যারামিটারের টাইপ সংকুচিত করতে চাইলে। Flutter-এ `didUpdateWidget(covariant MyWidget oldWidget)` -এ এটি দেখা যায়।

অধ্যায় ৩০ — Flutter ইন্টারভিউ প্রশ্ন

৪১. Flutter কীভাবে React Native থেকে আলাদা?

React Native নেটিভ widget-কে JavaScript bridge দিয়ে নিয়ন্ত্রণ করে। Flutter নিজের ইঞ্জিন (Skia/Impeller) দিয়ে প্রতিটি পিক্সেল নিজে আঁকে — কোনো bridge নেই, তাই দ্রুত এবং সব প্ল্যাটফর্মে ছবছ একরকম দেখায়।

৪২. Widget কী?

UI-এর একটি অংশের **immutable বর্ণনা** — "আমি কেমন দেখতে চাই"। এটি নিজে কিছু আঁকে না; Element ও RenderObject আসল কাজ করে।

৪৩. তিনটি গাছ (Widget, Element, RenderObject) ব্যাখ্যা করুন।

Widget Tree — হালকা, immutable বর্ণনা; প্রতি rebuild-এ নতুন। **Element Tree** — Widget ও RenderObject-এর সেতু; State এখানে থাকে; যতটা সম্ভব পুনঃব্যবহার হয়। **RenderObject Tree** — আসল layout ও paint; ভারী, খুব কমই নতুন হয়। এ কারণেই widget নতুন করে বানানো সম্ভব।

৪৪. StatelessWidget ও StatefulWidget-এর পার্থক্য?

StatelessWidget-এর নিজস্ব পরিবর্তনশীল অবস্থা নেই — সব ফিল্ড final। StatefulWidget-এর সাথে একটি **State** অবজেক্ট থাকে যা rebuild-এর মধ্যেও বেঁচে থাকে এবং **setState()** দিয়ে পরিবর্তন ঘোষণা করে।

৪৫. StatefulWidget-এ দুটি ক্লাস কেন?

Widget immutable হতে হয় (প্রতি rebuild-এ নতুন তৈরি হয়), কিন্তু state-কে টিকে থাকতে হয়। তাই পরিবর্তনশীল অংশ আলাদা **State** ক্লাসে রাখা হয় যা Element tree-তে সংরক্ষিত থাকে।

৪৬. setState() ঠিক কী করে?

Element-টিকে "dirty" চিহ্নিত করে যাতে পরের ফ্রেমে **build()** আবার চলে। এটি সাথে সাথে UI বদলায় না — পরবর্তী ফ্রেমে হয়। এবং এটি শুধু **সেই State** ও তার নিচের subtree-কে প্রভাবিত করে।

৪৭. initState, didChangeDependencies, build, didUpdateWidget, dispose — কখন চলে?

initState একবার (তৈরির সময়) · **didChangeDependencies** initState-এর পর এবং নির্ভরশীল InheritedWidget বদলালে · **build** প্রতি rebuild-এ · **didUpdateWidget** প্যারেন্ট নতুন কনফিগ দিলে · **dispose** স্থায়ীভাবে সরানোর সময়।

৪৮. initState -এ context ব্যবহার করা যায় না কেন?

তখনো Element পুরোপুরি গাছে যুক্ত হয়নি, তাই `Theme.of(context)` -এর মতো `InheritedWidget` লুকআপ কাজ করে না। প্রয়োজন হলে `didChangeDependencies` ব্যবহার করুন, অথবা

```
WidgetsBinding.instance.addPostFrameCallback |
```

৪৯. BuildContext কী?

এটি আসলে সেই widget-এর **Element**। গাছে নিজের অবস্থান জানে, তাই `of(context)` দিয়ে **উপরের দিকে** নিকটতম Theme/Navigator/Provider খুঁজে পাওয়া যায়।

৫০. "No Scaffold widget found" এরর কেন হয়?

আপনি এমন context ব্যবহার করছেন যা Scaffold-এর **উপরে**, ভিতরে নয়। সমাধান — `Builder` দিয়ে নতুন context নিন, অথবা অংশটি আলাদা widget-এ সরান।

৫১. Key কী এবং কখন লাগে?

Key দিয়ে Flutter চেনে কোন নতুন widget কোন পুরনো Element-এর সাথে মিলবে। একই ধরনের widget-এর **তালিকায় ক্রম বদলালে** (যোগ, বাদ, সরানো) key ছাড়া state ভুল আইটেমে চলে যায়।

৫২. ValueKey, ObjectKey, UniqueKey, GlobalKey — পার্থক্য?

`ValueKey` — একটি মান দিয়ে (id)। `ObjectKey` — পুরো অবজেক্টের পরিচয় দিয়ে। `UniqueKey` — প্রতিবার নতুন, জোর করে নতুন Element বানাতে। `GlobalKey` — পুরো অ্যাপে অনন্য, অন্য জায়গা থেকে state/context ধরতে; ব্যবহৃত।

৫৩. Flutter-এর লেআউট নিয়ম কী?

"Constraints go down, sizes go up, parent sets position." প্যারেন্ট সীমা দেয় → চাইল্ড সীমার মধ্যে মাপ ঠিক করে জানায় → প্যারেন্ট অবস্থান ঠিক করে।

৫৪. tight ও loose constraint কী?

tight — min = max, চাইল্ডের পছন্দ নেই (Scaffold body, Expanded)। loose — min = 0, চাইল্ড ছোট হতে পারে (Center, Align)। এ কারণেই `Center` ছাড়া `Container(width: 200)` কাজ করে না।

৫৫. "RenderFlex overflowed" কেন হয়, কীভাবে ঠিক করবেন?

চাইল্ডের মোট মাপ প্যারেন্টের জায়গার চেয়ে বড়। সমাধান — Row/Column-এর চাইল্ডকে Expanded / Flexible দিয়ে মোড়ানো, টেক্সটে `overflow: ellipsis`, অথবা পুরোটা `SingleChildScrollView`-এ রাখা।

৫৬. "Vertical viewport was given unbounded height" কেন?

Column বা `SingleChildScrollView`-এর ভিতরে `ListView` রেখেছেন — প্যারেন্ট অসীম উচ্চতা দিচ্ছে। সমাধান: `Expanded`, নির্দিষ্ট উচ্চতা, বা `shrinkWrap: true` + `NeverScrollableScrollPhysics`।

৫৭. `shrinkWrap: true` এর অসুবিধা কী?

`ListView` সব চাইল্ড একসাথে তৈরি করে উচ্চতা মাপে — lazy loading-এর সুবিধা নষ্ট হয়। বড় লিস্টে ব্যবহার করবেন না।

৫৮. Expanded ও Flexible-এর পার্থক্য?

`Expanded` = `Flexible(fit: FlexFit.tight)` — বরাদ্দ জায়গা পুরোটা দখল করতেই হবে।

`Flexible(fit: loose)` — সর্বোচ্চ ততটুকু নিতে পারে, কিন্তু কম হলেও চলবে।

৫৯. ListView, ListView.builder ও ListView.separated?

সাধারণ `ListView` সব চাইল্ড একসাথে বানায় (অল্প ও নির্দিষ্ট আইটেমে ঠিক আছে)। `.builder` কেবল দৃশ্যমান আইটেম বানায় (lazy) — বড় লিস্টে অপরিহার্য। `.separated` builder-এর মতো, সাথে আইটেমের মাঝে বিভাজক।

৬০. Sliver কী?

স্ক্রলযোগ্য এলাকার একটি অংশ যা স্ক্রল অবস্থার সাথে মানিয়ে নিতে পারে। `CustomScrollView`-এর ভিতরে একাধিক sliver (`SliverAppBar`, `SliverList`, `SliverGrid`) দিয়ে কোলাপসিং হেডারসহ জটিল স্ক্রল বানানো যায়।

৬১. InheritedWidget কী?

গাছের নিচের দিকে ডেটা দক্ষতার সাথে ছড়িয়ে দেওয়ার প্রক্রিয়া — নিচের widget `of(context)` দিয়ে $O(1)$ সময়ে ডেটা পায় এবং ডেটা বদলালে শুধু নির্ভরশীলরাই rebuild হয়। Theme, MediaQuery, Provider — সবাই এটি ব্যবহার করে।

৬২. Provider কীভাবে কাজ করে?

ভিতরে InheritedWidget ব্যবহার করে। `ChangeNotifier` `notifyListeners()` ডাকলে Provider তার শ্রোতা Element-গুলোকে dirty চিহ্নিত করে, ফলে সেগুলোই কেবল rebuild হয়।

৬৩. `context.watch`, `context.read`, `context.select` — পার্থক্য?

`watch` — মান পড়ে ও বদলালে rebuild (build-এ ব্যবহার)। `read` — একবার পড়ে, rebuild নয় (callback-এ ব্যবহার)। `select` — শুধু নির্দিষ্ট একটি মান দেখে, সেটি বদলালেই কেবল rebuild (সবচেয়ে দক্ষ)।

৬৪. Provider বনাম Riverpod বনাম Bloc?

Provider — সহজ, BuildContext-নির্ভর, ছোট-মাকারি অ্যাপে চমৎকার। **Riverpod** — context লাগে না, compile-safe, টেস্ট সহজ, `.when()` দিয়ে async অবস্থা সুন্দরভাবে সামলায়। **Bloc** — event→state কঠোর প্রবাহ, বড় টিমে অনুমানযোগ্য ও টেস্টযোগ্য, কিন্তু অনেক বয়লারপ্লেট।

৬৫. FutureBuilder ও StreamBuilder?

FutureBuilder একটি Future-এর ফলাফল থেকে UI বানায় (একবার)। StreamBuilder Stream-এর প্রতিটি নতুন মানের UI আপডেট করে (বারবার)।

৬৬. FutureBuilder-এ `future`: সরাসরি ফাংশন কল করলে সমস্যা কী?

প্রতি rebuild-এ নতুন Future তৈরি হয়ে বারবার API কল হবে ও লোডিং ঝিকমিকি করবে। সমাধান — Future-টি `initState` -এ একটি ভ্যারিয়েবলে রেখে সেটি ব্যবহার করুন।

৬৭. Hot Reload ও Hot Restart-এর পার্থক্য?

Hot Reload — পরিবর্তিত কোড ইনজেক্ট করে rebuild করে, **State বেঁচে থাকে** (~১ সেকেন্ড)। Hot Restart — অ্যাপ পুরো রিস্টার্ট, State মুছে যায় (~৫ সেকেন্ড)। `main()`, `initState()`, global variable, enum বা ক্লাসের গঠন বদলালে Hot Restart লাগে।

৬৮. তিনটি বিন্ড মোড কী কী?

Debug (JIT, Hot Reload, ধীর) · Profile (AOT + পারফরম্যান্স টুল) · Release (AOT, সবচেয়ে দ্রুত ও ছোট, assert ও debug টুল বাদ)।

৬৯. `const widget` কেন পারফরম্যান্স বাড়ায়?

`const` widget কম্পাইল টাইমেই তৈরি হয়ে canonicalized হয় — rebuild-এ Flutter দেখে অবজেক্টটি **অভিন্ন**, তাই তার subtree সম্পূর্ণ বাদ দেয় (কোনো তুলনা বা rebuild নয়)।

৭০. rebuild কীভাবে কমাবেন?

const ব্যবহার · widget ছোট ছোট ভাগে ভাগ করা · Consumer/Selector দিয়ে rebuild-এর পরিসর সংকুচিত করা · AnimatedBuilder-এ `child` প্যারামিটার ব্যবহার · state যতটা সম্ভব নিচে রাখা।

৭১. Widget ফাংশন নাকি আলাদা ক্লাস — কোনটি ভালো?

আলাদা ক্লাস। কারণ ক্লাসের নিজস্ব Element থাকে, তাই স্বাধীনভাবে rebuild হতে পারে এবং `const` ব্যবহার করা যায়। ফাংশন ব্যবহার করলে পুরো প্যারেন্ট rebuild হয়।

৭২. Implicit ও Explicit অ্যানিমেশন?

Implicit (`AnimatedContainer`) — মান বদলালেই নিজে থেকে চলে, কোড কম। Explicit (`AnimationController`) — শুরু/থামা/পুনরাবৃত্তি সবকিছু আপনার নিয়ন্ত্রণে, তবে `TickerProvider` ও `dispose` লাগে।

৭৩. SingleTickerProviderStateMixin কেন লাগে?

AnimationController-কে প্রতি ফ্রেমে সংকেত (tick) দিতে একটি `Ticker` দরকার, যা `vsync` হিসেবে পাঠাতে হয়। এটি স্ক্রিন বন্ধ থাকলে ticker থামিয়ে ব্যাটারিও বাঁচায়। একাধিক controller থাকলে `TickerProviderStateMixin` ।

৭৪. Hero অ্যানিমেশন কীভাবে কাজ করে?

দুই রুটে একই `tag` -এর Hero থাকলে Navigator ট্রানজিশনের সময় widget-টিকে একটি overlay-তে তুলে নিয়ে পুরনো অবস্থান থেকে নতুন অবস্থানে উড়িয়ে নেয়।

৭৫. মেমোরি লিক কীভাবে এড়াবেন?

`dispose()` -এ সব controller, StreamSubscription, Timer, FocusNode, AnimationController বন্ধ করুন; observer সরান। async কাজের পরে `if (!mounted) return;` লিখুন।

৭৬. "setState() called after dispose()" কেন হয়?

async কাজ শেষ হওয়ার আগেই ব্যবহারকারী পেজ ছেড়ে চলে গেছে, কিন্তু callback তখনো setState ডাকছে। সমাধান — setState-এর আগে `if (!mounted) return;` ।

৭৭. mounted কী?

State অবজেক্টটি এখনো Element tree-তে যুক্ত আছে কিনা তা বলে দেওয়া bool। `await` -এর পরে context বা setState ব্যবহারের আগে অবশ্যই যাচাই করুন।

৭৮. Flutter-এ ছবি ক্যাশ কীভাবে হয়?

`Image.network` মেমোরিতে ক্যাশ করে কিন্তু ডিস্কে নয় — অ্যাপ বন্ধ করলে আবার ডাউনলোড হয়।
`cached_network_image` ডিস্কেও ক্যাশ করে, তাই প্রোডাকশনে এটিই ব্যবহার করা উচিত।

৭৯. Platform Channel কী?

Dart থেকে নেটিভ কোড (Kotlin/Swift) ডাকার সেতু — `MethodChannel` (একবারের কল), `EventChannel` (নেটিভ থেকে stream), `BasicMessageChannel`। যখন কোনো ফিচারের Flutter প্যাকেজ নেই তখন লাগে।

৮০. WidgetsFlutterBinding.ensureInitialized() কেন লাগে?

`runApp()` -এর আগে যদি প্ল্যাটফর্ম চ্যানেল ব্যবহার করতে হয় (Firebase init, SharedPreferences, ওরিয়েন্টেশন সেট), তাহলে আগে Flutter ইঞ্জিনের বাইন্ডিং তৈরি করে নিতে হয়।

৮১. MediaQuery.of বনাম MediaQuery.sizeOf ?

`of` পুরো `MediaQueryData`-র উপর নির্ভরশীল করে — কিবোর্ড উঠলেও rebuild হয়। `sizeOf` শুধু মাপের উপর নির্ভর করে, তাই অপ্ৰয়োজনীয় rebuild কম।

৮২. SafeArea কী করে?

নচ, স্ট্যাটাস বার ও জেসচার বারের নিচে কনটেন্ট চলে যাওয়া ঠেকাতে প্রয়োজনীয় padding যোগ করে।

৮৩. Material ও Cupertino কী?

Material = Google-এর Android ডিজাইন ভাষা। Cupertino = Apple-এর iOS স্টাইল। `Platform.isIOS` দেখে আলাদা widget ব্যবহার করা যায়, তবে বেশিরভাগ অ্যাপ দুই প্ল্যাটফর্মেই Material ব্যবহার করে।

৮৪. অ্যাপের সাইজ কীভাবে কমাবেন?

`--split-per-abi` বা App Bundle ব্যবহার · অব্যবহৃত প্যাকেজ ও অ্যাসেট বাদ · ছবি WebP-তে রূপান্তর ও আগেই রিসাইজ · অপ্ৰয়োজনীয় ফন্ট ওজন বাদ · `--analyze-size` দিয়ে যাচাই।

৮৫. Flutter Web-এর সীমাবদ্ধতা কী?

প্রথম লোড ভারী (কয়েক MB), SEO দুর্বল, টেক্সট নির্বাচন ও ব্রাউজার-নেটিভ আচরণ সীমিত। ড্যাশবোর্ড বা অভ্যন্তরীণ টুলে ভালো, পাবলিক মার্কেটিং সাইটে নয়।

অধ্যায় ৩১ — আর্কিটেকচার ও অ্যাডভান্স প্রশ্ন

৮৬. Clean Architecture কী এবং Flutter-এ কীভাবে প্রয়োগ করবেন?

তিন স্তর — **Presentation** (UI, state), **Domain** (মডেল, repository interface, use case; বিশুদ্ধ Dart), **Data** (API, ডেটাবেজ, repository implementation)। নির্ভরতা সবসময় **ভিতরের দিকে** যায় — Domain কাউকে চেনে না। সুবিধা: টেস্টযোগ্যতা ও ডেটা উৎস বদলানোর স্বাধীনতা।

৮৭. Repository pattern কেন ব্যবহার করবেন?

ডেটা কোথা থেকে আসছে (API, ক্যাশ, ডেটাবেজ) তা UI থেকে লুকিয়ে রাখে। ফলে অফলাইন সাপোর্ট যোগ করা বা REST থেকে GraphQL-এ যাওয়া UI না ছুঁয়েই সম্ভব, এবং টেস্টে fake repository বসানো যায়।

৮৮. Dependency Injection কী এবং Flutter-এ কীভাবে করবেন?

অবজেক্ট নিজে তার নির্ভরতা তৈরি না করে বাইরে থেকে গ্রহণ করে। Flutter-এ — কনস্ট্রাক্টর ইনজেকশন (সবচেয়ে সহজ), `Provider`, বা `get_it` service locator। সুবিধা: টেস্টে মক বসানো যায়, কাপলিং কমে।

৮৯. SOLID নীতিগুলো Flutter-এ কীভাবে প্রয়োগ হয়?

S — একটি widget/ক্লাসের একটিই দায়িত্ব। **O** — নতুন আচরণ যোগ করুন কম্পোজিশন দিয়ে, বিদ্যমান কোড না বদলে। **L** — সাবক্লাস প্যারেন্টের জায়গায় বসতে পারবে। **I** — বড় interface-এর বদলে ছোট ছোট। **D** — concrete ক্লাসের নয়, abstract-এর উপর নির্ভর করুন।

৯০. state কোথায় রাখা উচিত?

যত **নিচে** সম্ভব — যেসব widget-এর সেটি দরকার তাদের নিকটতম সাধারণ পূর্বপুরুষে ("lifting state up")। উপরে রাখলে অপ্রয়োজনীয় rebuild বাড়ে। শুধু একটি widget-এ দরকার হলে `setState`-ই যথেষ্ট।

৯১. ephemeral state ও app state-এর পার্থক্য?

Ephemeral — একটি widget-এর নিজস্ব, অন্য কারো লাগে না (ট্যাব ইনডেক্স, চেকবক্স) → `setState`। **App state** — একাধিক স্ক্রিনে দরকার (লগইন, কার্ট, থিম) → `Provider/Riverpod/Bloc`।

৯২. অফলাইন সাপোর্ট কীভাবে বানাবেন?

Repository-তে cache-first বা network-first কৌশল: API সফল হলে ডেটা লোকাল ডেটাবেজে সেভ করুন; ব্যর্থ হলে ক্যাশ থেকে দিন। লেখার কাজ একটি "outbox" টেবিলে জমিয়ে রাখুন এবং `connectivity_plus` দিয়ে নেট ফিরলে সিঙ্ক করুন।

৯৩. টোকেন রিফ্রেশ কীভাবে সামলাবেন?

HTTP interceptor-এ 401 ধরুন → refresh token দিয়ে নতুন access token নিন → মূল রিকোয়েস্ট আবার পাঠান। একসাথে অনেক রিকোয়েস্ট 401 পেলে যাতে একবারই রিফ্রেশ হয় সেজন্য একটি lock/queue রাখুন। রিফ্রেশও ব্যর্থ হলে লগআউট করান।

৯৪. API কী ও গোপন তথ্য কোথায় রাখবেন?

কখনোই সোর্স কোডে হার্ডকোড করবেন না। `--dart-define` বা `.env` (git-ignored) ব্যবহার করুন। তবে মনে রাখবেন — **ক্লায়েন্ট অ্যাপে কিছুই সত্যিকারের গোপন নয়**; সংবেদনশীল কী সার্ভারে রাখুন এবং অ্যাপ শুধু আপনার নিজের ব্যাকএন্ডে কল করুক।

৯৫. Flutter অ্যাপে নিরাপত্তার মূল বিষয় কী কী?

HTTPS বাধ্যতামূলক · টোকেন `flutter_secure_storage`-এ · সংবেদনশীল অ্যাপে certificate pinning · root/jailbreak শনাক্তকরণ · রিলিজে code obfuscation (`--obfuscate --split-debug-info`) · সব ভ্যালিডেশন সার্ভারেও পুনরায় করা।

৯৬. বড় লিস্টের পারফরম্যান্স কীভাবে অপ্টিমাইজ করবেন?

`ListView.builder` ব্যবহার · আইটেমে `const` ও স্থির উচ্চতা (`itemExtent`) · ছবি ক্যাশ ও সঠিক মাপে · pagination · `shrinkWrap` এড়ানো · প্রতিটি আইটেমে ভারী হিসাব না করা।

৯৭. jank কী এবং কীভাবে ধরবেন?

একটি ফ্রেম আঁকতে ১৬ms (৬০fps) এর বেশি লাগলে স্ক্রল আটকে আটকে দেখায় — এটিই jank। ধরার উপায়: profile মোডে চালিয়ে DevTools-এর Performance ট্যাব দেখুন; লাল বার মানে সমস্যা।

৯৮. build() কেন বিশুদ্ধ (pure) হতে হবে?

Flutter যেকোনো সময়, যত খুশি বার build() ডাকতে পারে। তাই এতে কোনো পার্শ্বপ্রতিক্রিয়া (API কল, setState, ফাইল লেখা) থাকলে অনির্দেশ্য আচরণ ও অসীম লুপ হবে।

৯৯. একই widget-এর একাধিক ইনস্ট্যান্স state শেয়ার করে কেন?

করে না — প্রতিটির নিজস্ব State অবজেক্ট থাকে। কিন্তু key ছাড়া লিস্ট ক্রম বদলালে Flutter ভুল Element-এর সাথে মিলিয়ে দেয়, ফলে state ভুল আইটেমে দেখা যায়।

১০০. GetX কেন এড়িয়ে চলার পরামর্শ দেওয়া হয়?

এটি state management, routing, DI ও utility সব একসাথে করে — ফলে অ্যাপ GetX-এর সাথে শক্তভাবে বাঁধা পড়ে। এটি BuildContext ছাড়া নেভিগেশন ও global state উৎসাহিত করে যা Flutter-এর নকশার বিরুদ্ধে, টেস্ট করা কঠিন করে এবং বড় অ্যাপে রক্ষণাবেক্ষণ দুঃসাধ্য হয়ে ওঠে।

১০১. widget test ও integration test-এর পার্থক্য?

Widget test নকল পরিবেশে একটি widget পরীক্ষা করে — দ্রুত, ডিভাইস লাগে না। Integration test আসল ডিভাইস/ইমুলেটরে পুরো অ্যাপ চালিয়ে ব্যবহারকারীর প্রবাহ পরীক্ষা করে — ধীর কিন্তু বাস্তবসম্মত।

১০২. pumpAndSettle() ও pump() -এর পার্থক্য?

pump() একটি ফ্রেম আঁকে। pumpAndSettle() সব অ্যানিমেশন শেষ না হওয়া পর্যন্ত বারবার ফ্রেম আঁকে। অসীম অ্যানিমেশন (যেমন CircularProgressIndicator) থাকলে pumpAndSettle টাইমআউট করবে।

১০৩. CI/CD-তে Flutter অ্যাপ কীভাবে সেট করবেন?

GitHub Actions/Codemagic-এ: checkout → flutter-action → pub get → analyze → test → build appbundle → artifact আপলোড বা Play Store-এ স্বয়ংক্রিয় প্রকাশ (fastlane)। keystore ও কী secrets-এ রাখুন।

১০৪. অ্যাপের সংস্করণ কীভাবে পরিচালনা করবেন?

version: 1.2.3+45 — 1.2.3 হলো versionName (ব্যবহারকারী দেখে), 45 হলো versionCode/buildNumber (প্রতি আপলোডে বাড়াতেই হবে, নাহলে Play Store বাতিল করবে)।

১০৫. Flutter-এ localization (বহুভাষা) কীভাবে করবেন?

flutter_localizations + intl ব্যবহার করে .arb ফাইলে অনুবাদ রাখুন, flutter gen-l10n চালান, তারপর AppLocalizations.of(context).welcome দিয়ে ব্যবহার করুন। MaterialApp-এ supportedLocales ও localizationsDelegates সেট করতে হবে।

অধ্যায় ৩২ — কোডিং সমস্যা ও লাইভ টাস্ক

ইন্টারভিউতে প্রায়ই লাইভ কোড লিখতে বলা হয়। নিচের সমস্যাগুলো সবচেয়ে বেশি জিজ্ঞেস করা হয়। প্রতিটি নিজে সমাধান করার চেষ্টা করুন, তারপর উত্তর মিলিয়ে দেখুন।

৩২.১ Dart লজিক সমস্যা

```
// ১. একটি String উল্টে দিন
String reverse(String s) => s.split('').reversed.join();

// ২. Palindrome কিনা যাচাই করুন
bool isPalindrome(String s) {
  final clean = s.toLowerCase().replaceAll(RegExp(r'^a-z0-9'), '');
  return clean == clean.split('').reversed.join();
}

// ৩. লিস্ট থেকে ডুপ্লিকেট বাদ দিন (ক্রম রেখে)
List<T> unique<T>(List<T> list) => list.toSet().toList();

// ৪. দ্বিতীয় বৃহত্তম সংখ্যা
int? secondLargest(List<int> nums) {
  final s = nums.toSet().toList()..sort();
  return s.length < 2 ? null : s[s.length - 2];
}

// ৫. একটি String-এ প্রতিটি অক্ষর কতবার আছে
Map<String, int> charCount(String s) {
  final map = <String, int>{};
  for (final c in s.split('')) {
    map[c] = (map[c] ?? 0) + 1;
  }
  return map;
}

// ৬. প্রথম যে অক্ষর একবারই আছে
String? firstUnique(String s) {
  final counts = charCount(s);
  for (final c in s.split('')) {
    if (counts[c] == 1) return c;
  }
  return null;
}

// ৭. Fibonacci (মেমোইজেশনসহ)
final _memo = <int, int>{};
int fib(int n) {
  if (n <= 1) return n;
  return _memo[n] ??= fib(n - 1) + fib(n - 2);
}
```

```

// ৮. FizzBuzz
void fizzBuzz(int n) {
    for (var i = 1; i <= n; i++) {
        final s = (i % 15 == 0) ? 'FizzBuzz'
            : (i % 3 == 0) ? 'Fizz'
            : (i % 5 == 0) ? 'Buzz' : '$i';
        print(s);
    }
}

// ৯. অবজেক্টের লিস্ট গ্রুপ করুন
Map<K, List<T>> groupBy<T, K>(List<T> items, K Function(T) key) {
    final map = <K, List<T>>{};
    for (final item in items) {
        map.putIfAbsent(key(item), () => []).add(item);
    }
    return map;
}

// ১০. নেস্টেড লিস্ট সমতল করুন
List flatten(List list) => list.expand(
    (e) => e is List ? flatten(e) : [e]).toList();

// ১১. দুটি sorted লিস্ট একত্র করুন
List<int> merge(List<int> a, List<int> b) {
    final r = <int>[];
    int i = 0, j = 0;
    while (i < a.length && j < b.length) {
        r.add(a[i] <= b[j] ? a[i++] : b[j++]);
    }
    return r..addAll(a.sublist(i))..addAll(b.sublist(j));
}

// ১২. Binary search
int binarySearch(List<int> list, int target) {
    int low = 0, high = list.length - 1;
    while (low <= high) {
        final mid = low + (high - low) ~/ 2;
        if (list[mid] == target) return mid;
        if (list[mid] < target) { low = mid + 1; } else { high = mid - 1; }
    }
    return -1;
}

// ১৩. দুটি সংখ্যার যোগফল target হয় এমন জোড়া (O(n))
List<int>? twoSum(List<int> nums, int target) {
    final seen = <int, int>{};
    for (var i = 0; i < nums.length; i++) {
        final need = target - nums[i];
        if (seen.containsKey(need)) return [seen[need]!, i];
        seen[nums[i]] = i;
    }
    return null;
}

```

```
// ১৪. Debounce – সার্চ বক্সে খুব দরকারি
class Debouncer {
  final Duration delay;
  Timer? _timer;
  Debouncer({this.delay = const Duration(milliseconds: 500)});

  void run(VoidCallback action) {
    _timer?.cancel();
    _timer = Timer(delay, action);
  }

  void dispose() => _timer?.cancel();
}
```

৩২.২ Flutter লাইভ টাস্ক

সাধারণত যা বানাতে বলা হয় (২০-৪৫ মিনিট)

১. কাউন্টার অ্যাপ — বাড়ানো, কমানো, রিসেট। শূন্যের নিচে যাবে না।
২. টুডু লিস্ট — যোগ, মুছে ফেলা, সম্পন্ন চিহ্নিত করা।
৩. লগইন ফর্ম — ভ্যালিডেশন ও লোডিং অবস্থাসহ।
৪. API থেকে লিস্ট — লোডিং, এরর ও খালি অবস্থা সামলানো (এটিই সবচেয়ে বেশি আসে)।
৫. সার্চ ফিল্টার — debounce সহ।
৬. প্রোফাইল কার্ড UI — একটি ডিজাইন দেখিয়ে ছবছ বানাতে বলা।
৭. শপিং কার্ট — quantity বাড়ানো-কমানো ও মোট হিসাব।

লাইভ কোডিংয়ে ভালো করার ৬টি কৌশল

১. কোড লেখার আগে কথা বলুন — "আমি প্রথমে মডেল বানাব, তারপর service, তারপর UI।" নীরবে কোড লিখলে ইন্টারভিউয়ার আপনার চিন্তা বুঝতে পারেন না।
২. প্রশ্ন করুন — "ডেটা কি লোকালে সেভ থাকবে?" প্রশ্ন করা দুর্বলতা নয়, পেশাদারিত্ব।
৩. লোডিং ও এরর অবস্থা ভুলবেন না — শুধু সফল অবস্থা বানালে নম্বর কাটা যায়।
৪. dispose() লিখুন — ইন্টারভিউয়াররা এটি বিশেষভাবে খেয়াল করেন।
৫. নাম ভালো দিন — a, temp নয়।
৬. আটকে গেলে বলুন — "এই API-এর সঠিক নাম মনে নেই, তবে ধারণাটা হলো..." সততা ভালো ছাপ ফেলে।

৩২.৩ ডিবাগিং টাস্ক — "এই কোডে ভুল কী?"

```
// ভুল ১
class _MyState extends State<MyWidget> {
  final controller = TextEditingController();
```

```

// X dispose() নেই → মেমোরি লিক
}

// ভুল ২
Future<void> load() async {
  final data = await api.get();
  setState(() => _data = data);
  // X if (!mounted) return; নেই → dispose-এর পরে setState
}

// ভুল ৩
FutureBuilder(
  future: api.getProducts(),      // X প্রতি rebuild-এ নতুন API কল
  builder: ...,
)

// ভুল ৪
Column(
  children: [
    Text('শিরোনাম'),
    ListView(children: items),    // X unbounded height এরর
  ],
)

// ভুল ৫
onPressed: () {
  final cart = context.watch<CartProvider>(); // X read হওয়া উচিত
  cart.add(product);
}

// ভুল ৬
ListView(
  children: items.map((i) => ItemTile(i)).toList(),
  // X builder ব্যবহার করা উচিত ছিল; এবং key নেই
)

```

অধ্যায় ৩৩ — সিভি, পোর্টফোলিও ও চাকরির কৌশল

৩৩.১ পোর্টফোলিও — যা সত্যিই কাজে দেয়

উপাদান	কেন গুরুত্বপূর্ণ
২-৩টি সম্পূর্ণ প্রজেক্ট	১০টি অসম্পূর্ণের চেয়ে ২টি পূর্ণাঙ্গ অনেক ভালো
GitHub রিপো	পরিচ্ছন্ন কমিট ইতিহাস, অর্থবহ কমিট বার্তা
ভালো README	স্ক্রিনশট, ফিচার তালিকা, প্রযুক্তি, চালানোর নির্দেশনা
APK ডাউনলোড লিংক	নিয়োগকর্তা সাথে সাথে চালিয়ে দেখতে পারেন
১ মিনিটের ডেমো ভিডিও	স্ক্রিন রেকর্ডিং — বিশাল পার্থক্য গড়ে দেয়
Play Store-এ একটি অ্যাপ	বেশিরভাগ প্রার্থীর এটি থাকে না — বড় সুবিধা

README-এর কাঠামো

প্রজেক্টের নাম ও এক লাইনের বর্ণনা → ৩-৫টি স্ক্রিনশট পাশাপাশি → ফিচার তালিকা (bullet) → ব্যবহৃত প্রযুক্তি (Flutter, Provider, sqflite...) → ফোল্ডার কাঠামো → কীভাবে চালাবেন (`flutter pub get` , `flutter run`) → APK লিংক।

৩৩.২ সিভিতে যা লিখবেন

দক্ষতা (Skills)

Flutter • Dart • Provider / Riverpod • REST API • Firebase
SQLite • Git • Clean Architecture • Responsive UI

প্রজেক্ট

ShopEase – ই-কমার্স অ্যাপ [GitHub] [APK]
• REST API থেকে ২০০+ পণ্য লোড, ক্যাটাগরি ফিল্টার ও সার্চ
• Provider দিয়ে কার্ট ও স্টেট ম্যানেজমেন্ট, লোকাল স্টোরেজে সংরক্ষণ
• অফলাইন ক্যাশিং যোগ করে বারবার API কল ৬০% কমানো হয়েছে
• Flutter • Provider • http • sqflite • cached_network_image

সিভির সোনালি নিয়ম

"করেছি" নয়, "কী ফল হয়েছে" লিখুন।

দুর্বল: "একটি ই-কমার্স অ্যাপ বানিয়েছি।"

শক্তিশালী: "অফলাইন ক্যাশিং যোগ করে API কল ৬০% কমিয়েছি ও লোডিং সময় ২ সেকেন্ড থেকে ০.৩ সেকেন্ডে নামিয়েছি।"

সংখ্যা না জানলে আনুমানিক দিন, কিন্তু ফলাফলের ভাষায় লিখুন।

৩৩.৩ ইন্টারভিউয়ের ধাপ ও প্রস্তুতি

ধাপ	যা যাচাই হয়	প্রস্তুতি
স্ক্রিনিং কল	যোগাযোগ ও আগ্রহ	নিজের পরিচয় ৬০ সেকেন্ডে বলার অনুশীলন
টেকনিক্যাল রাউন্ড	Dart, Flutter, state, async	অধ্যায় ২৯-৩১
লাইভ কোডিং	বাস্তবে কোড লিখতে পারা	অধ্যায় ৩২
বাড়ির কাজ	কোডের মান ও কাঠামো	পরিচ্ছন্ন ফোল্ডার, README, টেস্ট
সিস্টেম ডিজাইন	বড় অ্যাপের পরিকল্পনা	অধ্যায় ২৫
HR রাউন্ড	মানানসই কিনা, বেতন	বাজারদর জেনে যান

৩৩.৪ যেসব প্রশ্ন প্রায় সবসময় আসে

"নিজের সম্পর্কে বলুন"

৬০ সেকেন্ডের কাঠামো: আপনি কে + কতদিন Flutter করছেন → সবচেয়ে ভালো প্রজেক্ট ও তাতে আপনার ভূমিকা → কী প্রযুক্তি ব্যবহার করেছেন → কেন এই চাকরিতে আগ্রহী। মুখস্থ শোনাবেন না, কিন্তু আগে থেকে গুছিয়ে রাখুন।

"আপনার সবচেয়ে কঠিন বাগ কী ছিল?"

STAR পদ্ধতিতে বলুন — পরিস্থিতি, কাজ, পদক্ষেপ, ফলাফল। যেমন: "লিস্টে স্ক্রল করলে চেকবক্সের অবস্থা এলোমেলো হয়ে যাচ্ছিল। DevTools দিয়ে দেখি Flutter ভুল Element-এর সাথে মেলাচ্ছে। প্রতিটি আইটেমে

`ValueKey(id)` যোগ করে সমাধান হয়। এখান থেকে key-এর ভূমিকা ভালোভাবে শিখি।"

"এই প্রজেক্টে state management কেন এটি বেছে নিলেন?"

সিদ্ধান্তের কারণ বলুন, কেবল নাম নয়: "অ্যাপে ৮টি স্ক্রিন ও একটি শেয়ারড কার্ট ছিল। Bloc-এর বয়লারপ্লেট এই আকারের জন্য বেশি হতো, আর setState দিয়ে স্ক্রিনের মধ্যে ডেটা শেয়ার করা যেত না — তাই Provider বেছে নিই।"

"আপনি কীভাবে নতুন জিনিস শেখেন?"

সংভাবে বলুন — অফিসিয়াল ডকুমেন্টেশন, ছোট প্রজেক্ট বানিয়ে পরীক্ষা করা, ওপেন সোর্স কোড পড়া। "আমি টিউটোরিয়াল দেখে থেমে থাকি না, দেখে বন্ধ করে নিজে আবার বানাই" — এটি ভালো উত্তর।

"আপনার কি কোনো প্রশ্ন আছে?" (এখানে "না" বলবেন না)

জিজ্ঞেস করুন: টিমের আকার ও কাঠামো কেমন? কোন state management ব্যবহার করেন? কোড রিভিউ প্রক্রিয়া আছে কি? CI/CD আছে? প্রথম ৩ মাসে আমার কাছে কী প্রত্যাশা করবেন?

৩৩.৫ কোথায় কাজ খুঁজবেন

- **বাংলাদেশ:** BDJobs, Skill.jobs, LinkedIn, কোম্পানির নিজস্ব ক্যারিয়ার পেজ, Facebook গ্রুপ (Flutter Bangladesh, BD Job Circular)।
- **ফ্রিল্যান্সিং:** Upwork, Fiverr, Freelancer — শুরুতে ছোট কাজ নিয়ে রেটিং তৈরি করুন।
- **রিমোট:** RemoteOK, WeWorkRemotely, Toptal (কঠিন পরীক্ষা), Turing।
- **নেটওয়ার্কিং:** LinkedIn-এ সপ্তাহে একটি পোস্ট দিন — যা শিখেছেন তা নিয়ে। এটি কাজ খোঁজার চেয়ে বেশি ফল দেয়।

প্রথম চাকরির জন্য বাস্তব পরামর্শ

১. ১০০% যোগ্যতা না থাকলেও আবেদন করুন — जब পোস্টের তালিকা আদর্শ প্রার্থীর, বাস্তবের নয়।
২. ইন্টার্নশিপ বা কম বেতনের প্রথম কাজকে "না" বলবেন না — প্রথম ৬ মাসের অভিজ্ঞতা সবচেয়ে দামি।
৩. প্রত্যাখ্যান স্বাভাবিক — ২০টি আবেদনে ২টি সাক্ষাৎকার হলে সেটি ভালো হার।
৪. প্রতিটি ইন্টারভিউয়ের পর নোট লিখুন — কোন প্রশ্নে আটকেছিলেন, সেটি পড়ে নিন। ৫টি ইন্টারভিউ পরে আপনি অনেক শক্ত প্রার্থী হবেন।

পরিশিষ্ট

দ্রুত রেফারেন্স

চিটশিট, এরর অভিধান, প্যাকেজ তালিকা ও শেখার রিসোর্স।

পরিশিষ্ট ক — চিটশিট

দ্রুত কমান্ড

কমান্ড	কাজ
flutter create --org com.x my_app	প্রজেক্ট তৈরি
flutter run / -d chrome / --release	চালানো
flutter pub add / remove / get / upgrade	প্যাকেজ
flutter pub outdated	পুরনো প্যাকেজ দেখা
flutter clean	ক্যাশ পরিষ্কার
flutter analyze / dart format . / dart fix --apply	কোডের মান
flutter test --coverage	টেস্ট
flutter build apk --split-per-abi	ছোট APK
flutter build appbundle --release	Play Store
flutter doctor -v / flutter upgrade	পরিবেশ

Widget দ্রুত বাছাই

যা করতে চান	Widget
উপর-নিচে সাজানো	Column
পাশাপাশি সাজানো	Row
একটির উপর আরেকটি	Stack + Positioned
জায়গা ভাগ করা	Expanded / Flexible / Spacer
ফাঁকা জায়গা	const SizedBox
জায়গা শেষ হলে পরের লাইনে	Wrap
লম্বা লিস্ট	ListView.builder
গ্রিড	GridView.builder
কোলাপসিং হেডার	CustomScrollView + SliverAppBar
ছোট কনটেন্ট স্ক্রল	SingleChildScrollView
নচ এড়ানো	SafeArea
ট্যাপ যোগ করা	InkWell / GestureDetector
async ডেটা দেখানো	FutureBuilder / StreamBuilder
কোণা গোল করা	ClipRRect
মসৃণ পরিবর্তন	AnimatedContainer
পেজের মধ্যে ছবি ওড়ানো	Hero

পরিশিষ্ট খ — এরর অভিধান

এরর	সমাধান
RenderFlex overflowed	Expanded / ellipsis / SingleChildScrollView
Vertical viewport unbounded height	Expanded, বা shrinkWrap + NeverScrollableScrollPhysics
setState() after dispose()	<code>if (!mounted) return;</code>
Null check operator on null	<code>!</code> এর বদলে <code>??</code> বা null চেক
LateInitializationError	ব্যবহারের আগে <code>late</code> ভ্যারিয়েবল সেট করুন
Could not find the correct Provider	Provider-কে ব্যবহারকারীর উপরে বসান
No Scaffold widget found	Builder ব্যবহার করুন বা আলাদা widget বানান
Unable to load asset	pubspec ইনডেন্টেশন → pub get → পূর্ণ রিস্টার্ট
Gradle task assembleDebug failed	flutter clean; minSdkVersion ≥ 21
SocketException	INTERNET পারমিশন; ইমুলেটরে 10.0.2.2
BoxDecoration with color	রং BoxDecoration-এর ভিতরে দিন
Instance of 'Future<X>'	<code>await</code> লিখতে ভুলে গেছেন
Startup lock	Flutter SDK-র <code>bin/cache/lockfile</code> মুছুন
Multiple heroes with same tag	লিস্টে অনন্য tag (id) ব্যবহার করুন
Duplicate GlobalKey	একই GlobalKey দুই জায়গায় ব্যবহার করবেন না

পরিশিষ্ট গ — প্যাকেজ ও রিসোর্স

প্যাকেজ	কাজ
http / dio	API কল
provider / flutter_riverpod / flutter_bloc	স্টেট ম্যানেজমেন্ট
go_router	নেভিগেশন ও deep link
shared_preferences / flutter_secure_storage	ছোট ডেটা / গোপন ডেটা
sqflite / hive / isar	লোকাল ডেটাবেজ
cached_network_image	ছবি ক্যাশ
image_picker / geolocator / permission_handler	ডিভাইস ফিচার
intl	তারিখ, সংখ্যা, ভাষা
fl_chart	চার্ট ও গ্রাফ
google_fonts / flutter_svg / lottie	ফন্ট, SVG, অ্যানিমেশন
url_launcher / share_plus / connectivity_plus	লিংক, শেয়ার, নেট
firebase_core / auth / firestore / messaging	Firebase
get_it	Dependency injection
mocktail / flutter_lints	টেস্ট ও কোড মান

শেখার রিসোর্স

- **docs.flutter.dev** — অফিসিয়াল ডকুমেন্টেশন; যেকোনো প্রশ্নের প্রথম গন্তব্য।
- **api.flutter.dev** — প্রতিটি widget-এর সম্পূর্ণ রেফারেন্স।
- **pub.dev** — প্যাকেজ; Likes, popularity ও শেষ আপডেট দেখে বাছুন।

- **dart.dev/guides** — Dart ভাষার গভীর গাইড ও effective Dart।
- **YouTube: Flutter** — "Widget of the Week" ও "Decoding Flutter" সিরিজ।
- **Flutter Bangladesh** (Facebook) — বাংলায় সাহায্য ও চাকরির খবর।
- **Stack Overflow** — এরর মেসেজ হুবহু কপি করে সার্চ দিন।

শেষ কথা

এই বইয়ের সবকিছু একবারে মনে রাখার চেষ্টা করবেন না — কেউ পারে না, দরকারও নেই। পেশাদার ডেভেলপাররাও প্রতিদিন ডকুমেন্টেশন দেখে। যা দরকার তা হলো **ধারণাগুলো বোঝা** — Widget কী, constraint কীভাবে কাজ করে, state কোথায় রাখতে হয়, async কেন লাগে। বাকিটা রেফারেন্স।

দিনে ১ ঘণ্টা কোড লিখুন। ৩ মাস পরে আপনি নিজেই অবাক হবেন কতদূর এসেছেন।

শুভকামনা রইল! 🚀